Week 13 — Virtual Memory, Faults, and Reclaim

Format: seven daily sections collected into one weekly module

Suggested workload: approximately 21 focused hours, adjustable to mastery

> A virtual address is not a physical frame. A VMA says whether the address is mapped and how. A page-table entry, cached in the TLB, says which frame currently answers it. A fault fills that entry. Reclaim later takes a frame back.

The lab is projects/linux-vm. Wrappers build through its Makefile and print only the claim the run supports. Kernel names below were read from Linux 6.12, commit `adc218676eef25575469234709c2d87185ca223a`, whose top-level `Makefile` sets `VERSION = 6`, `PATCHLEVEL = 12`, `SUBLEVEL = 0`. Those names are that snapshot. The host that runs the lab may be another kernel; the wrappers print `uname`. Re-open the pinned tree, or the sources for the kernel you boot, before treating a symbol, field, or policy as current.

Required order: syntax, Linux 0.12, evolution, ToyOS, then 6.12

Each day opens in this order, and the order is required:

1. the few C forms that day's source actually uses;
2. the Linux 0.12 path, read while the whole mechanism still fits in one function or one assembly entry;
3. why that early design changed, in the constraint the fixed code, data, brk, and stack fields, the flat `mem_map` scan, the missing `mmap`, or the single-task `oom()` could not keep;
4. the ToyOS reconstruction of paging from Weeks 7 and 8;
5. the Linux 6.12 tree `adc218676eef25575469234709c2d87185ca223a`, then a check against current upstream before a symbol is treated as current.

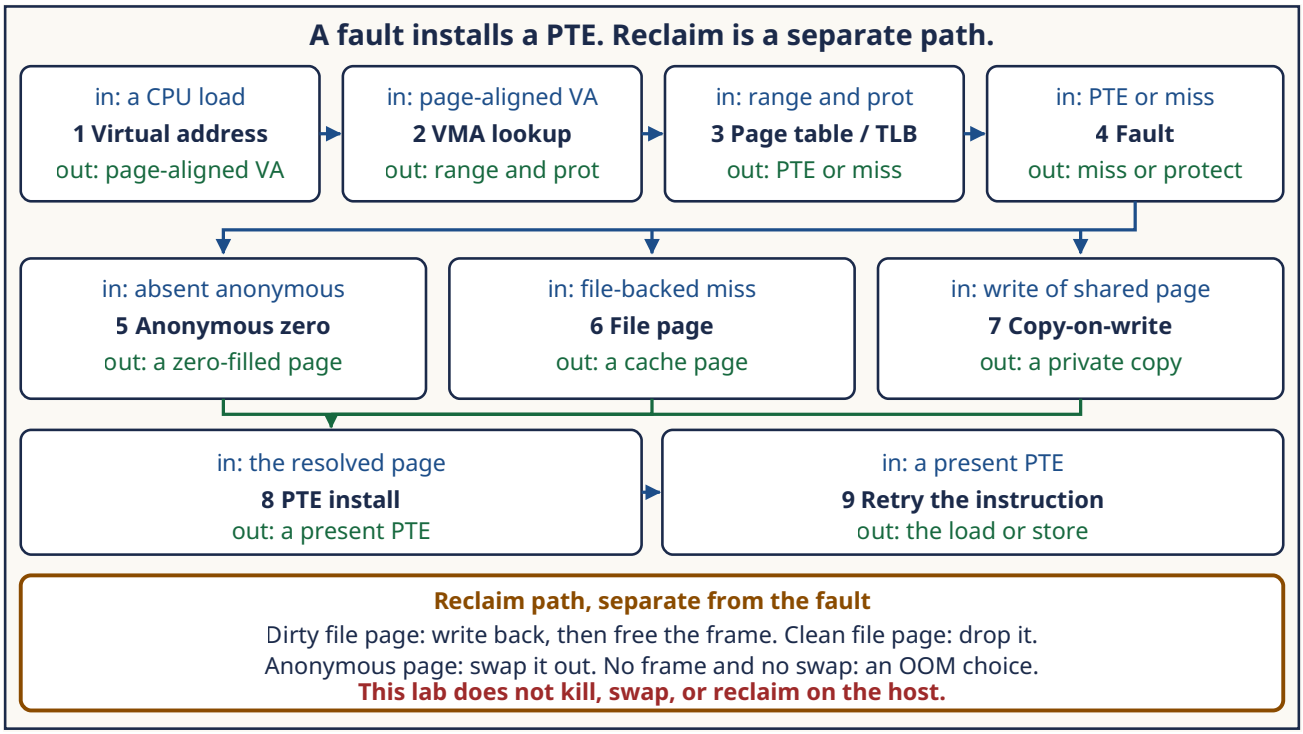
Linux 0.12 is the first conceptual implementation. Its 80386 two-level tables, segment slot, demand fault, and copy-on-write make the control flow visible. VMAs, `mmap`, `mprotect`, the buddy allocator, NUMA zones, folio reclaim, and today's OOM killer belong to the 6.12/current source.

The shared Linux 0.12 map indexes the early kernel and does not yet extract the memory files. Use the day sections for `mm/page.s`, `mm/memory.c`, and `kernel/sys.c`.

Why Week 13 follows I/O

Week 12 stopped at bytes. A successful `read` copies from a page-cache page. A `MAP_SHARED` store names that same page. An anonymous mapping has no file: the bytes are the process's own state. Both still have to become a mapped page before a load or store can complete.

```
virtual address
  which VMA, if any
        |
        v
page table and TLB
  a present PTE is a frame; a miss is a fault
        |
        +--> anonymous: zero-fill, or a private copy
        +--> file: the page cache, which may sleep for I/O
        |
        v
PTE installed, TLB refreshed, the instruction retries
        |
        v
later, and separately: reclaim
  clean file page dropped, dirty file page written back,
  anonymous page swapped, or an OOM choice
```



That picture is the week's map: the fault path above, reclaim below. They meet at the frame. One lab run does not walk the kernel reclaim code.

A file fault can sleep inside the page cache, the same way Week 12's blocking `read` slept. Reclaim can sleep in writeback. A hard interrupt handler must not take that sleep. Scheduler and interrupt depth need that fact first: a user fault is a place a task is allowed to sleep. This week builds the foundation. It does not start the scheduler deep dive.

Daily roadmap

Day 1 — Address spaces, VMAs, page tables, and the TLB

Day 1 separates the address space, the VMA, the page-table entry, and the TLB. Linux 0.12 had no VMA list: `start_code`, `end_code`, `end_data`, `brk`, and `start_stack` were the whole user map, placed by a segment base. ToyOS maps a page and invalidates it with `invlpg`, and

does not keep a VMA. Linux 6.12 looks the address up with `find_vma` in `lock_mm_and_find_vma` and enters from `do_user_addr_fault`. The lab reads the page size and finds one anonymous mapping in `/proc/self/maps` and `smaps`. Wrapper: challenges/day-001-address-space.sh.

Day 2 — Page faults, demand paging, and zero-fill

Day 2 is the not-present fault. Linux 0.12 `page_fault` calls `do_no_page`, which zero-fills past the executable image and reads the file image otherwise. ToyOS reports the fault and does not fill a page. Linux 6.12 `handle_mm_fault` reaches `do_anonymous_page`, which may install the shared zero page for a read or allocate a private folio for a write. The lab maps one anonymous page, checks that it is not resident, reads a zero, and prints a `getrusage` minor-fault delta with an explicit count caveat. Wrapper: challenges/day-002-page-faults.sh.

Day 3 — mmap, mprotect, COW, and shared mappings

Day 3 splits two sharing rules. `MAP_SHARED` means the child's store and the parent's mapping name one page. `MAP_PRIVATE` means a write copies. `mprotect` changes the VMA, so a write the VMA forbids is a signal, not a copy. Linux 0.12 already copied on write in `copy_page_tables` and `un_wp_page`, and had no `mmap` and no `mprotect`. The lab's child is the only process that faults; the parent decodes `wait` status. Wrapper: challenges/day-003-mappings.sh.

Day 4 — User allocators, brk, mmap, and lifetime

Day 4 separates the libc allocator from the two syscalls underneath it. Linux 0.12 `sys_brk` only moves `brk`; the page appears on the fault. Linux 6.12 `brk` calls `do_brk_flags`, and `do_mmap` builds a VMA. `munmap` drops that VMA. The lab records whether `sbrk` moved and which `/proc/self/maps` line contains a small and a large `malloc`. It does not treat either result as glibc's threshold. Wrapper: challenges/day-004-allocators.sh.

Day 5 — Physical pages, zones, the buddy allocator, and NUMA

Day 5 is where a fault's request for a frame goes. Linux 0.12 scans `mem_map` in `get_free_page`. A zone exists because a device may address only a low physical range, and a node exists because a frame has a distance. The buddy splits and coalesces power-of-two blocks. ToyOS uses a bitmap over a bounded window. The lab runs a deterministic simulator and does not allocate, migrate, or reserve host pages. Wrapper: challenges/day-005-buddy-numa.sh.

Day 6 — Reclaim, writeback, swap, and OOM

Day 6 asks which frame can be reused. A clean file page can be dropped. A dirty file page must be written back. An anonymous page needs a swap slot, or it cannot be taken. Linux 0.12 `oom()` exits the current task. Linux 6.12 `try_to_free_pages` drives `shrink_folio_list`, and `out_of_memory` selects a victim. The lab's LRU, writeback, swap, and OOM choice are a bounded simulator. It does not force reclaim, swap, or the OOM killer on the host. Wrapper: challenges/day-006-reclaim.sh.

Day 7 — VM observation and a debugging report

Day 7 keeps scopes apart. `/proc/self/maps` is this address space. `/proc/vmstat` is the whole machine. `perf stat` is one run's counters. ToyOS evidence for a fault is still the serial `EXPECTED-FAULT` line, not a reclaim scanner. The wrapper writes a report, reruns the deterministic suite, and records optional tools as pass or skip. Wrapper: challenges/day-007-vm-debug-report.sh.

Safety and evidence

The programs create temporary files only under `mkdtemp` directories in `/tmp`, unlink them, and remove the directories. The suite checks that those directories are gone. Children are waited for; a failing parent kills and reaps a child it still owns. Mappings are one page, except one bounded `malloc` of 2 MiB that is freed. The buddy and reclaim programs touch only their own arrays.

Nothing in the lab opens a socket, asks for root, writes a sysctl, or tries to allocate until the machine reclaims, swaps, or invokes the OOM killer. Optional `perf stat` and `/proc/vmstat` are observational. A missing tool prints `SKIP` and is not a failed core test. The minor-fault delta is a lower bound with a printed caveat, not a count of one C statement.

Completion checkpoint

Finishing the wrappers does not complete mastery. Answer from the source and the logs, and leave a question blank when the run did not speak to it:

1. Which object is the VMA, which word is the PTE, and what does the TLB cache?
2. What was resident before the first load of the anonymous page, what byte did that load return, and why is the minor-fault delta not an exact count of that load?
3. Which child store was visible in the parent, which was not, and what wait status did the `mprotect` child produce?
4. What did this run's `brk` and `maps` lines show, and which glibc threshold did you refuse to infer?
5. Which simulated allocation stayed on the local node, which fell back, and which request failed because the free pages were not one block of the requested order?
6. Which simulated victim was written back, which was dropped, which was swapped, and which OOM choice did not kill a host process?
7. Which numbers came from this process, and which optional counters were system-wide or a single `perf` run?
8. Which kernel names did you re-read in commit `adc218676eef25575469234709c2d87185ca223a`, and which host `uname` actually ran the lab?

Do not auto-fill mastery, confidence, hours, or completion. The quizzes sit beside this overview and do not grade the report.

Day 1 — Address Spaces, VMAs, Page Tables, and the TLB

Target time: approximately 3 hours

> The CPU issues a virtual address. Which object decides that the address is legal, and which word decides which frame answers it right now?

Every Linux 6.12 excerpt in this lesson is from tag `v6.12`, commit `adc218676eef25575469234709c2d87185ca223a`, checked out at `/home/jd/Documents/linux-6.12`. Linux 0.12 page numbers are printed pages of Zhao Jiong, *A Heavily Commented Linux Kernel Source Code*, Chinese Revision 5 (2019).

1. Syntax needed for this source

A range is two integers and a flags word. The page-table walk is a shift and a mask.

```
struct range {
    unsigned long start;
    unsigned long end;
    unsigned flags;
};

unsigned long index = (virtual_address >> shift) & mask;
unsigned long entry = table[index];
```

`start <= address < end` is the lookup. `entry` is not the byte the program asked for. It is the next table, or the frame, plus permission bits.

The half-open interval matters at a boundary. If one range ends at `0x5000` and the next starts there, `0x4fff` belongs to the first and `0x5000` belongs to the second. Length is simply `end - start`; adjacent ranges do not overlap.

Kernel flags are bit masks. `|` combines independent facts, `&` asks whether a fact is present, and `~` clears selected facts:

```
unsigned long flags = RANGE_READ | RANGE_WRITE;

if (flags & RANGE_WRITE)
    permit_store();
flags &= ~RANGE_WRITE;
```

Do not read `flags & RANGE_WRITE` as equality. Other bits may also be set. The same syntax describes VMA permissions and the present, writable, user, accessed, and dirty state encoded in page-table entries.

The x86 address split is shifts and masks. For a 4 KiB base page, the low 12 bits are the byte offset. On x86-64, the next four groups of 9 bits index the PTE, PMD, PUD, and PGD levels:

```
63                  48 47       39 38       30 29       21 20       12 11       0
+----------------------+-----------+-----------+-----------+-----------+-----------+
| sign extension       | PGD index | PUD index | PMD index | PTE index | page off  |
+----------------------+-----------+-----------+-----------+-----------+-----------+
```

That is an architecture example, not the definition of a VMA. A VMA can cover thousands of pages. Page tables can have missing entries inside its range. The VMA says an access is allowed in principle; the PTE says what translation exists now.

AT&T assembly used by early Linux writes source before destination:

```
movl %cr3,%eax
movl %eax,%cr3
```

Reloading `cr3` invalidates cached translations in Linux 0.12. Modern x86 has finer invalidation choices, including `invlpg` and range-oriented shootdown machinery. In both kernels, changing memory does not change a cached translation; software must perform the architecture's maintenance.

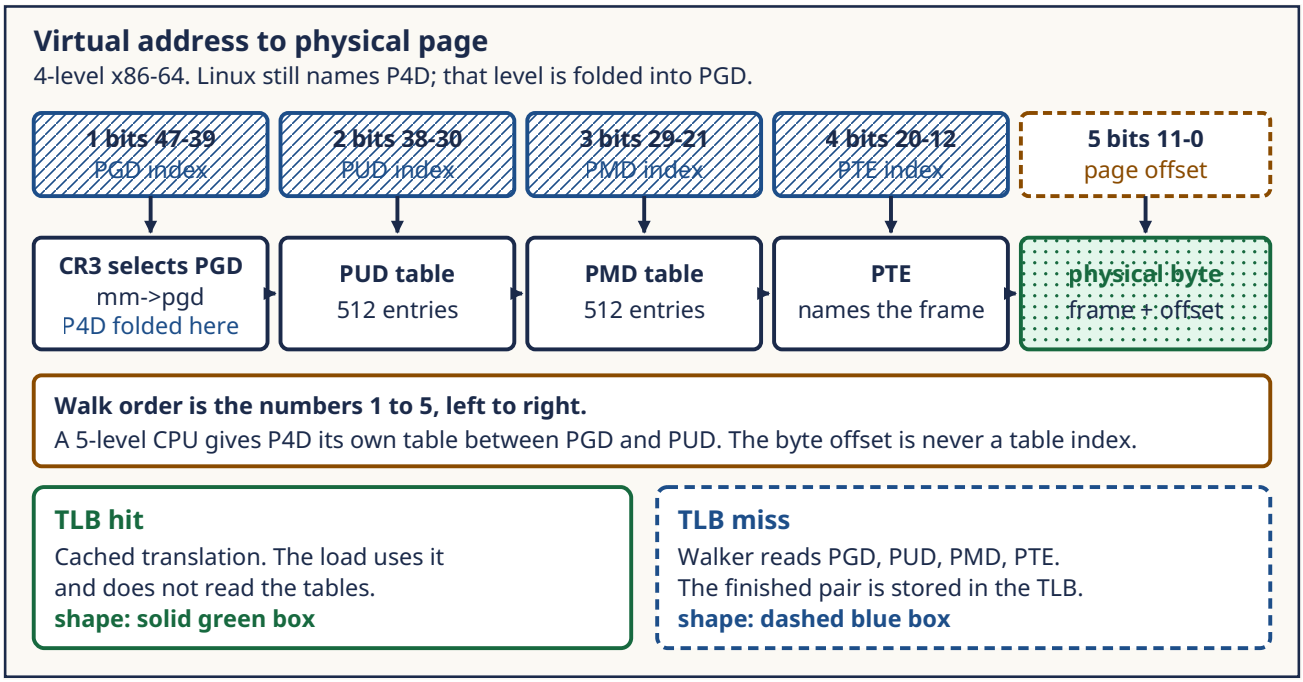


Figure 1. The range check is a software policy decision. The page-table walk is a hardware translation operation. A TLB hit can bypass the walk, but it cannot make an address legal when the kernel handles a fault.

2. Linux 0.12 first: a segment slot and two-level tables

Linux 0.12 does not have a VMA. `task_struct` holds `start_code`, `end_code`, `end_data`, `brk`, and `start_stack`. Each task occupies a `TASK_SIZE` window of 64MB, and the LDT segment base places that window at `nr * TASK_SIZE`. A user pointer is an offset inside the window.

The 80386 walks two levels, a page directory and a page table, over 4KB pages. `get_free_page` in `mm/page.s` scans the `mem_map` array and returns one zeroed physical page. `mem_map` counts references. It is not a list of virtual ranges.

What carries forward is the split between a virtual address and a physical page, plus permission bits in the page-table entry. The fixed task fields, the segment base, and the two-level 80386 tables belong to this kernel.

2.1 The address has two stages

On the 80386, a user instruction supplies an offset. The selector chooses an LDT descriptor, and the descriptor base is added to form a linear address. Paging translates that linear address to a physical address. Linux 0.12 uses this segmentation stage to give task `nr` a 64 MiB slot:

```
new_data_base = new_code_base = nr * TASK_SIZE;
p->start_code = new_code_base;
set_base(p->ldt[1], new_code_base);
set_base(p->ldt[2], new_data_base);
```

This excerpt is from `copy_mem` in `kernel/fork.c`, printed pages 407–408, and is also recorded in the shared Linux 0.12 map. `ldt[1]` is code and `ldt[2]` is data. Code and data receive the same base because this kernel does not support separate instruction and data spaces.

The stable mechanism is isolation: the same offset in two tasks reaches different linear-address slots and different page-table mappings. The historical mechanism is a fixed slot selected by segmentation. Modern Linux normally gives each process a page-table root and uses a mostly flat user segmentation model.

2.2 One page-directory page, then one page-table page

A 32-bit linear address is divided into 10 directory bits, 10 table bits, and 12 offset bits. Each directory or table has 1024 four-byte entries. Each page-table page therefore covers 1024 × 4 KiB = 4 MiB; one directory covers 1024 × 4 MiB = 4 GiB.

`copy_page_tables` receives linear bases and a byte size. It allocates child page tables as needed, copies entries, and makes ordinary shared pages read-only so a later write can copy. The page-table structure is therefore both a translation tree and a permission mechanism. The `mem_map` byte for a physical page counts how many mappings share it.

Those are different resources:

- an LDT descriptor describes a task's linear window;
- a page-directory entry owns or points to a page-table page;
- a page-table entry names a data frame and permission bits;
- a `mem_map` byte accounts for references to the data frame;
- the TLB caches a translation derived from those entries.

Freeing one resource does not imply that all the others disappeared. Week 7's frame bitmap asked only whether a frame was available. Week 10's fork path added sharing and reference counts. This week puts range policy, translation, and cached translation into separate columns.

2.3 `get_free_page` does not map

The early allocator searches `mem_map` for a zero byte, changes that byte to 1, zeroes the physical page, and returns its physical address. `put_page` walks or allocates page tables and installs that address in a PTE. Conflating the two hides an important failure boundary: a frame can exist before the page-table page needed to map it exists.

The early kernel has no object equivalent to a current `struct vm_area_struct`. Legality comes from task image fields, segment limits, and special-case fault logic. `start_code`, `end_code`, `end_data`, `brk`, and `start_stack` are enough for one executable image, one growing data area, and one stack. They are not a general range model.

2.4 What not to carry forward

Do not infer that current Linux gives PID 3 the third 64 MiB slot, that all page tables have two levels, or that a global byte array is current physical-page accounting. Keep only the durable questions:

1. Which address space does this execution context use?
2. Which region policy covers the address?
3. Which page-table entry currently translates it?
4. Could a processor still hold an older translation?

3. Evolution: why a list of ranges replaced those fields

Code, data, brk, and stack cannot name a shared library, a file mapping at an arbitrary offset, or a protection change on part of a mapping. The machine grew programs that needed many such ranges. The structure that answers "what is this address?" became a searchable set of ranges, each with protection and a backing. The page tables stayed the structure that answers "which frame, this instant?"

The pressure was not merely larger programs. One process needed several independently managed objects:

- an executable with read-only text and writable data;
- a dynamic linker and shared libraries at unrelated addresses;
- private and shared file mappings with page offsets;
- anonymous arenas independent of the historical break;
- thread stacks with guard gaps;
- device or special mappings with custom fault operations;
- protection changes on only part of an existing mapping.

A fixed `brk` field cannot answer which file and page offset backs address `x`. Splitting and merging ranges also matters. `mprotect` on the middle page of a three-page VMA may produce three VMAs: the unchanged prefix, the newly protected middle, and the unchanged suffix. Adjacent compatible ranges can later merge. The range set is mutable metadata, not a dump of the page tables.

Scalability changed the search structure. A linked list makes lookup linear in the number of mappings. Current Linux 6.12 stores VMAs in the mm's maple tree and uses a VMA iterator. That implementation may evolve again. The interface-level invariant is the searchable set of nonoverlap- ping half-open ranges.

Page tables also evolved independently. More virtual-address bits and larger physical memories required more levels. Huge mappings can stop a walk above the base-page PTE. Architectures encode permissions differently. None of those changes removes the need for the VMA. The page table answers what hardware can do now; the VMA lets the kernel decide what it may build after a fault.

The TLB exists because walking several memory-resident levels for every load would be expensive. That cache creates a coherence obligation. Changing a PTE, unmapping a range, or reducing permissions requires invalidating translations on every CPU that may have run this mm. A local `invlpg` is enough for ToyOS's single-CPU exercise. Linux needs batching, mm tracking, inter-processor interrupts, and architecture-specific rules.

4. ToyOS: a mapper, not a VMA list

Week 7's `map_page` in `projects/toyos/src/memory.c` walks four levels from `bootstrap_pml4` and stores a present PTE. The next instruction is `invlpg`, because the CPU may already have cached a translation. `frame_alloc` hands out one frame from a bitmap capped at 128MB. The heap is eight mapped pages at `HEAP_BASE`. Week 8's `map_page_in` repeats that walk in a per-process root so one user program has its own tables.

There is still no range list. An address is mapped or it is not. A fault is diagnosed in `arch.c`. It is not looked up in a VMA and filled.

Trace one ToyOS mapping:

1. `frame_alloc` scans the bitmap and marks a physical frame owned.
2. `map_page` starts at `bootstrap_pml4`.
3. Missing intermediate tables receive frames of their own.
4. The final entry receives the data frame plus present/write flags.
5. `invlpg` discards any translation cached for that virtual address.
6. A later load or store is translated by hardware.

This is deliberately below the VMA layer. A successful `map_page` says that a translation exists; it does not record whether a file, heap, stack, or shared-memory object owns the virtual range. `memory_init` maps eight heap pages up front. `kalloc` moves a bump pointer inside them, so an allocation neither creates a VMA nor relies on a first-touch fault.

Week 8's per-process root adds an ownership boundary: the process has a different page-table root from the kernel's bootstrap use. It still does not add fork, COW, an interval search structure, automatic stack growth, or a sleepable file-fault callback. Those omissions are the comparison, not defects to paper over.

If you extend ToyOS, add the layers in dependency order. First define a range record and nonoverlap invariant. Then make the fault handler find that record. Then allocate and map an anonymous frame only when its flags permit the access. Finally add unmap and TLB invalidation. Starting with "allocate on every fault" turns a wild pointer into valid memory and removes the protection a VMA is meant to provide.

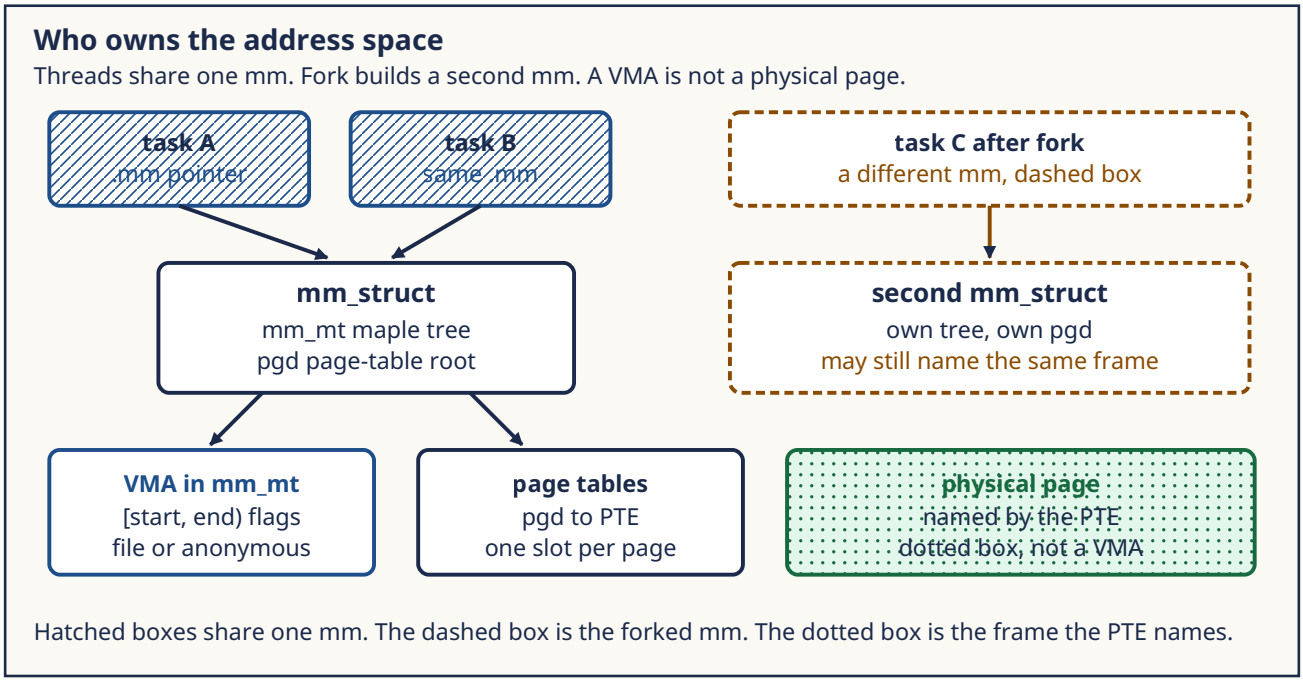


Figure 2. Linux 0.12 derives ownership from a task slot and fixed image fields. ToyOS has explicit page-table roots and mappings. Linux 6.12 adds an `mm_struct` with a searchable VMA set above its page tables.

5. Linux 6.12: VMA, then page table, then TLB

In commit `adc218676eef25575469234709c2d87185ca223a`, `struct vm_area_struct` in `include/linux/mm_types.h` carries `vm_start`, `vm_end`, `vm_mm`, `vm_page_prot`, and `vm_flags`. The comment in that tree says the VMA covers `[vm_start, vm_end)`. `lock_mm_and_find_vma` in `mm/memory.c` calls `find_vma`. `arch/x86/mm/fault.c` `do_user_addr_fault` is the x86 entry that reaches `handle_mm_fault`.

The hardware page walker fills the TLB from the tables. After software changes a PTE it must invalidate, which is why `do_wp_page` can call `flush_tlb_page`. A stale TLB entry is still a translation.

`/proc/self/maps` is an export of those VMAs. It is not the page tables and it is not a physical address.

5.1 The VMA record

The definition ties the interval, owner, permissions, and flags together:

```
/* tree adc218676eef (Linux 6.12)
 * path: include/linux/mm_types.h
 * lines: 667–691
 */
struct vm_area_struct {
    union {
        struct {
            /* VMA covers [vm_start; vm_end) addresses within mm */
            unsigned long vm_start;
            unsigned long vm_end;
        };
#ifdef CONFIG_PER_VMA_LOCK
        struct rcu_head vm_rcu;
#endif
    };
    struct mm_struct *vm_mm;
    pgprot_t vm_page_prot;
```

The RCU member is conditional on `CONFIG_PER_VMA_LOCK`. `vm_mm` owns the VMA. `vm_page_prot` is the architecture protection derived for PTEs. `vm_flags` includes generic policy such as read, write, execute, shared, and grow-down.

Later fields connect file-backed mappings to an `address_space`, private pages to an `anon_vma`, and device or file behavior to `vm_operations`. That is why a VMA is more than permissions. It records where a fault can obtain bytes and how mappings of those bytes relate.

5.2 Lookup proves policy before allocation

The locked fallback lookup in `mm/memory.c` is short:

```
/* tree adc218676eef (Linux 6.12)
 * path: mm/memory.c
 * lines: 6169–6188
 * function: lock_mm_and_find_vma
 */
vma = find_vma(mm, addr);
if (likely(vma && (vma->vm_start <= addr)))
    return vma;

if (!vma || !(vma->vm_flags & VM_GROWSDOWN)) {
    mmap_read_unlock(mm);
    return NULL;
}
```

`find_vma` can return the first VMA after `addr`; the start comparison is therefore essential. Only a grow-down candidate gets an extension attempt. A hole is not silently turned into anonymous memory. On x86 the caller next checks `access_error(error_code, vma)` before `handle_mm_fault`.

Linux 6.12 also has a per-VMA-lock fast path: `lock_vma_under_rcu` is tried for user faults before the mmap-lock path. That is a scalability optimization around the same policy object, not a replacement for VMA lookup.

5.3 Page tables are populated below `handle_mm_fault`

`handle_mm_fault` first validates architecture permissions:

```
/* tree adc218676eef (Linux 6.12)
 * path: mm/memory.c
 * lines: 6052–6078
 * function: handle_mm_fault
 */
ret = sanitize_fault_flags(vma, &flags);
if (ret)
    goto out;

if (!arch_vma_access_permitted(vma, flags & FAULT_FLAG_WRITE,
                    flags & FAULT_FLAG_INSTRUCTION,
                    flags & FAULT_FLAG_REMOTE)) {
    ret = VM_FAULT_SIGSEGV;
    goto out;
}
```

Later in the same function, the generic walk allocates missing intermediate page-table levels and eventually dispatches on the current entry: anonymous missing page, file fault, swap entry, NUMA hint, or write protection. Day 2 follows those branches. The important boundary today is that the page-table work begins only after a covering VMA and a permitted access exist.

5.4 The TLB is a derived cache

Suppose CPU 0 has cached virtual page `V` → frame `A`. CPU 1 changes the PTE to frame `B` while CPU 0 keeps running the same mm. Updating memory is not enough; CPU 0 can continue sending accesses to `A`. The kernel must order the PTE change and invalidate CPU 0's cached entry before old ownership can be reused.

Linux gathers invalidations so that unmapping 10,000 pages does not necessarily send 10,000 separate cross-CPU requests. The exact primitive depends on architecture and range. The contract is stronger than "execute `invlpg` after every PTE store": no processor may legally retain a translation that violates the completed mapping operation.

Permission upgrades and downgrades have different risks. Leaving a stale read-only entry after an upgrade causes an extra fault and can be repaired. Leaving a stale writable entry after a downgrade violates protection. The architecture APIs encode the ordering; generic mm code does not open- code a single universal instruction.

5.5 `/proc` reports policy and accounting

A typical line is:

```
7f...-7f... rw-p 00000000 00:00 0
```

The endpoints and permissions describe a VMA. The offset, device, inode, and optional name describe its backing or presentation. The line contains no PFN. `/proc/self/smaps` adds accounting such as `Size` and `Rss`, but that still does not expose a hardware TLB hit or prove one physical frame.

ASLR means addresses differ between runs. VMA merging means the interval can be larger than the one-page request if an adjacent compatible mapping is merged. The project lab checks that its returned address lies in the reported range and, in its controlled run, requires that range to be one page. Treat that requirement as a property of this lab setup, not a new `mmap` ABI guarantee.

6. What the lab can show

`address_lab` prints the page size, maps one anonymous page, and finds that range in `/proc/self/maps`. `smaps` Size matches the page. Rss is 0 before any touch. `[stack]` is present. `[heap]` may be present or absent; the program prints which, and refuses to treat that as a libc contract.

Run day-001-address-space.sh.

6.1 Predict, run, and label

Before running, predict three separate states:

Question	Prediction before touch
Does a VMA cover the returned pointer?	yes
Does `smaps` charge virtual `Size`?	one page in this lab
Must a private resident frame exist?	no

Then run only the existing wrapper:

```
./13-virtual-memory-faults-and-reclaim/challenges/day-001-address-space.sh
```

The wrapper builds and runs `projects/linux-vm/src/address_lab.c`. It does not inspect `/proc/pid/pagemap`, load a kernel module, pin memory, change sysctls, or touch another process. The mapping is anonymous and is `munmap`ped before exit.

6.2 Observation versus source proof

The lab observes: the runtime page size; a covering `maps` range; the range's writable text permission; `smaps` `Size` and `Rss`; a `[stack]` line; and whether this libc run happens to show `[heap]`.

The pinned source proves: what fields a VMA contains; that the locked fault fallback calls `find_vma`; that x86 checks access before `handle_mm_fault`; and that generic fault handling descends into page- table resolution.

The lab does not execute a source-level trace. Seeing `Rss: 0 kB` is consistent with an untouched anonymous mapping. It does not prove which conditional branch in `do_anonymous_page` would run on a future access. Conversely, reading `do_anonymous_page` proves an implementation path in the pinned tree; it does not prove the host kernel is that tree. Record `uname -r` beside observations.

7. Execution trace: one load, layer by layer

Assume a program loads from an anonymous page after `mmap` created a VMA but before a PTE was installed.

1. The compiler emits a load through a virtual address.
2. The CPU searches its TLB. There is no usable translation.
3. Hardware walks the current page tables. The leaf is not present.
4. x86 raises vector 14 and records the address in `CR2`.
5. `do_user_addr_fault` derives read/write/instruction/user flags.
6. The VMA fast path or `lock_mm_and_find_vma` finds the covering range.
7. `access_error` rejects an access that the VMA does not permit.
8. `handle_mm_fault` validates architecture-specific permission.
9. Generic mm descends the page-table hierarchy.
10. Anonymous fault handling chooses zero-page or private-page behavior.
11. The PTE is installed with the VMA-derived protection.
12. Return from the exception retries the original load.
13. Hardware now fills a TLB entry from the page table.
14. The load finally returns a byte.

Steps 6–8 answer "is this access legal?" Steps 9–11 answer "what mapping should satisfy it?" Steps 2, 3, and 13 are hardware translation. Keeping those questions separate prevents the common claim that a VMA translates every load.

8. Architecture lens

x86-64 commonly uses four or five page-table levels and 4 KiB base pages. AArch64 supports translation granules including 4 KiB, 16 KiB, and 64 KiB, with level geometry selected accordingly. RISC-V uses modes such as Sv39, Sv48, and Sv57. `struct vm_area_struct` remains a range policy object on all of them. The shifts, entry bits, and invalidation instructions do not.

ToyOS's four-level x86-64 walker is therefore one concrete implementation, not the definition of virtual memory. A lesson that hard-codes "bits 47–39 are always the top index" has confused one x86-64 mode with the generic Linux model.

9. Mastery: Explain · Draw · Observe · Build

Explain

- Why a valid VMA can have no present PTE for an address.
- Why a present PTE without a covering legal range cannot authorize the kernel to grow arbitrary user memory.
- Why `mem_map` in Linux 0.12 is not a VMA collection.
- Why changing a PTE requires TLB maintenance.
- Why `[heap]` presence is not a libc or kernel guarantee.

Draw

Redraw Figure 1. Add the fault address, the half-open VMA endpoints, the leaf PTE, one physical frame, and a TLB entry. Mark which arrows are software policy, hardware translation, and accounting.

Observe

Run the wrapper once. Save `uname -r`, `page_size`, the anonymous range, `Size`, `Rss`, and the heap-presence line. Write "host observation" beside each. Beside the three source excerpts, write the pinned commit and "source proof."

Build

Without changing the project, write pseudocode for a ToyOS `find_region(address)` over sorted nonoverlapping records. State its lock, its half-open comparison, and its answer for a hole. Then describe where `map_page` would be called after a permitted anonymous fault. Do not make the pseudocode allocate for an address that has no region.

10. Limits

The lab does not print a PTE, a frame number, or a TLB hit. Quote `find_vma` and `do_user_addr_fault` only from tree `adc218676eef` or from the sources of the kernel you boot. Record `uname`.

It also does not prove that the host uses four page-table levels, that the anonymous mapping owns a private frame, or that a particular CPU took a TLB miss. Those require architecture and tracing evidence outside this bounded lab. Keep the report at the layer the evidence supports.

Now check your understanding

You have finished the theory for this section. Before moving on, use the quick check below to find anything that still feels uncertain, then reinforce the idea with the practical lab when one is provided.

Quick theory check

Answer from memory first. Explanations appear only after you submit your choices.

- Start the interactive theory check
- View quiz source on GitHub

Put it into practice — Address Space

Format: shell lab

What you will do: set -euo pipefail

Predict the important result first, then run the lab and explain any difference between prediction and observation.

- Open the lab file

Copy and run:

```
bash 13-virtual-memory-faults-and-reclaim/challenges/day-001-address-space.sh
```

- View lab source on GitHub

Day 2 — Page Faults, Demand Paging, and Zero-Fill

Target time: approximately 3 hours

> The page is not resident. The load still has to return a byte. Who fills it, and who is allowed to sleep while that happens?

Every Linux 6.12 excerpt below is from tag `v6.12`, commit `adc218676eef25575469234709c2d87185ca223a`, at `/home/jd/Documents/linux-6.12`. Linux 0.12 page numbers are printed pages of Zhao Jiong, *A Heavily Commented Linux Kernel Source Code*, Chinese Revision 5 (2019).

1. Syntax needed for this source

The fault entry receives an error code and a fault address. A bit in the code selects the handler. The handler returns by installing a PTE and retrying the instruction, or by killing the access.

```
if ((error_code & present_bit) == 0)
    fill_missing(address);
else
    handle_protection(address);
```

A zero fill is a store of `0` into a fresh frame, or a PTE that points at a shared page of zeroes. The program's first load cannot observe an old owner's bytes.

The x86 error code is a bit set, not an enum. In modern names, `X86_PF_PROT` says the access violated protection rather than finding a non-present translation, `X86_PF_WRITE` says the access was a write, and `X86_PF_USER` records the privilege context reported by hardware. Newer x86 adds bits for reserved-entry errors, instruction fetch, protection keys, shadow stacks, and SGX. Always mask the bit you need:

```
bool was_write = error_code & X86_PF_WRITE;
bool was_present = error_code & X86_PF_PROT;
```

The fault address and instruction pointer are different values. `CR2` holds the linear address that could not be translated. The saved instruction pointer identifies the instruction to retry or report. A single instruction can fault on an address other than its own address.

The return value is also a mask. `vm_fault_t` can carry `VM_FAULT_MAJOR`, `VM_FAULT_RETRY`, `VM_FAULT_OOM`, `VM_FAULT_SIGBUS`, or `VM_FAULT_SIGSEGV`. Code tests a bit; it does not compare the whole word with one constant.

Demand paging depends on restart. The architecture preserves enough state that, after software installs a valid mapping, exception return repeats the access as though the missing translation had been there. Side effects before the fault follow the ISA's restart rules. Kernel C does not manually increment the user instruction pointer past a successful page fault.

Zero-fill is a security rule before it is an optimization. A newly observable anonymous byte must not contain a prior owner's data. The kernel can satisfy that invariant by allocating and clearing a frame immediately, or by mapping a global read-only zero page for reads and allocating on the first write.

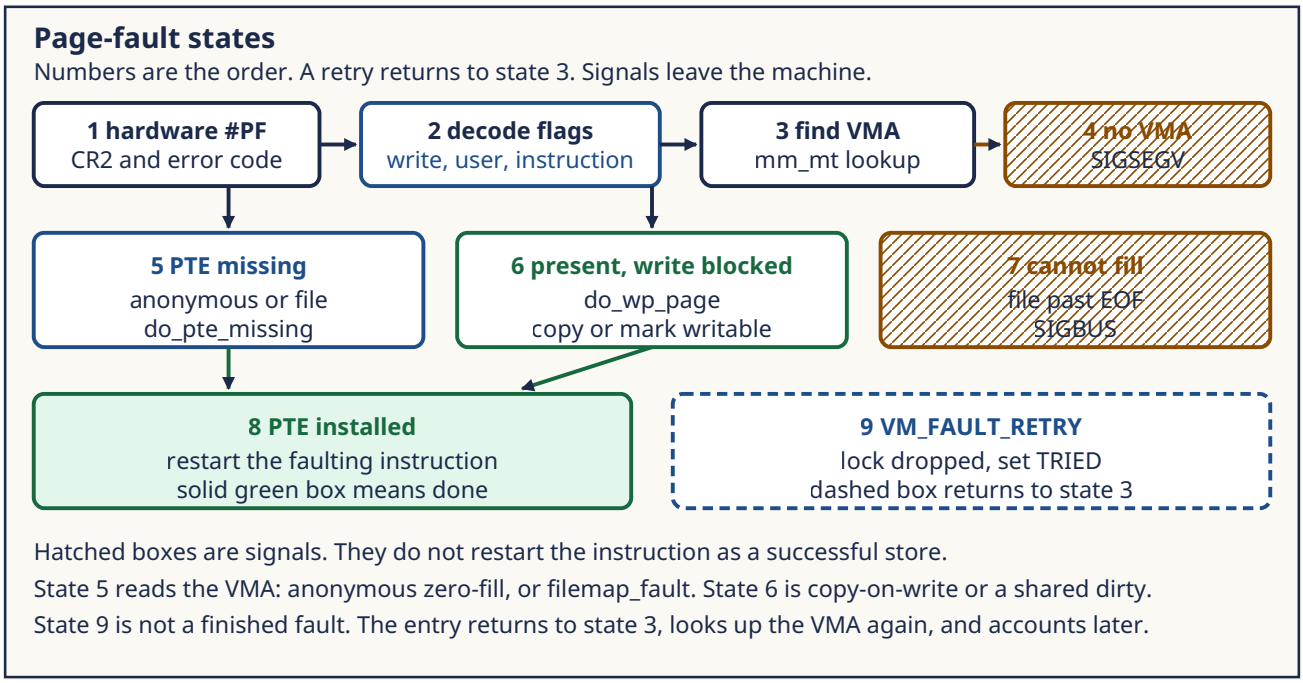


Figure 1. A fault is an exception, not necessarily an error. The resolver either establishes the mapping and retries, waits and retries, or converts the access into a signal or kernel failure.

2. Linux 0.12 first: do_no_page

`mm/page.s` `page_fault` is interrupt 14. It reads `cr2` and tests the error code. A not-present fault calls `do_no_page`. A protection fault calls `do_wp_page`.

`do_no_page` in `mm/memory.c` aligns the address and subtracts `start_code`. Past the executable image, the early path calls `get_empty_page`: a zeroed frame becomes the stack, the bss, or the heap. Inside the image, `share_page` may reuse a page another task already loaded from the same inode. Otherwise `get_free_page` and `bread_page` read the blocks, the tail past `end_data` is zeroed, and `put_page` installs the PTE. Failure calls `oom()`.

The durable idea is demand paging. The frame appears on the fault, not at `brk` time. The 80386 error-code test, the executable-inode read, and `oom()` killing the current task belong to this kernel.

2.1 Entry at vector 14

`mm/page.s`, printed pages 878–879, installs `page_fault` for interrupt 14. The processor has already pushed an error code. The assembly saves registers, reads `cr2`, pushes arguments for C, and uses bit 0 to choose the path:

```
movl %cr2,%edx
pushl %edx
testl $1,%eax
jne 1f
call do_no_page
jmp 2f
1:  call do_wp_page
```

The exact register shuffling around this excerpt is historical 80386 assembly. The stable mechanism is dispatch on hardware fault state: non-present faults can be filled; present write-protection faults may be COW; other invalid accesses must not be repaired as anonymous memory.

Early `invalidate` reloads `cr3`, flushing the 80386 translation cache after a PTE change. That whole-TLB operation is historical. The need to make cached translations agree with changed tables carries forward.

2.2 The complete missing-page decision

`do_no_page` begins on printed page 896. The compact branch from printed page 898 is:

```
if (!inode) {
    get_empty_page(address);
    return;
}
if (share_page(inode,tmp))
    return;
if (!(page = get_free_page()))
    oom();
for (i=0 ; i<4 ; block++,i++)
    nr[i] = bmap(inode,block);
bread_page(page,inode->i_dev,nr);
```

Before this branch, `address` is page-aligned and `tmp` is derived from the address minus `current->start_code`. The executable and one library window select an inode and offset. If neither applies, `inode` remains NULL and `get_empty_page` obtains a zeroed frame and maps it.

If an executable page is eligible to share, `share_page` searches another task with the same inode and makes both mappings read-only. Otherwise the kernel allocates a page, maps four 1 KiB filesystem blocks with `bmap`, and reads them through `bread_page`. For the last data page, bytes beyond `end_data` are cleared before `put_page` installs the PTE.

The order matters:

1. identify what the fixed image layout says should back the address;
2. try to reuse an existing clean executable page;
3. allocate a frame only if reuse failed;
4. read or zero the bytes;
5. publish the PTE;
6. return so the instruction can retry.

Publishing before filling would let another access observe uninitialized or partially read bytes. Current code uses locks and memory barriers to enforce the same "contents before reachable mapping" requirement.

2.3 `get_empty_page` separates allocation from mapping

`get_empty_page` calls `get_free_page`, then `put_page`. The first returns a zeroed physical page. The second creates page-table structure and maps that page at the linear address. If mapping fails, the physical page must be freed. Week 7's ToyOS split between `frame_alloc` and `map_page` is the same resource relationship.

Moving `brk` does not call either function. It changes the legal edge. Only a later access reaches `do_no_page`. That is the early form of virtual commit separated from physical allocation.

2.4 Early limits

Linux 0.12 has no VMA that can point at any file and arbitrary page offset. Its executable demand paging knows `current->executable`, `a.out` layout, four 1 KiB blocks per 4 KiB page, and a fixed library region. A missing page past those image regions is treated as empty memory.

Do not generalize the behavior "a non-present user address gets a zeroed page." In current Linux the address first needs a covering VMA, appropriate permissions, and a backing policy. A hole receives `SIGSEGV`. A file page past end can receive `SIGBUS`. A swap entry requires swap-in. The early kernel is valuable because the whole path fits on a page, not because all modern causes fit its branch.

3. Evolution: more reasons to miss

A not-present PTE can mean "zero-fill," "read this file offset," or "the page is on swap." A present but unwritable PTE can mean copy-on-write, or a protection the program is not allowed to override. Major and minor are accounting names for "this fault waited for I/O" and "this fault did not." They are not a single function's private counter.

A file fault may sleep while the page cache reads. That sleep is the same kind of wait as Week 12's blocking `read`. A hard interrupt handler still must not take it. User faults are a sleepable path. That fact is the foundation later scheduler and interrupt work needs.

The design evolved because "not present" stopped meaning one thing. A non-present architecture entry can encode or lead to:

- an untouched anonymous VMA;
- a file offset not currently mapped;
- an anonymous page stored in swap;
- a migration entry while a page moves;
- a device-private or special mapping;
- userfaultfd delegation to a userspace handler;
- a transparent huge-page opportunity or fallback;
- a guard or inaccessible region that must fail.

The VMA narrows those possibilities before the PTE is interpreted. Its backing and operations tell generic fault code whether this is anonymous, file-backed, hugetlb, PFN-mapped, or special.

Zero-page sharing evolved to avoid allocating a private physical page for memory that has only been read and therefore still contains all zeroes. One read-only frame can answer many processes. A write cannot modify that global frame. It takes the allocation path and installs a private writable folio. This is COW from an implicit all-zero source.

File mappings evolved around the page cache. `read` and `mmap` must not maintain unrelated copies of the same file bytes. A file fault looks in the file's `address_space`; a hit can install the cached folio without storage I/O. A miss can start I/O and sleep. The VMA's page offset turns the virtual page into a file index.

Swap adds another identity. Reclaim can replace a present anonymous PTE with a non-present swap entry. A later fault reads the swap entry, finds or reads the folio, and restores a present mapping. The virtual address did not become a new anonymous page; its prior bytes had to survive.

Concurrency adds retry. A file fault can drop `mmap_lock` while waiting, because holding it across slow I/O would block unrelated mapping changes. On return, the VMA may have changed or disappeared. `VM_FAULT_RETRY` tells the architecture path to look up the range and validate the access again.

Major and minor are outcomes, not synonymous with file and anonymous. A page-cache hit for a file mapping can be minor. A file cache miss that requires I/O can be major. Anonymous first touch is normally minor, but an anonymous page swapped out and read back can be major. The pinned tree's fault accounting tests `VM_FAULT_MAJOR` or a tried/retry state; the lab's counter is the resulting process accounting.

Sleeping is legal because a user page fault runs in process context with a current task that can block. `do_user_addr_fault` explicitly rejects the normal fault path if fault handling is disabled or there is no `mm`. Interrupt context cannot wait for file I/O, memory reclaim that sleeps, or a userspace userfaultfd handler. Week 14 will distinguish exception entry from hard IRQ context; today the resource point is that demand paging needs a schedulable task.

4. ToyOS: the fault stops

`arch.c` names vector 14 "page fault" and can print `EXPECTED-FAULT` for an injected unmapped write. `kernel.c` writes an unmapped address on purpose. Nothing in that path allocates a frame and returns to the instruction. Week 7's mapper installs pages before they are used. Demand zero-fill is a Linux behavior you are about to measure, not a ToyOS behavior you already have.

ToyOS nevertheless supplies useful prerequisites:

- an IDT entry names vector 14;
- the architecture handler can read and print fault state;
- `frame_alloc` can provide a zeroed frame;
- `map_page` can create intermediate tables and a leaf PTE;
- `invlpg` can invalidate one cached translation.

What is missing is the policy chain that makes recovery safe. There is no sorted range set, no access check against region flags, no backing object, no fault lock, and no return convention that distinguishes retry from fatal failure. Its injected write intentionally demonstrates diagnosis. The expected outcome is the serial `EXPECTED-FAULT` marker, not a resumed store.

A minimal demand-zero extension would need:

1. an anonymous region record created before access;
2. a lookup proving the fault address lies inside it;
3. a check that the access type is permitted;
4. one zeroed frame from `frame_alloc`;
5. a mapping in the faulting process's page-table root;
6. rollback if a page-table allocation fails;
7. exception return that retries the instruction.

Only step 4 is allocation. A wild store outside every region must remain fatal. This is why the VMA lesson came first.

5. Linux 6.12: handle_mm_fault and the anonymous page

In commit `adc218676eef`, `do_user_addr_fault` calls `handle_mm_fault`. `handle_mm_fault` calls `__handle_mm_fault` unless the VMA is hugetlb. For an anonymous miss, `do_anonymous_page` in `mm/memory.c` has two shapes. A read may install the shared zero page through `my_zero_pfn`. A write allocates with `alloc_anon_folio` and marks the folio up to date before `set_pte_at`. A file miss goes to `do_fault`, and `filemap_fault` in `mm/filemap.c` is the page-cache side, which can sleep.

Rss after a pure read can stay 0 when the PTE names the shared zero page. Rss after a store counts the private page. The lab prints the read-side Rss and does not assert it.

5.1 x86 translates hardware state into generic flags

`do_user_addr_fault` in `arch/x86/mm/fault.c` turns architecture bits into generic flags:

```
/* tree adc218676eef (Linux 6.12)
 * path: arch/x86/mm/fault.c
 * lines: 1285–1306
 * function: do_user_addr_fault
 */
if (error_code & X86_PF_SHSTK)
    flags |= FAULT_FLAG_WRITE;
if (error_code & X86_PF_WRITE)
    flags |= FAULT_FLAG_WRITE;
if (error_code & X86_PF_INSTR)
    flags |= FAULT_FLAG_INSTRUCTION;

if (user_mode(regs))
    flags |= FAULT_FLAG_USER;
```

The user flag comes from saved register state, not blindly from the hardware error-code user bit. This is an x86 entry concern. Generic mm receives `FAULT_FLAG_*` and need not decode x86 bits.

The function first tries `lock_vma_under_rcu`; the fallback calls `lock_mm_and_find_vma`. Both paths run `access_error` before `handle_mm_fault`. A missing range or forbidden access is rejected before anonymous allocation.

5.2 Missing page-table levels are resources too

`__handle_mm_fault` initializes a `vm_fault` and allocates upper levels:

```
/* tree adc218676eef (Linux 6.12)
 * path: mm/memory.c
 * lines: 5816–5841
 * function: __handle_mm_fault
 */
struct vm_fault vmf = {
    .vma = vma,
    .address = address & PAGE_MASK,
    .real_address = address,
    .flags = flags,
    .pgoff = linear_page_index(vma, address),
    .gfp_mask = __get_fault_gfp_mask(vma),
};
```

Immediately after this initializer the function obtains or allocates upper page-table levels. The data folio is not the only possible allocation. A sparse address space may need page-table pages before it can install one leaf. Folded levels on some architectures make a helper cheap without removing the generic level from source.

`real_address` preserves the original byte address; `address` is page aligned. `pgoff` is the backing-object page index. Those three values answer different questions and should not be used interchangeably.

5.3 Read miss: map the zero page

The anonymous read branch is explicit:

```
/* tree adc218676eef (Linux 6.12)
 * path: mm/memory.c
 * lines: 4755–4761
 * function: do_anonymous_page
 */
/* Use the zero-page for reads */
if (!(vmf->flags & FAULT_FLAG_WRITE) &&
        !mm_forbids_zeropage(vma->vm_mm)) {
    entry = pte_mkspecial(pfn_pte(my_zero_pfn(vmf->address),
                    vma->vm_page_prot));
```

The optimization is conditional. An architecture or mm can forbid it. The PTE is special and uses the VMA's protection. This excerpt proves why the lesson does not require read-side RSS to equal one page.

5.4 Write miss: allocate before publishing

The write path allocates a private folio:

```
/* tree adc218676eef (Linux 6.12)
 * path: mm/memory.c
 * lines: 4779–4805
 * function: do_anonymous_page
 */
/* Allocate our own private page. */
ret = vmf_anon_prepare(vmf);
if (ret)
    return ret;
folio = alloc_anon_folio(vmf);
if (IS_ERR(folio))
    return 0;
if (!folio)
    goto oom;
```

The function next derives folio size and aligned address, marks the folio uptodate, then builds the PTE. The full function adds reverse mapping, LRU membership, mm accounting, and the PTE under its lock. The barrier documented inside `__folio_mark_uptodate` orders initialized contents before `set_pte_at`. Only after publication may the retried store observe the page.

`alloc_anon_folio` may return more than one base page when multi-size THP policy permits. Therefore "one anonymous fault always allocates exactly one 4 KiB `struct page`" is not a Linux 6.12 invariant. The lab requests one base-page-sized mapping and observes its accounting; it does not prove the allocator's internal order.

5.5 File miss: cache hit or I/O

`filemap_fault` starts with the file's mapping and page index:

```
/* tree adc218676eef (Linux 6.12)
 * path: mm/filemap.c
 * lines: 3301–3344
 * function: filemap_fault
 */
max_idx = DIV_ROUND_UP(i_size_read(inode), PAGE_SIZE);
if (unlikely(index >= max_idx))
    return VM_FAULT_SIGBUS;

trace_mm_filemap_fault(mapping, index);

folio = filemap_get_folio(mapping, index);
```

The function's following hit/miss branch marks a cache miss major and can start synchronous mmap readahead. Beyond end is `SIGBUS`, not anonymous zero-fill. Waiting for the folio may drop the mmap lock, which is why retry exists.

5.6 Fault accounting follows the result

At the end of `handle_mm_fault`, `mm_account_fault` receives the flags and result. In this tree it treats `VM_FAULT_MAJOR` or `FAULT_FLAG_TRIED` as major, then increments major or minor accounting. The counter is broader than one source line: signal delivery, retries, page-cache races, and other process activity can occur around the measured load.

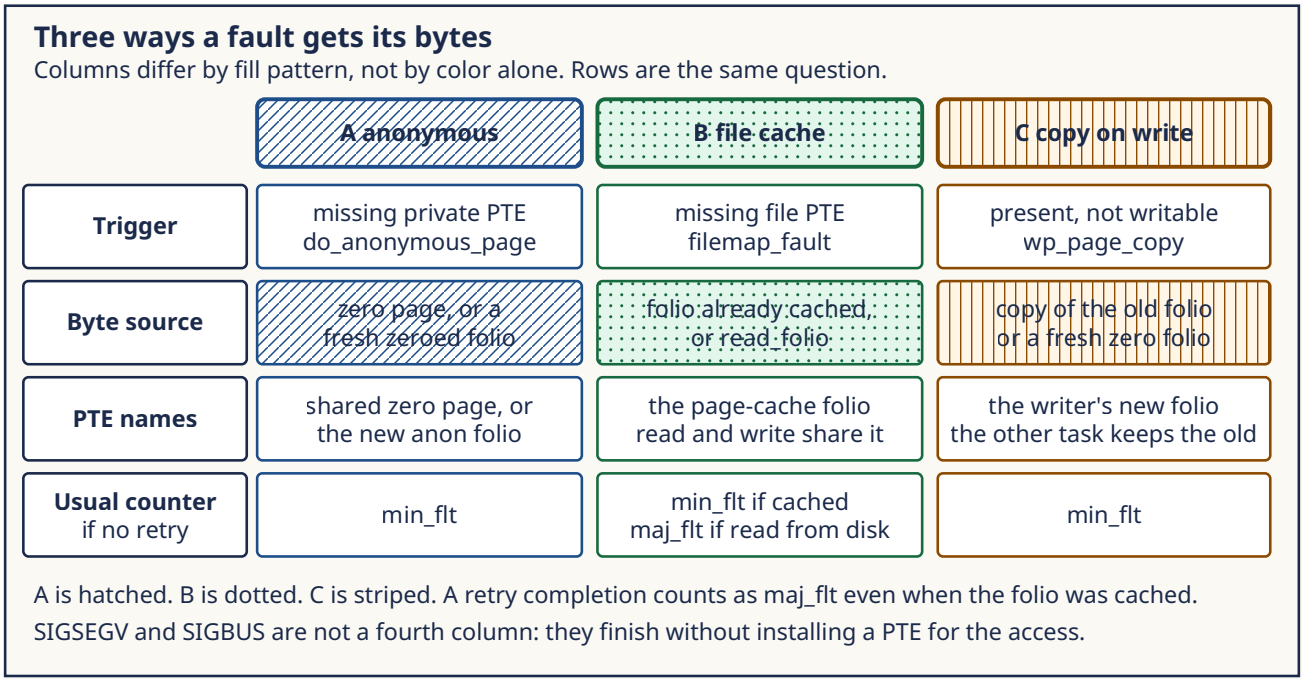


Figure 2. Both kernels defer data movement until access. Linux 6.12 first uses a VMA and can choose zero page, private folio, page cache, swap, huge page, device handler, or failure.

6. What the lab can show

`fault_lab` maps one anonymous page. `mincore` and `smaps` Rss say it is not resident. The first load returns `0`. `ru_minflt` across that load grows by at least one. The program writes a byte, checks `mincore` again, and requires Rss to match one page. The caveat line is part of the result: other activity in this process can raise the minor-fault count.

Run day-002-page-faults.sh.

6.1 The bounded experiment

Run only the existing wrapper:

```
./13-virtual-memory-faults-and-reclaim/challenges/day-002-page-faults.sh
```

`fault_lab` maps one anonymous page and does not create children, temporary files, memory pressure, swap pressure, or host-wide configuration changes. It samples `mincore`, the matching `smaps` VMA, and `getrusage`.

Before the load, it requires the `mincore` low bit to be clear and VMA RSS to be zero. Around one volatile load it samples `ru_minflt` and `ru_majflt`; the loaded byte must be zero. Then it stores `0xA5`, reads it back, checks residency, and requires RSS to equal the runtime page size.

6.2 Observation versus proof

The lab observes: this host returned zero; at least one additional minor fault was charged during the measured interval; the post-store page is resident; and the VMA's RSS equals one runtime page after the store.

The source proves for the pinned tree: x86 constructs generic fault flags; `__handle_mm_fault` allocates missing table levels; an eligible anonymous read can map `my_zero_pfn`; a write allocates an anonymous folio; and a missing file-cache folio sets `VM_FAULT_MAJOR`.

The observation does not prove that the host ran these exact functions. The source does not predict an exact counter delta for the process around stdio and `/proc` parsing. That is why the program asserts `minor_fault_delta >= 1`, prints major delta without treating it as a disk claim for the anonymous page, and explicitly labels other activity.

The read-side RSS is printed but not asserted. A zero-page mapping can leave private anonymous RSS at zero; architecture or policy can choose a private page. The post-store check is the strong point: writable private state must exist for this ordinary anonymous mapping.

7. Execution traces

7.1 Untouched anonymous read

1. `mmap` creates an anonymous readable/writable VMA.
2. No leaf PTE is needed merely to reserve the range.
3. The volatile load misses in the TLB and page table.
4. x86 enters `do_user_addr_fault` with a read fault.
5. VMA lookup and access checks succeed.
6. `handle_mm_fault` descends the tables.
7. `do_anonymous_page` sees no `FAULT_FLAG_WRITE`.
8. If zero-page use is allowed, it installs a special read-only PTE.
9. Exception return retries the load.
10. The load reads zero without owning a private data folio.

7.2 First anonymous store

1. The store either finds no PTE or a read-only zero-page PTE.
2. The fault is marked as a write.
3. VMA policy confirms `VM_WRITE`.
4. Anonymous preparation establishes reverse-map state.
5. A private folio is allocated and initialized.
6. A writable, dirty, young PTE is built from VMA protection.
7. Accounting and reverse mappings are installed.
8. The PTE becomes visible and the TLB is made coherent.
9. The store retries and changes only this mapping's private bytes.

7.3 File read not in cache

1. VMA `vm_file` and `pgoff` identify the `address_space` and index.
2. `filemap_get_folio` misses.
3. The result gains `VM_FAULT_MAJOR`.
4. Readahead or direct read starts storage I/O.

5. The task may sleep; mmap lock may be dropped.
6. On wake, races and file size are checked again.
7. An uptodate cache folio is installed in the process page tables.
8. The original user load retries.

These traces share exception entry but differ in backing resource. "A page fault allocates a page" is only accurate for some branches.

8. Architecture and context

x86 supplies `CR2` and an error code. AArch64 reports a fault address and syndrome in different registers. RISC-V reports `stval` and a trap cause. Generic Linux converts each architecture's state into VMA lookup, `FAULT_FLAG_*`, and `vm_fault_t` outcomes. Do not carry x86 bit numbers into portable fault code.

A user fault usually runs in process context and may sleep. A kernel access to user memory can also fault, but only designated exception-table paths are recoverable. A fault while page faults are disabled, in an atomic section, or without an mm cannot enter the ordinary sleepable path. The same vector number does not imply the same recovery permission.

9. Mastery: Explain · Draw · Observe · Build

Explain

- Why zero-fill is a confidentiality requirement.
- Why a read can complete without private RSS.
- Why a file-cache hit can be minor and swap-in can be major.
- Why `brk` or `mmap` success does not imply resident memory.
- Why fault retry must redo VMA lookup after a dropped lock.
- Why ToyOS must not allocate for every faulting address.

Draw

Draw Figure 1 with three successful branches: zero page, private anonymous folio, and file-cache folio. Add one failure for a VMA hole and one retry for a file wait. Label where sleeping is legal.

Observe

Run the wrapper once and record `uname -r`, runtime page size, pre-touch `mincore`, pre-touch RSS, loaded byte, minor and major deltas, read-side RSS, post-store residency, and post-store RSS. Label all of them "host observation."

Build

Write pseudocode for ToyOS demand-zero handling, but do not edit ToyOS. Include region lookup, permission check, frame allocation, map rollback, and retry. Add explicit failure results for no region, write to read-only, and no frame. State which lock protects the region and page-table update.

10. Limits

The delta is not the number of C statements you can point at. Major faults are printed and are not a disk claim for this anonymous page. The lab does not force reclaim in order to create a major fault.

It also does not observe `CR2`, PTE bits, a zero-page PFN, folio order, readahead, swap, or a scheduler sleep. Those claims come from the pinned source or require different instrumentation. Do not infer host kernel internals from a single counter delta.

Now check your understanding

You have finished the theory for this section. Before moving on, use the quick check below to find anything that still feels uncertain, then reinforce the idea with the practical lab when one is provided.

Quick theory check

Answer from memory first. Explanations appear only after you submit your choices.

- Start the interactive theory check
- View quiz source on GitHub

Put it into practice — Page Faults

Format: shell lab

What you will do: set -euo pipefail

Predict the important result first, then run the lab and explain any difference between prediction and observation.

- Open the lab file

Copy and run:

```
bash 13-virtual-memory-faults-and-reclaim/challenges/day-002-page-faults.sh
```

- View lab source on GitHub

Day 3 — mmap, mprotect, Copy-on-Write, and Shared Mappings

Target time: approximately 3 hours

> Two processes have mappings of the same bytes. A store in the child either changes the parent or it does not. What did the flags decide?

Every Linux 6.12 excerpt below is from tag `v6.12`, commit `adc218676eef25575469234709c2d87185ca223a`, checked out at `/home/jd/Documents/linux-6.12`. Linux 0.12 page numbers are printed pages of Zhao Jiong, *A Heavily Commented Linux Kernel Source Code*, Chinese Revision 5 (2019).

1. Syntax needed for this source

`mmap` and `mprotect` take a protection mask and, for `mmap`, a flag mask. The flags are not a hint the kernel may ignore when you asked for sharing.

```
void *page = mmap(NULL, length, PROT_READ | PROT_WRITE,
                  MAP_SHARED, fd, 0);
mprotect(page, length, PROT_READ);
```

`MAP_SHARED` and `MAP_PRIVATE` select the write rule. `PROT_READ` without `PROT_WRITE` selects whether a write is legal at all. `fork` copies the VMA description into the child. It does not, by itself, copy every frame.

Check the result against `MAP_FAILED`, not `NULL`:

```
void *p = mmap(NULL, length, PROT_READ | PROT_WRITE,
               MAP_PRIVATE | MAP_ANONYMOUS, -1, 0);
if (p == MAP_FAILED)
    return 1;
```

`NULL` asks the kernel to choose an address. `MAP_FAILED` is `(void *)-1`. `prot` and `flags` are different masks: `PROT_*` describes permitted access, while `MAP_SHARED` or `MAP_PRIVATE` selects visibility. File offsets must be page aligned for ordinary mappings. Inside the kernel the byte offset has become `pgoff`, a page index:

```
index = vm_pgoff + ((address - vm_start) >> PAGE_SHIFT)
```

`mprotect` requires a page-aligned start and applies permissions to whole pages. It cannot protect one C object independently when another object shares that page.

Fork status is decoded with wait macros:

```
int status;
waitpid(child, &status, 0);
if (WIFSIGNALED(status) && WTERMSIG(status) == SIGSEGV)
    puts("child faulted");
```

A signal number is not a normal `WEXITSTATUS`.

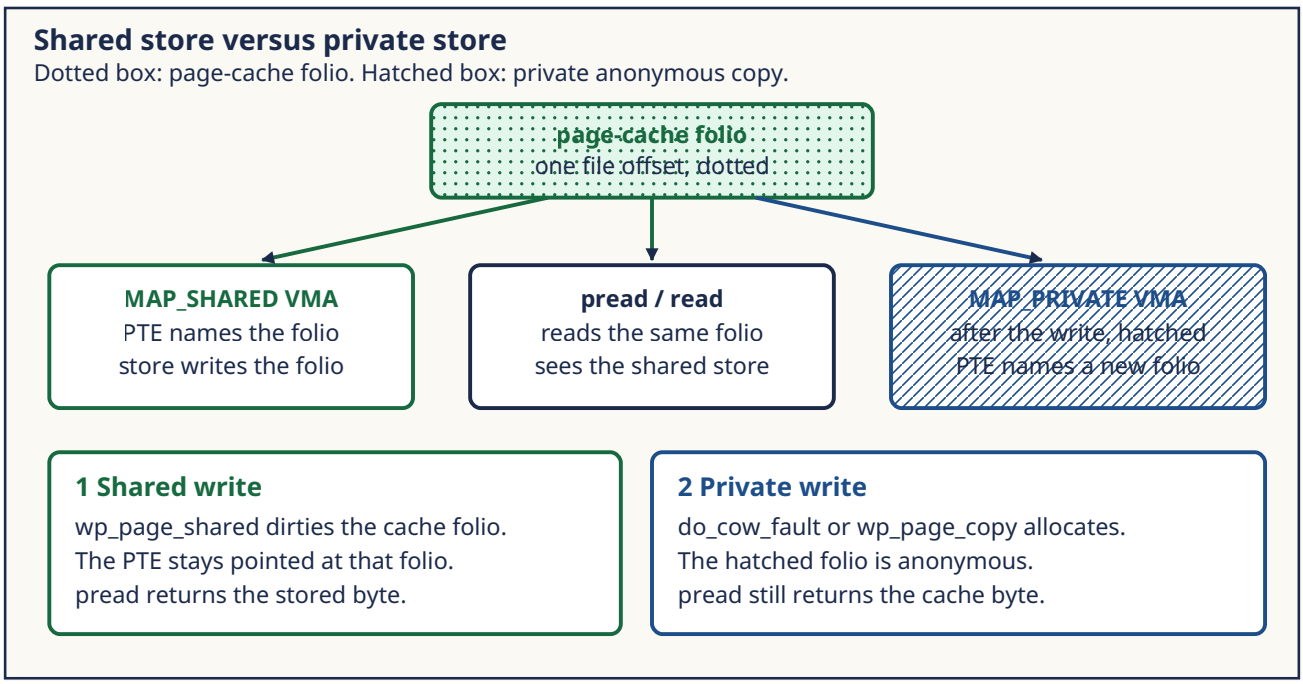


Figure 1. Shared writes retain the page-cache folio as their target. Private writes divert into an anonymous COW folio owned by the writing mm.

2. Linux 0.12 first: copy-on-write without mmap

`copy_page_tables`, used from `copy_mem` during fork, builds the child's tables and clears the write bit on shared pages while `mem_map` counts the extra reference. A later write takes the protection fault into `do_wp_page` and `un_wp_page`. If `mem_map` says the page has one owner, the write bit is set again. If it says more than one, the kernel allocates, copies, and installs the new frame.

There is no `mmap` and no `mprotect` in this kernel. Sharing is the fork rule for the existing code and data. A private file mapping at an arbitrary offset cannot be expressed with `start_code` and `brk`.

2.1 Why fork needs COW

Copying every page during `fork` wastes memory and bandwidth, especially when the child immediately calls `exec`. Leaving pages shared writable would violate private process memory. COW shares bytes while neither side writes and copies only the page a side modifies.

`copy_process` calls `copy_mem` in `kernel/fork.c`, printed pages 407–410. After assigning the child's segmented slot, `copy_mem` calls:

```
if (copy_page_tables(old_data_base,new_data_base,data_limit)) {
    free_page_tables(new_data_base,data_limit);
    return -ENOMEM;
}
```

The child gets new page-table pages, while leaf entries initially name the parent's frames. "Copy page tables" does not mean "copy every data page."

`copy_page_tables` clears the writable bit in child and parent PTEs for ordinary user pages and increments the frame's `mem_map` byte:

```
parent PTE --\
              > physical frame -- mem_map count 2
child PTE  --/
```

Downgrading the parent's PTE requires TLB invalidation. Otherwise a stale writable translation could bypass the future COW fault.

2.2 Reuse when exclusive, copy when shared

The write-protection fault reaches `do_wp_page` and `un_wp_page`. If the page is above `LOW_MEM` and its `mem_map` count is 1, the mapping is exclusive: set writable and retry. If the count is greater than one, allocate a page, decrement the old reference, install the new writable page, copy 4096 bytes, and invalidate.

The writer gets the new frame. COW does not inherently mean "copy for the child"; a parent that writes first can be the side that separates.

Carry forward shared read access, write protection, reference accounting, and copy on private write. Leave behind the LDT slot, global byte array, whole-TLB reload, and fixed 4 KiB copy assumption. Linux 0.12 has no `mmap` or `mprotect` implementing arbitrary user ranges.

3. Evolution: mappings that are not the whole program

Shared libraries, `MAP_SHARED` files, and a protection change on one range need a VMA with its own flags and operations. Copy-on-write stayed. The question "who is allowed to write?" moved into the VMA, so a protection fault and a copy fault are different outcomes. Week 12 already showed the file half: a `MAP_SHARED` store is visible to `pread` because both name page-cache pages. This day adds the child and the forbidden write.

Independent objects forced the change: shared libraries, database windows, anonymous arenas, guard pages, and JIT permission transitions each need their own range, offset, backing, lifetime, and access policy.

"Writable" alone is insufficient:

- shared plus writable means stores modify common backing/cache bytes;
- private plus writable means stores are legal but private to this mm;
- no write permission means the store is illegal, whatever the map type.

Sparse population matters too. `mmap` publishes a VMA, not a copy of a whole file. One private file VMA can simultaneously contain untouched pages, read-only page-cache folios, and anonymous COW folios for pages already written.

`mprotect` adds mutable ranges. Protecting the middle of a VMA may split it into prefix, changed middle, and suffix; compatible neighbors can later merge. VMA locks, mmap locks, PTE locks, and TLB gathering protect different parts of that operation.

Modern validation also includes file mode, no-exec mounts, protection keys, security hooks, and architecture policy. A userspace protection mask is a request. Only a successful call establishes the resulting VMA rule.

4. ToyOS: one user root, no COW

Week 8's `map_page_in` gives the single user process a page-table root. Fork of that address space, write-protect, and a copy on the fault are not in the tree. Comparing ToyOS with the lab means saying so, not pretending the serial log contains a COW trace.

ToyOS has the lower mechanisms: frame allocation, per-process page-table roots, writable leaf flags, local `invlpg`, and vector 14 diagnosis. It lacks forked VMAs, frame reference counts, file/page-cache backing, and a recoverable COW fault.

Its one-bit bitmap cannot represent two owners. If either process cleared the bit on exit, a still-mapped shared frame could be reallocated. A safe anonymous COW extension first needs reference accounting. Fork would copy region metadata and page-table structure, clear writable in both private views, increase frame references, and invalidate stale translations. The fault path would reject illegal stores, reuse an exclusive page, or allocate and copy a shared one.

File `MAP_SHARED` needs more than anonymous COW: a backing object, common cached pages, dirty tracking, writeback, and a lifetime independent of one process. ToyOS's current serial output proves none of those paths.

5. Linux 6.12: do_mmap, do_wp_page, mprotect_fixup

In commit `adc218676eef`, `do_mmap` and `mmap_region` in `mm/mmap.c` build the VMA. `do_fault` in `mm/memory.c` sends a private write to `do_cow_fault` and a shared write to `do_shared_fault`. `do_wp_page` handles the write-protect fault, including the shared case where `VM_SHARED` is set. `do_mprotect_pkey` in `mm/mprotect.c` calls `mprotect_fixup` to change the VMA's flags and the PTEs.

5.1 `do_mmap` converts ABI bits into VMA policy

The entry validates and page-aligns length, checks offset overflow, derives VMA flags, and selects an address:

```
/* tree adc218676eef (Linux 6.12)
 * path: mm/mmap.c
 * lines: 293–323 and 344–357
 * function: do_mmap
 */
len = PAGE_ALIGN(len);
if (!len)
    return -ENOMEM;

if ((pgoff + (len >> PAGE_SHIFT)) < pgoff)
    return -EOVERFLOW;
```

Earlier code rejects zero byte length; later code derives VMA flags and selects an unmapped address. For a shared writable file mapping, later code requires file write access and sets `VM_SHARED | VM_MAYSHARE`. A private mapping requires file read access, but writes become anonymous private state. This policy exists before a data page is faulted.

5.2 `mmap_region` publishes backing and range

The non-merge path allocates and initializes a VMA:

```
/* tree adc218676eef (Linux 6.12)
 * path: mm/mmap.c
 * lines: 1433–1452 and 1497–1514
 * function: __mmap_region
 */
vma = vm_area_alloc(mm);
if (!vma) {
    error = -ENOMEM;
    goto unacct_error;
}

vma_iter_config(&vmi, addr, end);
vma_set_range(vma, addr, end, pgoff);
vm_flags_init(vma, vm_flags);
vma->vm_page_prot = vm_get_page_prot(vm_flags);
```

The following source preallocates iterator state, selects file, shmem, or anonymous backing, checks each error, and publishes the VMA. Compatible neighboring VMAs may merge, so one syscall does not guarantee one permanent VMA object. Anonymous shared mappings receive shmem backing; they are not ordinary private anonymous pages plus a visibility bit.

5.3 Missing private and shared file writes split

`do_fault` makes the policy branch:

```
/* tree adc218676eef (Linux 6.12)
 * path: mm/memory.c
 * lines: 5430–5436
 * function: do_fault
 */
} else if (!(vmf->flags & FAULT_FLAG_WRITE))
    ret = do_read_fault(vmf);
else if (!(vma->vm_flags & VM_SHARED))
    ret = do_cow_fault(vmf);
else
    ret = do_shared_fault(vmf);
```

`do_cow_fault` preallocates an anonymous folio, obtains the file source page, copies it with `copy_mc_user_highpage`, marks the copy uptodate, and finishes the private mapping. The cache source remains the file's page.

`do_shared_fault` obtains the cache folio, calls the filesystem's `page_mkwrite` operation when present, finishes the mapping, and runs `fault_dirty_shared_page`. The cache folio becomes the shared dirty state eligible for later writeback.

5.4 Present private pages reach `do_wp_page`

Fork COW often starts with a present, read-only PTE. The central branch is:

```
/* tree adc218676eef (Linux 6.12)
 * path: mm/memory.c
 * lines: 3700–3745
 * function: do_wp_page
 */
if (vma->vm_flags & (VM_SHARED | VM_MAYSHARE)) {
    if (!vmf->page)
        return wp_pfn_shared(vmf);
    return wp_page_shared(vmf, folio);
}
```

The later private branch performs the exclusive-reuse check or calls the copy path. The full function handles userfaultfd and references around this excerpt. The Linux 0.12 optimization survives in richer form: reuse an exclusive anonymous folio, otherwise copy. Reverse maps, pins, KSM, folios, and concurrency make a single byte reference count insufficient.

5.5 `mprotect` edits policy and translations

The syscall path checks granularity:

```
/* tree adc218676eef (Linux 6.12)
 * path: mm/mprotect.c
 * lines: 720–737
 * function: do_mprotect_pkey
 */
if (start & ~PAGE_MASK)
    return -EINVAL;
if (!len)
    return 0;
len = PAGE_ALIGN(len);
end = start + len;
if (end <= start)
    return -ENOMEM;
if (!arch_validate_prot(prot, start))
    return -EINVAL;
```

After VMA and security checks, `mprotect_fixup` changes both layers:

```
/* tree adc218676eef (Linux 6.12)
 * path: mm/mprotect.c
 * lines: 664–683
 * function: mprotect_fixup
 */
vma = vma_modify_flags(vmi, *pprev, vma, start, end, newflags);
if (IS_ERR(vma)) {
    error = PTR_ERR(vma);
    goto fail;
}
```

The following block resets flags, derives page protection, and calls `change_protection`. Changing only `vm_flags` would leave hardware permissions stale. Changing only PTEs would make later fault policy disagree with the range. The `mmu_gather` batches page-table and TLB work, including invalidation needed when write permission is removed.

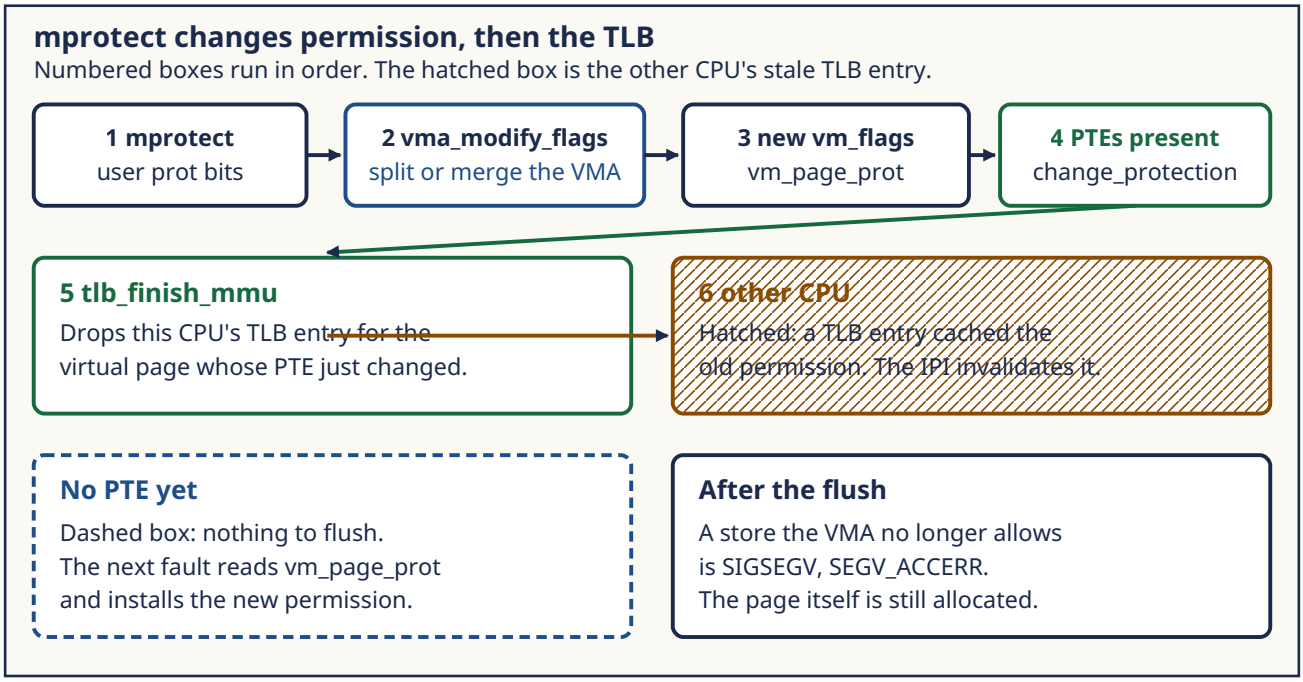


Figure 2. `mprotect` changes the VMA rule and present translations, then invalidates stale cached permission. A later forbidden store becomes a signal; it is not eligible COW.

A write the VMA forbids becomes `VM_FAULT_SIGSEGV` from `handle_mm_fault` when `arch_vma_access_permitted` fails. That signal is what the parent decodes. It is not an oops.

The three cases must stay distinct:

```
private + VM_WRITE + read-only COW PTE  -> reuse/copy, retry
private + no VM_WRITE + store          -> SIGSEGV
shared  + VM_WRITE + write fault       -> shared dirty path
```

6. What the lab can show

`mapping_lab` maps a file `MAP_SHARED`. The child stores a byte. After `wait`, the parent sees it. A second mapping is `MAP_PRIVATE`. The child stores and reports that store on a pipe. The parent's byte stays the original value. A third mapping is `mprotect`ed to `PROT_READ`. Only the child writes it. The parent reads the old byte and prints a signaled wait status whose signal is `SIGSEGV`.

Run day-003-mappings.sh.

6.1 Bounded resources

Run only the existing wrapper:

```
./13-virtual-memory-faults-and-reclaim/challenges/day-003-mappings.sh
```

`mapping_lab` creates a private temporary directory under `/tmp/week13-map.XXXXXX`, two one-page files, and three bounded children. Its waits have alarm protection. It reaps children, unmaps ranges, closes descriptors, and removes the files and directory. Cleanup also runs on ordinary parent exit.

It does not mount a filesystem, flush host caches, force writeback, inspect PFNs, induce memory pressure, change sysctls, or leave children behind. Shared visibility in this experiment is not power-loss durability.

6.2 Observation versus source proof

The lab observes: the shared child's store is visible through the parent's mapping; the private child's store succeeds while the parent retains the original byte; and a child store after `mprotect(PROT_READ)` ends that child with `SIGSEGV` while the parent still reads the old byte.

The pinned source proves: `do_mmap` derives VMA policy; `mmap_region` publishes file, shmem, or anonymous backing; `do_fault` separates COW and shared writes; `do_wp_page` reuses or copies private anonymous folios; and `mprotect_fixup` changes VMA flags, page protection, and PTEs.

The pipe note `private_child_store=yes` proves child-side progress. It is not a PFN trace. Parent bytes staying unchanged prove visible isolation, not the exact helper used by the host kernel. The shared byte proves common mapping semantics, not storage durability; the lab calls neither `msync` nor `fsync`.

7. Execution traces

7.1 Shared child store

1. Parent opens a writable one-page file and maps it `MAP_SHARED`.
2. Its initial load brings the file's page-cache folio into the mapping.
3. Fork gives the child a mapping of the same shared object.
4. Child stores `'C'`.
5. A shared write fault, if needed, makes the folio writable and dirty.
6. Child exits and parent waits, establishing ordering for this test.
7. Parent's load names the same cache-backed bytes and sees `'C'`.

No pipe carries that byte. The mapping is the communication object.

7.2 Private child store

1. Parent maps another file `MAP_PRIVATE` and reads initial `'P'`.
2. Parent and child may initially map the same file-cache folio read-only.
3. Child's store is legal because the VMA allows write.
4. Private policy diverts it into a COW anonymous folio.
5. Child reads `'C'` and sends a progress note through the pipe.
6. Parent waits and verifies the note.
7. Parent still reads `'P'`; the file-cache source was not changed.

The pipe is needed only to prove the child executed after its intentionally private store.

7.3 Protected child store

1. Parent creates private anonymous writable memory and stores `1`.
2. `mprotect(PROT_READ)` changes VMA and PTE permissions.
3. Fork gives the child the same read-only policy.

4. Child attempts to store `2`.
5. VMA validation finds no write permission.
6. The kernel delivers `SIGSEGV`, rather than overriding policy for COW.
7. Parent decodes `WIFSIGNALED` and `WTERMSIG`.
8. Parent can still read `1`.

8. Resource and lifetime relationships

A VMA holds a reference to its file through `vm_file`. The file points to an `address_space`; page-cache folios live there. A private COW folio joins anonymous reverse mapping associated with an mm. PTEs name whichever folio currently answers each virtual page.

Closing the descriptor after successful `mmap` does not unmap the range. The descriptor table and VMA hold independent references, extending Week 12's descriptor → open file → inode graph. `munmap` ends the mapping lifetime; process exit drops remaining VMAs.

One private file VMA can transition page by page:

```
untouched -> file-cache read mapping -> anonymous private COW page
```

Unmapping drops virtual references. The cache folio may remain for other users. An anonymous folio becomes reclaimable only after mappings and pins are gone. "Unmap frees the file page" is too strong.

TLB entries are derived references without C reference counts. The kernel must invalidate them before old physical ownership or old permissions can be treated as gone.

9. Architecture lens

COW is generic policy implemented through architecture PTE permission bits and fault status. x86, AArch64, and RISC-V encode those facts differently. Architecture fault entry converts them into generic flags before `do_wp_page`.

Huge mappings can copy or split at a granularity larger than one base page. Protection keys can deny access beyond ordinary PTE read/write bits. Some architectures update accessed and dirty state in hardware while others need software assistance. The invariant is private write isolation, not a universal sequence of one 4 KiB `memcpy`.

10. Mastery: Explain · Draw · Observe · Build

Explain

- Why fork COW must write-protect the parent's existing PTE.
- Why `MAP_PRIVATE | PROT_WRITE` allows a store without changing the file.
- Why `PROT_READ` produces `SIGSEGV` instead of a private copy.
- Why anonymous `MAP_SHARED` needs common backing.
- Why closing an fd does not end a successful mapping.
- Why shared visibility does not prove durability.

Draw

Redraw Figure 1 with parent PTE, child PTE, page-cache folio, COW folio, and reference arrows. Then redraw Figure 2 with VMA change, PTE change, TLB invalidation, and signal.

Observe

Run the wrapper once. Record `uname -r` and all `yes` markers. Label bytes as process observations, the pipe as child-progress evidence, and the wait status as signal evidence. Record the pinned source commit separately.

Build

Design, but do not implement, minimum ToyOS anonymous-fork COW state: frame reference count, private/shared region flag, parent and child PTE downgrade, TLB invalidation, exclusive reuse, copy rollback, and exit-time reference drop. Explain why the current one-bit bitmap is insufficient.

11. Limits

The wait status names the signal. It does not name the kernel function that sent it. The lab does not dump PTEs or prove which physical frame was copied. Temporary files live under `/tmp/week13-map.*` and are removed before exit.

The lab also does not prove dirty writeback, durability, VMA split shape, TLB shootdown count, folio order, or the host kernel's exact COW helper. Use pinned source for implementation claims and runtime values only for what the processes actually observed.

Now check your understanding

You have finished the theory for this section. Before moving on, use the quick check below to find anything that still feels uncertain, then reinforce the idea with the practical lab when one is provided.

Quick theory check

Answer from memory first. Explanations appear only after you submit your choices.

- Start the interactive theory check
- View quiz source on GitHub

Put it into practice — Mappings

Format: shell lab

What you will do: set -euo pipefail

Predict the important result first, then run the lab and explain any difference between prediction and observation.

- Open the lab file

Copy and run:

```
bash 13-virtual-memory-faults-and-reclaim/challenges/day-003-mappings.sh
```

- View lab source on GitHub

Day 4 — User Allocators, `brk`, `mmap`, and Lifetime

Target time: approximately 3 hours

> `malloc` is a userspace contract about an object's lifetime. The kernel records a virtual-memory operation. Those two books stay separate even when a call happens to cross from one into the other.

The engineering tree for every Linux 6.12 excerpt below is tag `v6.12`, commit `adc218676eef25575469234709c2d87185ca223a`, checked out at `/home/jd/Documents/linux-6.12`. Linux 0.12 page numbers are the printed pages of Zhao Jiong, *A Heavily Commented Linux Kernel Source Code*, Chinese Revision 5 (2019).

1. Syntax: lifetime, `void *`, size, and alignment

A C object has a lifetime. Automatic storage ends when the block ends. Allocated storage begins when an allocation function returns a pointer and ends when `free` is called on that pointer, or when the program ends. After the lifetime ends, the pointer value is indeterminate. A later load or store through it is undefined behavior in the abstract machine. That rule is independent of whether the underlying page is still mapped.

`void *` is an object pointer with no element type. It can hold the address of any object. You convert it before you use it:

```
void *raw = malloc(n);
unsigned char *bytes;

if (raw == NULL)
    return ENOMEM;
bytes = raw;          /* conversion; no arithmetic on raw */
bytes[0] = 0;
```

Pointer arithmetic on `void *` is a GCC extension, not ISO C. The size you pass to `malloc` is a `size_t`. The difference between two pointers into the same array is a `ptrdiff_t`. Mixing those types in a length calculation is how a "small" allocation becomes a huge one, or how a subtraction underflows.

`malloc(n)` returns a pointer aligned for any object with fundamental alignment, or `NULL`. `alignof(max_align_t)` is the alignment that promise is aiming at. It is an alignment of a C object, typically 16 bytes on LP64 System V. It is smaller than a page. `aligned_alloc` and `posix_memalign` exist when the caller needs a stricter alignment. Passing a pointer that is only `max_align_t`-aligned to a function that requires page alignment is a caller bug, even when `malloc` succeeded.

The caller owns the size. The allocated object does not carry a C type that remembers `n`. A correct `free` receives the same pointer value `malloc` returned. It does not receive an interior pointer, and it does not receive a size. The implementation may store a size in bytes before the returned address. That header is part of the allocator, outside the lifetime of the caller's object.

`sizeof` on a pointer yields the width of the pointer, not the width of the allocation. `sizeof *p` after `p = malloc(n)` is the size of the pointed-to type as the compiler sees it. If `p` is `void *`, that expression is a constraint violation in ISO C. If `p` is `char *`, `sizeof *p` is 1, which is the size of one `char`, regardless of `n`.

A store through the pointer is a store. It becomes a page fault only when the translation is missing or forbidden. The C abstract machine does not mention faults. The fault is what the machine does when the virtual address is not yet backed. Section 8 traces that crossing.

2. Linux 0.12 first: `sys_brk` moves one field

Linux 0.12 gives each task one break. `sys_brk` lives in `kernel/sys.c`, printed page 419. The whole function is the check and the assignment:

```
int sys_brk(unsigned long end_data_seg)
{
    if (end_data_seg >= current->end_code &&
        end_data_seg < current->start_stack - 16384)
        current->brk = end_data_seg;
    return current->brk;
}
```

Trace. The argument is a linear address the caller wants as the end of the data segment. The function accepts it when it sits at or after `current->end_code` and at least 16 KiB before `current->start_stack`. On success it stores the address in `current->brk` and returns that value. On failure it leaves `current->brk` alone and still returns `current->brk`. The caller compares the return value with the request. A mismatch is the error. There is no `-ENOMEM` here.

The book is explicit about who calls this. Chapter 15.7, printed page 1023, says the `malloc` in the development-environment library adjusts the end of the process data segment and calls the `brk` system call. The kernel function of the same name in `lib/malloc.c` is a different allocator. It takes pages with `get_free_page` and hands out buckets inside those pages. From 0.98 that kernel function is `kmalloc`. A user program linked with libc never calls `get_free_page`.

A fault in the new gap is `do_no_page`. Week 12 Day 6 quoted the branch: when `inode` is NULL, past the executable image, `get_empty_page` zero-fills a frame and returns. That is the anonymous fill. It is not `bread_page` of the executable. `get_empty_page` is how the early kernel turns the break field into a page. Shrinking the break in this function only stores a smaller address. It does not walk an arbitrary VMA list.

What carries forward is the split in responsibility. Userspace asks for a byte range. The kernel records how far that range may extend. Physical pages arrive on the fault. The single `brk` field, the 16 KiB gap written as a literal, the 80386 segment limits, and the "return the current break instead of an errno" convention belong to this kernel.

The shared Linux 0.12 map says to name the stable mechanism, the historical detail, and the modern path. The stable mechanism is "a syscall moves the edge of a data mapping, and faults fill pages." The historical detail is one break per task and the 16 KiB stack gap. The modern path is `SYSCALL_DEFINE1(brk)` in `mm/mmap.c`, section 5 below.

3. Evolution: arenas, fragmentation, and a policy that is not an ABI

One break is a single growing and shrinking edge. A process that needs a file mapping, a shared mapping, a guard page, and a large anonymous range wants several ranges with holes between them. `mmap` and `munmap` name those ranges. `brk` remains the historical edge of the initial heap. Both are virtual-memory operations. Neither is `malloc`.

A page is the unit the kernel maps. A C object is often smaller. If every `malloc(24)` entered the kernel, the program would pay a syscall, a VMA, and a page for 24 bytes. A userspace allocator keeps a reservoir of address space, carves objects out of it, and returns a freed object to a free list inside that reservoir. The reservoir is an arena. The carved piece is a chunk. The header in front of the chunk remembers the size the C object itself does not remember.

`free` finishes the C lifetime and returns the chunk to the arena. The arena may coalesce neighboring free chunks. It may satisfy the next `malloc` from that free list. It may leave the virtual range mapped. Unmapping is a separate decision: the allocator calls `brk` to shrink, or `munmap` on a mapping it created, when its own policy says the address space is worth returning. A correct program treats `free` as the end of the object. It does not treat `free` as a promise that `/proc/self/maps` will shrink.

Arenas fragment. Internal fragmentation is wasted bytes inside a chunk that is larger than the request, because the allocator rounds up to a size class or to its alignment. External fragmentation is free bytes that are not contiguous enough for the next request, even though their sum would be enough. A picture of a heap that is "half free" can still refuse a large request. The refusal is an allocator decision. It can happen while the kernel still has physical pages, and it can happen while the process still has address space, because the free bytes are the wrong shape.

Threads push the same idea further. One global arena serializes every allocate and free. A per-thread arena, or a per-thread cache in front of a shared arena, lets a thread allocate without taking the global lock. A chunk freed on a different thread than the one that allocated it may have to cross back to the owning arena. That crossing is userspace synchronization. The kernel sees it only if the allocator later issues `brk`, `mmap`, `munmap`, or `madvise`.

Some allocators choose `mmap` for a request above a size they configure, and `brk` or an existing arena below it. That choice is a parameter of the allocator you linked. It can change with `mallopt`, with a build option, with a different libc, or with an arena that already has a hole of the right size. This lesson states no numeric cutoff. Measuring one process on one libc and publishing the number as "how Linux allocates" confuses a userspace policy with the kernel ABI. The kernel ABI is the syscall: the arguments, the errors, and the mapping that appears.

Address-space layout randomization, `RLIMIT_DATA`, and a guard gap before the stack are reasons a modern `brk` can fail even when the machine has free RAM. The failure is "this address space will not extend," reported by returning the old break. `mmap` of a fresh anonymous range can still succeed in the same process, because it does not need the bytes immediately above the current break.

4. ToyOS: a kernel bump heap on frames that are already mapped

ToyOS reconstructs the lower half of the picture and stops before a user allocator.

Week 7's frame allocator in `projects/toyos/src/memory.c` answers "which physical frame?" `frame_alloc` scans a bitmap, sets a bit, zeroes the frame, and returns a physical address. `frame_free` clears the bit. There is no `brk`, no `mmap`, and no userspace `malloc`.

Week 7's kernel heap maps frames at `HEAP_BASE` and then bump-allocates with `kalloc`. The pointer `kalloc` returns is already backed. The first store does not fault, because `memory_init` installed the translations first. The bump pointer only moves forward. A free list, coalescing, and `free` that returns a chunk to the heap are later work. The lesson of that code is ownership: the caller receives an address and a lifetime, and the heap does not yet record who must release it.

A user allocator sits one layer above what ToyOS has. It would need a syscall that creates a VMA, a fault path that calls `frame_alloc` and installs a PTE, and a userspace free list that usually does not call back into the kernel. ToyOS has the frame and the mapper. It has a kernel bump allocator on pre-mapped frames. It does not have the user arena. Reading `kalloc` as if it were `malloc` skips the VMA.

5. Linux 6.12: `brk` edits a VMA

`SYSCALL_DEFINE1(brk)` is in `mm/mmap.c`. The lock and the lower bound come first:

```
/* tree adc218676eef (Linux 6.12)
 * path: mm/mmap.c
 * lines: 115–144
 * function: brk
 */
SYSCALL_DEFINE1(brk, unsigned long, brk)
{
    unsigned long newbrk, oldbrk, origbrk;
    struct mm_struct *mm = current->mm;
    struct vm_area_struct *brkvma, *next = NULL;
    unsigned long min_brk;
    bool populate = false;
    LIST_HEAD(uf);
    struct vma_iterator vmi;

    if (mmap_write_lock_killable(mm))
        return -EINTR;

    origbrk = mm->brk;

#ifdef CONFIG_COMPAT_BRK
    if (current->brk_randomized)
        min_brk = mm->start_brk;
    else
        min_brk = mm->end_data;
#else
    min_brk = mm->start_brk;
#endif
    if (brk < min_brk)
        goto out;
```

The return type of the modern syscall is still the break, an address, on the success path at line 210. The early `-EINTR` is the exception: the lock was interrupted. A request below `min_brk` jumps to `out`, which restores `origbrk` and returns it. `CONFIG_COMPAT_BRK` changes the floor. When the break was randomized, the floor is `mm->start_brk`. Otherwise the historical floor `mm->end_data` remains. That `#ifdef` is a kernel configuration fact. It is still about the VMA edge, not about a userspace size class.

Growth and shrinkage are different operations. Shrinkage is allowed once the request is within the data rlimit, and it becomes an unmap of the abandoned tail. Growth checks the stack guard and then extends or creates an anonymous VMA:

```
/* tree adc218676eef (Linux 6.12)
 * path: mm/mmap.c
 * lines: 156–210
 * function: brk
 */
    newbrk = PAGE_ALIGN(brk);
    oldbrk = PAGE_ALIGN(mm->brk);
    if (oldbrk == newbrk) {
        mm->brk = brk;
        goto success;
    }

    /* Always allow shrinking brk. */
    if (brk <= mm->brk) {
        vma_iter_init(&vmi, mm, newbrk);
        brkvma = vma_find(&vmi, oldbrk);
        if (!brkvma || brkvma->vm_start >= oldbrk)
            goto out; /* mapping intersects with an existing non-brk vma. */
        mm->brk = brk;
        if (do_vmi_align_munmap(&vmi, brkvma, mm, newbrk, oldbrk, &uf,
                    /* unlock = */ true))
            goto out;

        goto success_unlocked;
    }

    if (check_brk_limits(oldbrk, newbrk - oldbrk))
        goto out;

    vma_iter_init(&vmi, mm, oldbrk);
    next = vma_find(&vmi, newbrk + PAGE_SIZE + stack_guard_gap);
    if (next && newbrk + PAGE_SIZE > vm_start_gap(next))
        goto out;

    brkvma = vma_prev_limit(&vmi, mm->start_brk);
    if (do_brk_flags(&vmi, brkvma, oldbrk, newbrk - oldbrk, 0) < 0)
        goto out;

    mm->brk = brk;
    if (mm->def_flags & VM_LOCKED)
        populate = true;

success:
    mmap_write_unlock(mm);
success_unlocked:
    userfaultfd_unmap_complete(mm, &uf);
    if (populate)
        mm_populate(oldbrk, newbrk - oldbrk);
    return brk;
```

`PAGE_ALIGN` means the VMA edge moves by pages. A `brk` that lands inside the page already covered by `mm->brk` only stores the byte address and returns. The mapping does not change. A shrink calls `do_vmi_align_munmap` on `[newbrk, oldbrk)`. A grow calls `do_brk_flags`. `populate` is true only when `VM_LOCKED` is already in `mm->def_flags`. An ordinary `brk` returns after publishing the VMA. The pages are not allocated in this function.

`do_brk_flags` (lines 1778–1845) is the grow. It ORs in `VM_DATA_DEFAULT_FLAGS | VM_ACCOUNT`, checks `may_expand_vm`, `sysctl_max_map_count`, and `security_vm_enough_memory_mm`, then either merges into the existing brk VMA or allocates a new anonymous one:

```
/* tree adc218676eef (Linux 6.12)
 * path: mm/mmap.c
 * lines: 1814–1834
 * function: do_brk_flags
 */
    if (vma)
        vma_iter_next_range(vmi);
    /* create a vma struct for an anonymous mapping */
    vma = vm_area_alloc(mm);
    if (!vma)
        goto unacct_fail;

    vma_set_anonymous(vma);
    vma_set_range(vma, addr, addr + len, addr >> PAGE_SHIFT);
    vm_flags_init(vma, flags);
    vma->vm_page_prot = vm_get_page_prot(flags);
    vma_start_write(vma);
    if (vma_iter_store_gfp(vmi, vma, GFP_KERNEL))
        goto mas_store_fail;

    mm->map_count++;
    validate_mm(mm);
    ksm_add_vma(vma);
```

`vma_set_anonymous` means there is no file. `vma_iter_store_gfp` inserts the VMA into the mm's maple tree, the same structure Week 12 Day 6 used for a file mapping. `mm->total_vm` and `mm->data_vm` grow by `len >> PAGE_SHIFT` at lines 1834–1835. Those counters are virtual pages. They are the reason `/proc/pid/status` can show `VmData` moving while `RssAnon` stays put until something writes the new pages.

The failure path at lines 212–215 restores `origbrk` and returns it. The C library turns "returned break is not the requested break" into `ENOMEM` for the caller of `brk` or `sbrk`. The syscall itself handed back an address.

6. `mmap` is the other operation the kernel is willing to see

A large or independent reservation uses `mmap`. Week 12 Day 6 followed a file mapping through `filemap_fault`. An allocator's anonymous reservation takes the same syscall and leaves `file` NULL:

```
/* tree adc218676eef (Linux 6.12)
 * path: mm/mmap.c
 * lines: 504–515
 * function: ksys_mmap_pgoff
 */
unsigned long ksys_mmap_pgoff(unsigned long addr, unsigned long len,
                  unsigned long prot, unsigned long flags,
                  unsigned long fd, unsigned long pgoff)
{
    struct file *file = NULL;
    unsigned long retval;

    if (!(flags & MAP_ANONYMOUS)) {
        audit_mmap_fd(fd, flags);
        file = fget(fd);
        if (!file)
            return -EBADF;
```

`MAP_ANONYMOUS` skips `fget`. The mapping has no inode. `do_mmap` still builds a VMA through `mmap_region`. `munmap` removes that VMA later. `madvise` and `mprotect` change advice or protection on it. Those are the operations an arena is allowed to ask for. The kernel does not receive the chunk size, the size class, the arena id, or the caller's type. A bug that frees an object twice is a userspace lifetime bug until it corrupts the arena so badly that a later syscall passes a bad address. The kernel is not the referee of `free`.

`VM_LOCKED` and `MAP_POPULATE` are the paths that fault pages in before returning. An ordinary allocator does not set them. The pointer it hands back can be a valid virtual address with no PTE, or with a PTE that still names the shared zero page until the first write.

7. Architecture lens: alignment is three different contracts

The architecture comparison separates page-table facts from C facts.

The C contract is `alignof(max_align_t)` and the stricter alignments the caller requests. It is a property of the ABI and of `malloc`. It does not change because the process crossed into the kernel.

The mapping contract is `PAGE_SIZE`. On x86-64 Linux the base page is 4 KiB. AArch64 Linux can be built with a 4 KiB, 16 KiB, or 64 KiB translation granule. RISC-V Linux selects Sv39, Sv48, or Sv57, and the base page in those modes is 4 KiB. `brk` page-aligns with `PAGE_ALIGN`. A `mmap` offset for a file is rejected on x86-64 when it is not a multiple of the page size, as Week 12 Day 6 quoted. An anonymous mapping's length is rounded up to pages inside the kernel. A 24-byte C object and a 4 KiB mapping can both be "aligned" and still be aligned to different numbers.

The fault contract is the PTE. A read of a fresh anonymous page may install the zero page inside `do_anonymous_page` (section 8). A write allocates a private folio. The ISA decides the PTE format, the fault status register, and the TLB maintenance. The decision "this VMA is anonymous and writable" is generic mm code. ToyOS hard-codes a 4 KiB page and an x86-64 page table. That is a teaching choice. It is the same shape as one legal Linux configuration, and it is a different shape from an AArch64 kernel built with a 16 KiB granule.

Byte order does not enter this path. Allocation is addresses and lengths. The little-endian bias of the three mainstream Linux ABIs is an ecosystem fact about data, recorded in the architecture comparison, and it does not decide whether `brk` or `mmap` runs.

8. Execution trace: from `malloc` to a private folio

Take a request the arena cannot satisfy from its free lists, on a process whose allocator then extends the heap with `brk`. The names below are this tree. The userspace branch is the allocator's; the lesson does not pin a size at which that branch must be taken.

1. The caller passes `size` and checks for `NULL`. The abstract machine has a new object only if the pointer is non-NULL.
2. The arena finds no free chunk of that size class. It asks the kernel to extend address space. One implementation calls `brk`. Another calls `mmap` with `MAP_ANONYMOUS`. Both are legal. The trace follows `brk`.

3. `brk` takes `mmap_write_lock`, page-aligns the request, and calls `do_brk_flags`. A VMA is merged or inserted. `mm->brk`, `total_vm`, and `data_vm` change. The syscall returns the new break. No folio has been allocated for the new range.
4. The arena carves a chunk, writes its header, and returns `header + overhead` to the caller. That pointer is `void *` until the caller converts it.
5. A read of the new object may fault. `do_anonymous_page` (`mm/memory.c` lines 4756–4776) can install the zero page when the fault is not a write and the mm allows the zero page. The allocator's header write is a write, so the interesting fault is the write.
6. The write fault reaches "Allocate our own private page" (`mm/memory.c` lines 4779–4784). `alloc_anon_folio` runs. `CONFIG_TRANSPARENT_HUGEPAGE` may try a higher-order folio first (lines 4705–4709, `vma_alloc_folio`). The fallback at line 4727 is `folio_prealloc`. Either way a folio comes from the physical allocator, which Day 5 opens. The PTE is installed, the instruction restarts, and the header store completes.
7. The caller writes the object and later calls `free`. The arena links the chunk onto a free list and may coalesce it with a neighbor. `brk` and `munmap` stay idle. `/proc/self/maps` still shows the heap VMA or the anonymous VMA. `RssAnon` still includes the folio until the arena advises the kernel to drop it, or until the process exits and the mm is torn down.
8. A second `malloc` of a similar size pops the free list and returns. No syscall, no fault, no new folio.

A shrink that does cross into the kernel is the other trace. The arena calls `brk` with a lower address, or `munmap` on a mapping it owns. `do_vmi_align_munmap` drops the tail. Folios that were private to that range become freeable. Chunks the arena still considers allocated must not sit in that tail. That is an arena invariant. The kernel will unmap whatever range the syscall names.

9. Two pictures

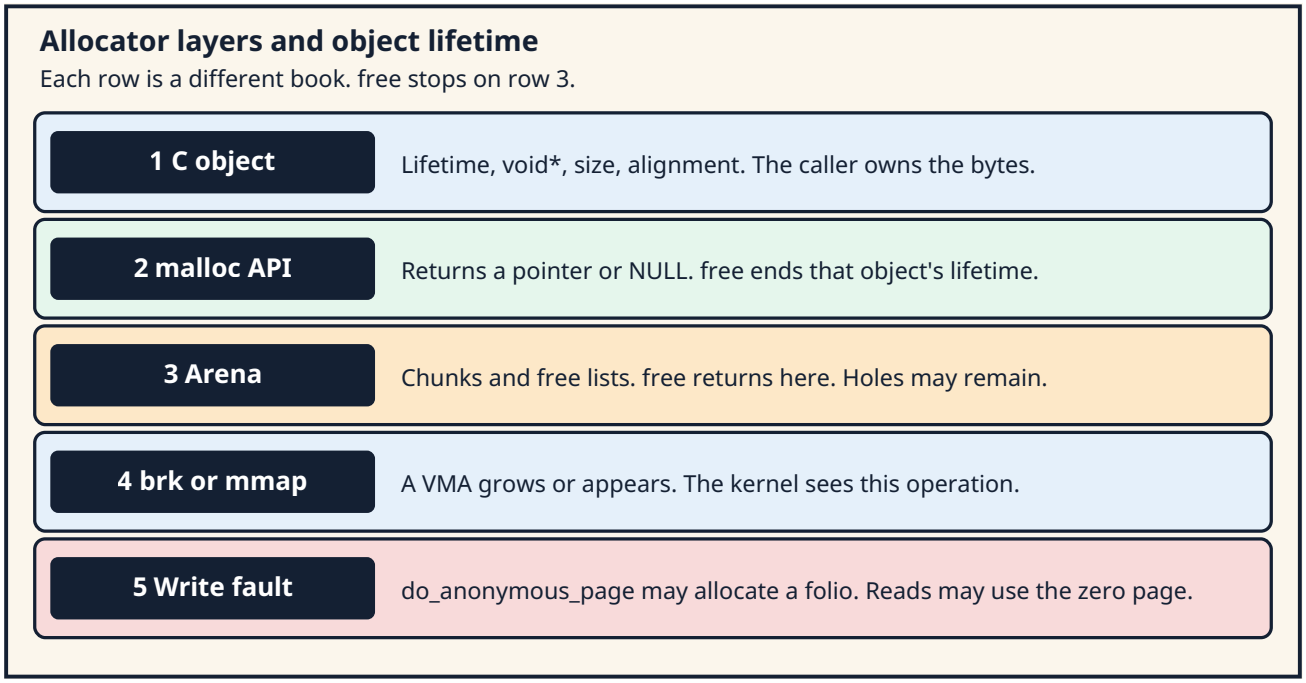


Figure 1. The caller sees a pointer and a lifetime. The arena sees chunks. The kernel sees a VMA and, on a write fault, a folio. `free` stops at the arena.

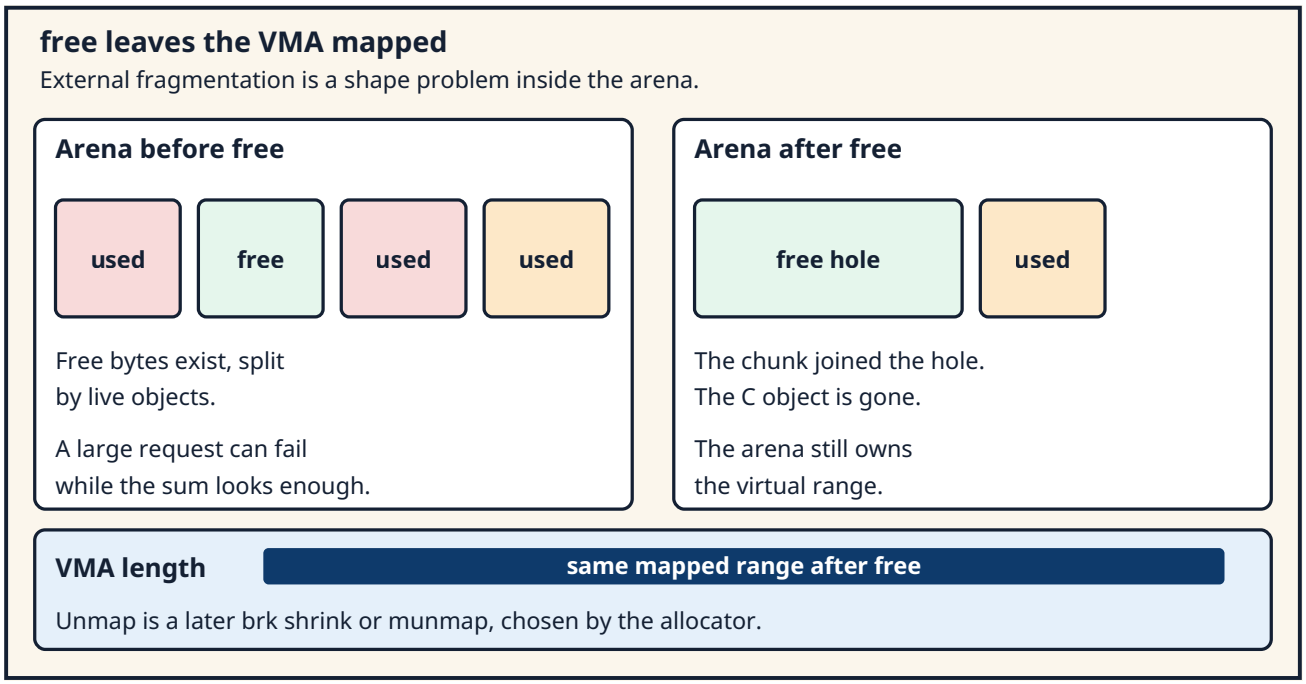


Figure 2. External fragmentation is a shape problem inside the arena. Returning a chunk does not shorten the VMA.

10. A lab that reads one process and invents no threshold

Save this as a file you compile yourself. The wrapper `../challenges/day-004-allocators.sh` builds `projects/linux-vm` `allocator_lab`. That program records `sbrk` around a 32-byte `malloc` and a 2 MiB `malloc`, prints the maps region that contains each pointer, and `munmap`s one explicit page. The source marker `glibc_mmap_threshold_asserted=no` is required. A `brk_changed` value from that run describes this libc on this boot. It is not `M_MMAP_THRESHOLD`. The program below is the same observation, small enough to read in one sitting.

```
#define _DEFAULT_SOURCE
#include <inttypes.h>
#include <stdalign.h>
#include <stddef.h>
#include <stdint.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>

static void dump_maps(const char *tag)
{
    FILE *fp = fopen("/proc/self/maps", "re");
    char line[512];

    printf("== %s  sbrk(0)=%p\n", tag, sbrk(0));
    if (!fp) {
        perror("fopen maps");
        return;
    }
    while (fgets(line, sizeof line, fp)) {
        if (strstr(line, "[heap]") || strstr(line, "[stack]"))
            fputs(line, stdout);
    }
    fclose(fp);
}

int main(void)
{
    void *a = malloc(1);
    void *b = malloc(64);
    size_t align = alignof(max_align_t);

    printf("sizeof(void*)=%zu alignof(max_align_t)=%zu\n",
           sizeof(void *), align);
    if (!a || !b) {
        perror("malloc");
        return 1;
    }
    printf("a=%p a%%align=%zu  b=%p b%%align=%zu\n",
           a, (size_t)(uintptr_t)a % align,
           b, (size_t)(uintptr_t)b % align);
    dump_maps("after malloc");
    free(a);
    free(b);
    dump_maps("after free");
    puts("free ended the C objects. Compare the two [heap] lines.");
    puts("Record what this libc did. State no size cutoff.");
    return 0;
}
```

Compile with `cc -std=c17 -Wall -Wextra -Werror`. Predict, before you run, that both pointers are multiples of `alignof(max_align_t)`, and that the `[heap]` line, if one exists, has the same end address after `free` as after `malloc`. Then run. If this libc satisfies both requests from an arena that already existed at startup, `sbrk(0)` will not move. That observation supports the arena story. It does not identify a threshold, because the program never searched for one.

If you want a second observation, call `mmap` yourself with `MAP_PRIVATE | MAP_ANONYMOUS` for one page, write one byte, print the address, `munmap` it, and show that this mapping appeared and disappeared as its own VMA. That program is using the syscall on purpose. It is still a fact about your call, not a fact about every `malloc` of that length.

Leave the machine alone. Do not loop `malloc` until it fails. Day 6 explains why "allocation failed" has several causes, and Day 7 is the place to record evidence without creating pressure.

11. Mastery: Explain · Draw · Observe · Build

The exercise is `../challenges/day-004-allocators.sh`.

Explain

- Where a C object's lifetime starts and ends, and why a mapped page can outlive that lifetime.
- Why `void *` from `malloc` is converted before it is indexed, and why `sizeof` on the pointer is the wrong size.
- What Linux 0.12 `sys_brk` writes, and which function fills a page later.
- What `do_brk_flags` inserts, and which counters move at that moment.
- Why an arena `free` and `munmap` are different operations.
- Why a size at which one libc calls `mmap` is a property of that libc's policy.

Draw

Draw Figure 1 from memory. Put one address on the arrow from `malloc` to the caller, and a different address, the VMA start, on the arrow from `brk` or `mmap` to the kernel. Mark the write fault as the step that reaches `do_anonymous_page` and `alloc_anon_folio`.

Observe

Run the wrapper. Record `uname -r`, the libc you linked, and the tree id if you opened the sources. Paste the two `[heap]` lines, or write "no `[heap]` line" if the allocator used only anonymous mappings. Quote `alignof(max_align_t)` from the program. Quote no threshold. Confirm the lab printed `glibc_mmap_threshold_asserted=no`.

Build

Write a one-page account of one `malloc`/`free` pair using the functions from this tree: `brk`, `do_brk_flags`, `ksys_mmap_pgoff`, `do_anonymous_page`, `alloc_anon_folio`. End with the sentence you will use when someone says the kernel frees memory at `free`: the kernel drops pages when a VMA changes or the process exits; `free` returns a chunk to the allocator that owns the arena.

Now check your understanding

You have finished the theory for this section. Before moving on, use the quick check below to find anything that still feels uncertain, then reinforce the idea with the practical lab when one is provided.

Quick theory check

Answer from memory first. Explanations appear only after you submit your choices.

- Start the interactive theory check
- View quiz source on GitHub

Put it into practice — Allocators

Format: shell lab

What you will do: set -euo pipefail

Predict the important result first, then run the lab and explain any difference between prediction and observation.

- Open the lab file

Copy and run:

```
bash 13-virtual-memory-faults-and-reclaim/challenges/day-004-allocators.sh
```

- View lab source on GitHub

Day 5 — Physical Pages, Zones, the Buddy Allocator, and NUMA

Target time: approximately 3 hours

> The physical allocator hands out page frames. Page tables name a translation. SLUB hands out objects carved from frames it already obtained. Three allocators, three questions.

The engineering tree is Linux 6.12, tag `v6.12`, commit `adc218676eef25575469234709c2d87185ca223a`, at `/home/jd/Documents/linux-6.12`. Linux 0.12 page numbers are printed pages of Zhao Jiong, Chinese Revision 5 (2019).

1. Syntax: bit orders, intrusive lists, and a tree that is not this allocator

An order is an exponent. A block of order `n` contains `1 << n` pages. Order 0 is one page. Order 3 is eight pages. The byte length is `(1 << n) * PAGE_SIZE`, so the same order is a different number of bytes when `PAGE_SIZE` changes. Shifting is the arithmetic:

```
unsigned int order = 3;
unsigned int pages = 1u << order;          /* 8 */
unsigned long bytes = (unsigned long)pages * 4096ul;
unsigned long buddy = pfn ^ (1ul << order); /* neighbor at this order */
```

`pfn ^ (1 << order)` flips the bit that distinguishes the two halves of an order-`(order+1)` block. That is the buddy address. The XOR is the reason the buddy is unique: each block has one sibling at its order.

A free block is linked with an intrusive doubly linked list. The node is stored inside the object, not beside it:

```
/* tree adc218676eef (Linux 6.12)
 * path: include/linux/types.h
 * lines: 193–194
 */
struct list_head {
    struct list_head *next, *prev;
};
```

`LIST_HEAD` initializes a node to point at itself. The list is empty when `next == &head`. The buddy free lists are an array of these heads, one per migrate type, inside each order:

```
/* tree adc218676eef (Linux 6.12)
 * path: include/linux/mmzone.h
 * lines: 117–120
 */
struct free_area {
    struct list_head    free_list[MIGRATE_TYPES];
    unsigned long       nr_free;
};
```

The highest order in this tree is `MAX_PAGE_ORDER`. It is 10 unless the architecture forces another value:

```
/* tree adc218676eef (Linux 6.12)
 * path: include/linux/mmzone.h
 * lines: 29–33
 */
#ifndef CONFIG_ARCH_FORCE_MAX_ORDER
#define MAX_PAGE_ORDER 10
#else
#define MAX_PAGE_ORDER CONFIG_ARCH_FORCE_MAX_ORDER
#endif
```

Order 10 is 1024 pages. At a 4 KiB page that is 4 MiB. At a 64 KiB granule it is 64 MiB. Quote the macro, then multiply by the running `PAGE_SIZE`.

A maple tree, the structure Day 4 used for VMAs, answers "which VMA covers this virtual address?" A buddy list answers "which free physical block of this order is available in this zone?" The two structures sit in different layers. Finding a VMA does not allocate a frame. Popping a buddy list does not install a PTE.

A bitmap is the third shape. Linux 0.12 uses one byte per page, not one bit, because the byte stores a reference count. ToyOS uses one bit per frame, because a frame is either free or owned. The buddy allocator uses neither as its primary free structure. It uses the per-order lists, and it stores the order of a free block in the page's private field. The comment above `__free_one_page` in `mm/page_alloc.c` (lines 747–768) says that directly: lists of heads of `1 << order` free pages, marked `PageBuddy`, order in `page_private`.

2. Linux 0.12 first: one byte per page, scanned from the top

`mem_map[]` is a byte array in `mm/memory.c`, printed page 883. Each byte is the number of users of one physical page above 1 MiB. `LOW_MEM` is that 1 MiB floor. Pages below it belong to the kernel image and the buffer cache. `free_page` refuses them:

```
void free_page(unsigned long addr)
{
    if (addr < LOW_MEM) return;
    if (addr >= HIGH_MEMORY)
        panic("trying to free nonexistent page");
    addr -= LOW_MEM;
    addr >>= 12;
    if (mem_map[addr]--) return;
    mem_map[addr] = 0;
    panic("trying to free free page");
}
```

Trace. The address is a physical address. Subtract `LOW_MEM`, divide by 4096, and the result indexes `mem_map`. The post-decrement is the whole free operation. If the byte was already 0, the decrement wraps it and the function panics. A shared page, whose byte is greater than 1, loses one reference and stays allocated. There is no buddy, no order, and no merge.

`get_free_page` lives in `mm/swap.c`, printed pages 908–909, because a failed search calls `swap_out`. The scan and the retry are:

```
unsigned long get_free_page(void)
{
    register unsigned long __res asm("ax");
repeat:
    __asm__("std ; repne ; scasb\n\t"
        "jne 1f\n\t"
        "movb $1,1(%%edi)\n\t"
        /* the book continues: zero the page, leave its address in eax */
        : "=a" (__res)
        : "0" (0), "i" (LOW_MEM), "c" (PAGING_PAGES),
          "D" (mem_map + PAGING_PAGES - 1)
        : "di", "cx", "dx");
    if (__res >= HIGH_MEMORY)
        goto repeat;
    if (!__res && swap_out())
        goto repeat;
    return __res;
}
```

The quoted `scasb` scans backward from the last byte of `mem_map` for a 0. On a hit the next instruction stores 1 into that byte. The omitted instructions zero the page and form the address `LOW_MEM` plus the index shifted by 12. A hit past `HIGH_MEMORY` retries. A miss calls `swap_out` and, if that freed a page, retries. The comment in the book is the part to keep: this function names a free physical page and does not map it into a process. `put_page` does the mapping. The kernel's own low memory is already identity-mapped, so kernel callers skip `put_page`.

What carries forward is a physical page allocator with a use count, separate from the page tables. The historical details are the byte array, the backward `scasb`, the 1 MiB floor, the panic on a double free, and the direct call into swap when the array has no zero. Day 6 reads that swap path. Today's modern path is `__alloc_pages_noprof` and `__free_one_page`.

`mem_init` (printed page 899) sets every `mem_map` byte to `USED` and then clears the bytes that cover usable main memory. `show_mem` prints counts from that array when Shift+Scroll Lock is pressed. That is the 0.12 observation tool. Day 7 replaces it with `/proc`.

3. Evolution: zones, buddies, per-CPU lists, and a node order

A single array of pages is enough when every page is equally usable and the machine has one CPU. Both assumptions broke.

Devices with a short DMA address need frames below a limit. The kernel groups frames into zones. `enum zone_type` in `include/linux/mmzone.h` (lines 723–745) documents the reason in the comment above the enumerators: `ZONE_DMA` and `ZONE_DMA32` exist for peripherals that cannot DMA to all of memory, and both enumerators are compiled in only under their `CONFIG_` options. `ZONE_NORMAL` is ordinary addressable memory. `ZONE_HIGHMEM` is a 32-bit-era zone for memory the kernel maps in pieces. `ZONE_MOVABLE` prefers pages that can be migrated so that hot-remove and huge pages have a chance. A build that defines none of the optional zones still has `ZONE_NORMAL`. Quoting "Linux has DMA, Normal, and HighMem" as a fact about every 6.12 kernel invents zones the `.config` did not compile.

Contiguous frames became the second problem. A device, a page table page, or a transparent huge page wants `2^n` physically contiguous pages. Carving them from a flat free list splits the address space into holes. The buddy system keeps a free list per order. An allocation of order `k` that finds only order `n` (`n > k`) splits the block, puts the unused halves on the lower free lists, and returns the piece of order `k`. A free that finds its buddy also free merges, and may merge again at the next order. The merge restores a high-order block only when the sibling is free. A busy sibling leaves external fragmentation: enough free pages, in the wrong shape, for an order-0 allocation, and not enough for order 4.

The zone lock became the third problem. Every allocation and free took the same lock. A per-CPU page list (`struct per_cpu_pages`, reached from `zone->per_cpu_pageset`) holds a few order-0 pages, and some higher orders the kernel treats as cheap, so a common allocation pops a local list. The buddy runs when the list is empty or when the free path returns a batch to the zone. Those lists are a cache in front of the buddy. They are not a second physical memory.

NUMA is the fourth problem. A frame on another node is real RAM and a slower trip. Each node (`struct pglist_data`) has its own zones. An allocation starts with a zonelist: the preferred zone, then fallbacks. `struct zonelist` (lines 1232–1248) is an array of `struct zoneref`. The comment in the header says the first entry is the goal and the rest are fallbacks in decreasing priority. Falling off the local node is a policy outcome of that ordered list, together with the nodemask and cpusets. It is not a property of the buddy XOR.

Watermarks sit on each zone so the allocator can refuse to drain a zone that kswapd is supposed to refill. Section 6 quotes the check. The numbers themselves come from `vm.min_free_kbytes` and from per-zone state. This lesson does not publish a universal watermark.

SLUB arrived on top of all of this, not inside it. The kernel needs objects smaller than a page: task structs, inodes, dentries. SLUB asks the page allocator for a slab page, carves objects, and frees an object back to the slab. The page returns to the buddy when the slab is empty. `kmem_cache_alloc_noprof` (`mm/slub.c` lines 4139–4147) calls `slab_alloc_node`. It does not search `free_area[]`. A `kmalloc` of 128 bytes and an `alloc_pages` of order 0 are different allocators that meet only when a slab needs a new page.

4. ToyOS: a bitmap that answers only the 0.12 question

ToyOS `frame_alloc` in `projects/toyos/src/memory.c` walks `frame_bitmap`, sets the first clear bit, zeroes `PAGE_SIZE` bytes, and returns the physical address. `frame_free` clears the bit and panics if the frame was already free. That is the 0.12 idea — a frame is free or it is not, and a double free is fatal — with a bit instead of a byte and with no `swap_out`. The bitmap stops at the physical window Week 7 built from the Multiboot map. There is one order.

The bitmap cannot hand out eight contiguous frames as a single block, and it cannot merge. There are no zones, because the teaching machine has no DMA map and one node. There is no per-CPU list, because the teaching kernel is uniprocessor in its allocator. The mapper in the same file installs PTEs. `frame_alloc` does not. The Week 7 lesson states that split on purpose: physical ownership and virtual translation are different questions, and the identity map at boot makes them look like the same address.

ToyOS has no SLUB. `kalloc` bump-allocates inside a virtual heap that was mapped with ordinary frames. When that heap is exhausted, `kalloc` fails. It does not split a buddy, and it does not wake a reclaimer. The failure is "the bump pointer reached the end." Day 6 is what Linux does instead of stopping there.

5. Linux 6.12: `alloc_pages` walks a zonelist, then a buddy

The public name `alloc_pages` is a macro in `include/linux/gfp.h` line 333. It lands on `__alloc_pages_noprof`:

```
/* tree adc218676eef (Linux 6.12)
 * path: mm/page_alloc.c
 * lines: 4715–4764
 * function: __alloc_pages_noprof
 */
struct page *__alloc_pages_noprof(gfp_t gfp, unsigned int order,
                        int preferred_nid, nodemask_t *nodemask)
{
    struct page *page;
    unsigned int alloc_flags = ALLOC_WMARK_LOW;
    gfp_t alloc_gfp;
    struct alloc_context ac = { };

    if (WARN_ON_ONCE_GFP(order > MAX_PAGE_ORDER, gfp))
        return NULL;

    gfp &= gfp_allowed_mask;
    gfp = current_gfp_context(gfp);
    alloc_gfp = gfp;
    if (!prepare_alloc_pages(gfp, order, preferred_nid, nodemask, &ac,
            &alloc_gfp, &alloc_flags))
        return NULL;

    alloc_flags |= alloc_flags_nofragment(zonelist_zone(ac.preferred_zoneref), gfp);

    /* First allocation attempt */
    page = get_page_from_freelist(alloc_gfp, order, alloc_flags, &ac);
    if (likely(page))
        goto out;

    page = __alloc_pages_slowpath(alloc_gfp, order, &ac);
    /* out: optional memcg charge, tracepoint, then return page */
    return page;
}
```

The first attempt uses the low watermark. `prepare_alloc_pages` fills `ac` with the zonelist for `preferred_nid` and the highest zone the `gfp` mask allows. `get_page_from_freelist` walks that list:

```
/* tree adc218676eef (Linux 6.12)
 * path: mm/page_alloc.c
 * lines: 3335–3352
 * function: get_page_from_freelist
 */
    no_fallback = alloc_flags & ALLOC_NOFRAGMENT;
    z = ac->preferred_zoneref;
    for_next_zone_zonelist_nodemask(zone, z, ac->highest_zoneidx,
                    ac->nodemask) {
        struct page *page;
        unsigned long mark;
        /* cpuset and dirty-limit checks come first */
```

Inside the loop the function tests watermarks (section 6) and then tries the zone. A remote node can clear `ALLOC_NOFRAGMENT` and retry (lines 3389–3402), because the comment says locality outranks fragmentation avoidance once the allocator has left the local node.

`rmqueue` (lines 3066–3092) tries the per-CPU list when `pcp_allowed_order(order)` says the order is cheap, and otherwise calls `rmqueue_buddy`:

```
/* tree adc218676eef (Linux 6.12)
 * path: mm/page_alloc.c
 * lines: 3073–3081
 * function: rmqueue
 */
    if (likely(pcp_allowed_order(order))) {
        page = rmqueue_pcplist(preferred_zone, zone, order,
                       migratetype, alloc_flags);
        if (likely(page))
            goto out;
    }

    page = rmqueue_buddy(preferred_zone, zone, order, alloc_flags,
                             migratetype);
```

`rmqueue_pcplist` (lines 3019–3044) locks `zone->per_cpu_pageset` and pops `pcp->lists[order_to_pindex(migratetype, order)]`. A miss falls through to the buddy.

6. Split, merge, and the watermark

`__rmqueue_smallest` searches from the requested order upward:

```
/* tree adc218676eef (Linux 6.12)
 * path: mm/page_alloc.c
 * lines: 1586–1608
 * function: __rmqueue_smallest
 */
    for (current_order = order; current_order < NR_PAGE_ORDERS; ++current_order) {
        area = &(zone->free_area[current_order]);
        page = get_page_from_free_area(area, migratetype);
        if (!page)
            continue;

        page_del_and_expand(zone, page, order, current_order,
                    migratetype);
        return page;
    }
    return NULL;
```

`page_del_and_expand` (lines 1421–1429) unlinks the high-order page and calls `expand`. `expand` (lines 1393–1418) walks `high` down to `low`. Each step halves `size` and, unless the half is turned into a debug guard page, adds `&page[size]` to the free list of the new lower order and records that order with `set_buddy_order`. The low half, at the original `page`, is the block the caller receives. The high half is the buddy left free.

The free path merges. `__free_one_page` (lines 771–840) loops while `order < MAX_PAGE_ORDER`. `find_buddy_page_pfn` names the sibling. If the sibling is free and the migrate types may merge, the sibling leaves its free list, the combined block's address is the lower of the two PFNs, and `order` increments. The loop stops at a busy buddy, at a migrate type that must not merge, or at `MAX_PAGE_ORDER`. The survivor is linked at the final order.

Migrate types keep unmovable, movable, and reclaimable pages from splintering every pageblock. `fallbacks` (lines 1618–1622) is the order in which a depleted type may steal from another. A merge across a pageblock boundary checks `migratetype_is_mergeable` (lines 803–815). The XOR still names the buddy. The migrate type can refuse the merge.

The watermark check that gates the zone is `__zone_watermark_ok`:

```
/* tree adc218676eef (Linux 6.12)
 * path: mm/page_alloc.c
 * lines: 3122–3171
 * function: __zone_watermark_ok
 */
    free_pages -= __zone_watermark_unusable_free(z, order, alloc_flags);

    if (unlikely(alloc_flags & ALLOC_RESERVES)) {
        if (alloc_flags & ALLOC_MIN_RESERVE) {
            min -= min / 2;
            if (alloc_flags & ALLOC_NON_BLOCK)
                min -= min / 4;
        }
        if (alloc_flags & ALLOC_OOM)
            min -= min / 2;
    }

    if (free_pages <= min + z->lowmem_reserve[highest_zoneidx])
        return false;

    if (!order)
        return true;
```

Order 0 is done once the free count beats the adjusted min plus the lowmem reserve. A higher order also needs a free block of that order or above (lines 3174–3197). `ALLOC_OOM` and `ALLOC_MIN_RESERVE` may dip into the reserve. They are flags on the allocation, not a statement that the node is empty. `get_page_from_freelist` compares against the high watermark as well (lines 3414–3423) and sets `ZONE_BELOW_HIGH` when the zone has fallen beneath it. The fast path's allocation watermark is still the low mark from line 4719. Day 6 follows what happens when every zone in the zonelist fails this test.

7. Three machines that share pages and nothing else

Keep the names straight when you read a fault.

The physical allocator returns `struct page *` for a frame, or a compound folio of `1 << order` frames. It updates `free_area`, the per-CPU lists, and zone counters. It does not write a user PTE.

The page-table walker runs on a fault or on a call such as `mmap` that builds tables. `do_anonymous_page` from Day 4 calls `vma_alloc_folio` or `folio_prealloc`, then stores a PTE. The PTE names the frame. The frame's identity was decided by the allocator. Replacing the PTE changes the translation and leaves the buddy lists alone until the frame's reference count says it can be freed.

SLUB returns `void *` to an object inside a slab page. The slab page's `struct page` is one the buddy allocated earlier. Freeing the object returns it to the slab's freelist. Freeing the page, when the slab is empty, calls back into `__free_pages`. A SLUB debug poison pattern is a statement about an object. A buddy `PageBuddy` flag is a statement about a free block of pages. Seeing one when you expected the other means you are in the wrong allocator.

`/proc/buddyinfo` prints the per-order free counts. `/proc/zoneinfo` prints watermarks and zone sizes. `/proc/slabinfo` prints SLUB caches. They are three files because they are three accounts. Day 7 uses the process-level files; these three are the kernel-side counterparts, and reading them is safe.

8. Architecture lens: the order is pages, the zone is a platform

`PAGE_SIZE` is 4 KiB on the ToyOS teaching machine and on the usual x86-64 and RISC-V Linux configurations. An AArch64 kernel may use a 16 KiB or 64 KiB granule. Order 1 is then 8 KiB, 32 KiB, or 128 KiB. Code that converts an order to bytes multiplies by `PAGE_SIZE`. Code that hard-codes 4096 has picked one configuration.

DMA zones follow the platform. A 64-bit x86 machine often has `ZONE_DMA` and `ZONE_DMA32` because old devices and 32-bit DMA masks still exist. An AArch64 server may have a different DMA limit, or none that the kernel models as a separate zone. The zonelist order is also a platform and policy choice: node-local `ZONE_NORMAL` first is the common picture, and a memoryless node or a CPU-less node changes what "local" means. The generic code walks whatever `build_zonelists` installed. The XOR that names a buddy does not change across ISAs. The PTE that later points at the page does.

NUMA distance is a firmware table (SLIT/SRAT on ACPI platforms, or the architecture's equivalent) interpreted by the kernel. It is not an ISA instruction. The allocator's use of that distance is the zonelist. A store on the CPU still goes through the cache-coherence protocol of that architecture once the page is mapped. The buddy allocator does not emit a fence.

9. Execution trace: one order-0 page from the local node

1. A fault, a slab refill, or a driver calls `alloc_pages` with order 0 and a `gfp` mask. `__alloc_pages_noprof` rejects an order above `MAX_PAGE_ORDER` and builds `alloc_context` for the preferred node.
2. `get_page_from_freelist` starts at `preferred_zoneref`. For a normal kernel allocation that is the local node's preferred zone, often `ZONE_NORMAL` when the mask allows it.
3. `__zone_watermark_ok` runs with the low watermark. A zone that fails is skipped. The walker continues along the zonelist, possibly onto another node.
4. On a zone that passes, `rmqueue` tries `rmqueue_pcplist`. The per-CPU list for that migrate type has an order-0 page. The page is popped and returned. The buddy lists do not move.
5. If the per-CPU list is empty, `__rmqueue_smallest` finds the smallest `current_order >= 0` with a free page of the right migrate type. Suppose it finds order 2. `expand` links the upper half as an order-1 buddy and the upper quarter of the lower half as an order-0 buddy, and returns the lowest page. Three free-list operations, one page for the caller.
6. `post_alloc_hook` (line 1500) sets the page refcounted. The caller's layer — fault handler, SLUB, or driver — uses the page. The buddy bit is clear. The page is allocated.
7. The matching free calls `__free_pages` with the same order the caller still knows. `free_one_page` takes `zone->lock` and `__free_one_page` merges while buddies are free, then links the result. A per-CPU free of a cheap order may instead push the page onto the local list and delay the buddy merge until a batch is flushed. Both end states are "the page is free." They are not the same list.

A high-order allocation that finds every free list empty in the watermark sense does not stop inside this trace. It enters `__alloc_pages_slowpath`. Day 6 starts there.

10. Two pictures

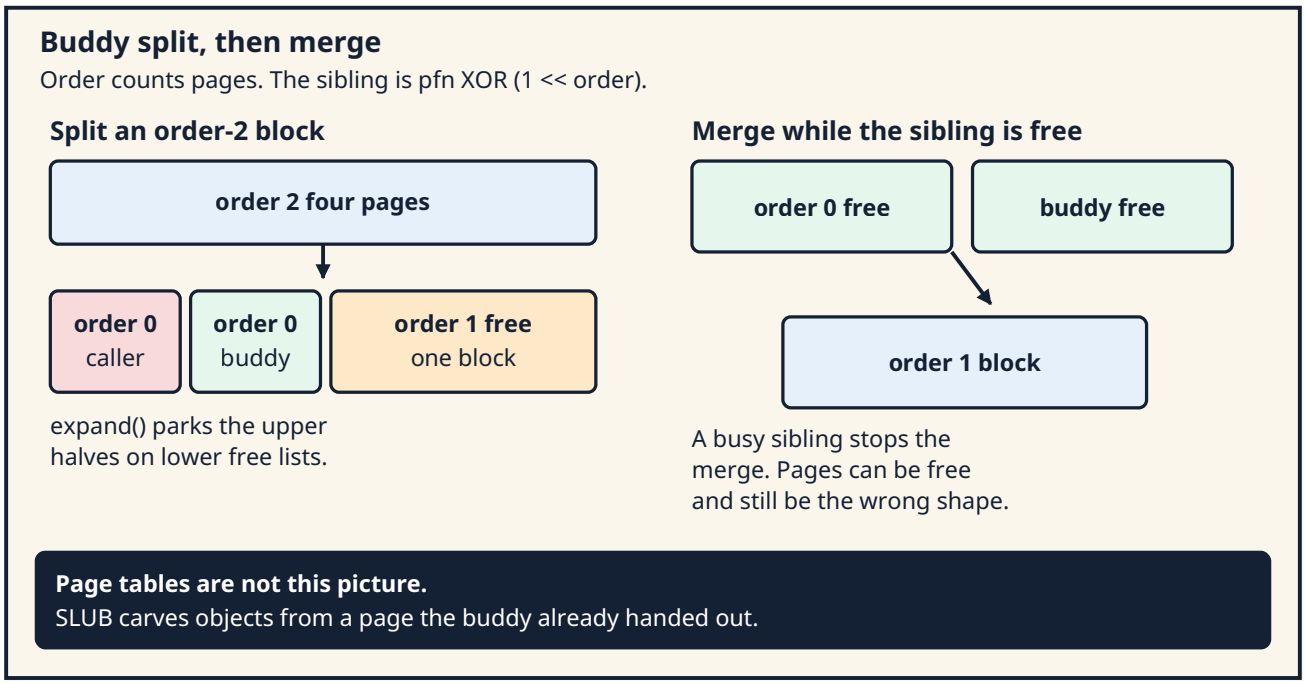


Figure 1. Split parks the unused half on a lower free list. The order-2 picture leaves one order-0 buddy and one order-1 block. Merge runs only when that sibling is free.

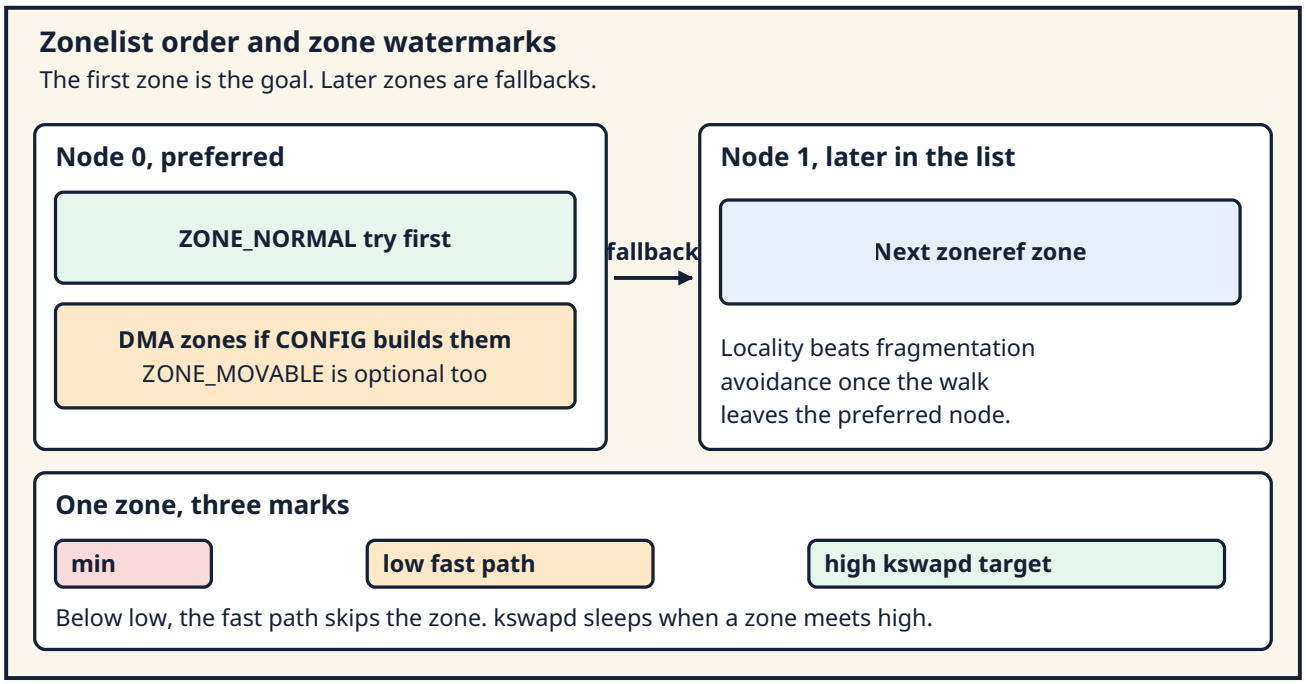


Figure 2. The zonelist is an order of zones. The watermark is a test on one zone, not a test that the machine is out of RAM.

11. A small buddy you can run, and a kernel file you only read

The wrapper is `../challenges/day-005-buddy-numa.sh`. It builds `projects/linux-vm buddy_lab`. That simulator has two nodes, a small DMA zone and a NORMAL zone on each, splits and coalesces, falls back when the local NORMAL zone is empty, and fails an order the

zones cannot hold. The markers `simulator_only`, `numa_fallback`, and `exhausted_alloc` are the evidence. The simulator does not call `get_page_from_freelist` and does not allocate host pages. Its zone order is a teaching rule. Do not report its node 1 as a `numastat` measurement.

The program below is a 16-page buddy in ordinary C. It is a model of the split and merge, not a copy of `mm/page_alloc.c`. Beside it, read the first lines of `/proc/buddyinfo` and `/proc/zoneinfo` if those files exist. Do not allocate until the allocator fails.

```
#include <stdio.h>

#define N 16
static unsigned char order_of[N];
static unsigned char is_free[N];

static void init(void)
{
    for (int i = 0; i < N; i++)
        is_free[i] = 0;
    order_of[0] = 4; /* 2^4 == 16 pages, one block at index 0 */
    is_free[0] = 1;
}

static int alloc_order(unsigned ord)
{
    for (unsigned o = ord; o <= 4; o++) {
        for (int i = 0; i < N; i += (1 << o)) {
            if (!is_free[i] || order_of[i] != o)
                continue;
            is_free[i] = 0;
            while (o > ord) {
                o--;
                int buddy = i + (1 << o);
                is_free[buddy] = 1;
                order_of[buddy] = (unsigned char)o;
            }
            order_of[i] = (unsigned char)ord;
            return i;
        }
    }
    return -1;
}

static void free_order(int i, unsigned ord)
{
    is_free[i] = 1;
    order_of[i] = (unsigned char)ord;
    while (ord < 4) {
        int buddy = i ^ (1 << ord);
        if (buddy < 0 || buddy >= N || !is_free[buddy] ||
            order_of[buddy] != ord)
            break;
        is_free[i] = is_free[buddy] = 0;
        if (buddy < i)
            i = buddy;
        ord++;
        is_free[i] = 1;
        order_of[i] = (unsigned char)ord;
    }
}

int main(void)
{
    init();
    int a = alloc_order(0);
    int b = alloc_order(0);
    printf("alloc order0 -> %d and %d\n", a, b);
    free_order(a, 0);
    free_order(b, 0);
    printf("after free, block0 free=%u order=%u\n",
```

```
            is_free[0], order_of[0]);
    puts("Read /proc/buddyinfo for the kernel's per-order counts.");
    return a < 0 || b < 0;
}
```

Predict that the two order-0 allocations come from a split of the order-4 block, and that freeing both restores `is_free[0]` with order 4. Then predict, separately, that `/proc/buddyinfo` will show many order-0 free pages and fewer high-order pages. The second prediction is about fragmentation on this machine at this moment. It will not match the 16-page toy, and it should not. If `buddyinfo` is absent, say so. That is a measurement result.

The toy has no zones, no per-CPU list, no migrate types, and no watermark. Adding those to the toy without a caller that needs them hides the XOR. The kernel excerpts are where those pieces live.

12. Mastery: Explain · Draw · Observe · Build

The exercise is `../challenges/day-005-buddy-numa.sh`.

Explain

- What `mem_map[i]` means in Linux 0.12, and why `get_free_page` does not install a PTE.
- Why a zone exists, and which zone enumerators depend on `CONFIG_`.
- How `expand` and `__free_one_page` use the buddy at order `n`.
- What a zonelist orders, and what `__zone_watermark_ok` compares.
- Where a per-CPU page list sits relative to `rmqueue_buddy`.
- Why `kmem_cache_alloc_noprof` and `alloc_pages` are different allocators.
- Why `MAX_PAGE_ORDER` is 10 only when `CONFIG_ARCH_FORCE_MAX_ORDER` is unset.

Draw

Draw Figure 1 for an order-2 block serving an order-0 allocation, with the order-0 buddy and the order-1 block labeled. Beside it, draw the zonelist from the local `ZONE_NORMAL` to one fallback, and mark the low watermark as the fast-path test in `__alloc_pages_noprof`.

Observe

Run the wrapper. Record the simulator's fallback and the exhausted order, and say that no host page moved. Copy one line of `/proc/buddyinfo` and one watermark line from `/proc/zoneinfo` (`min`, `low`, `high` under a zone you name). Record `uname -r`. If the files are missing, record the error.

Build

Write a one-page trace of a single order-0 allocation that misses the per-CPU list and splits an order-2 block, using `__rmqueue_smallest`, `expand`, and `__free_one_page`. Finish with the sentence you will use when someone points at a page table and calls it the buddy allocator: the page table names a translation, the buddy names a free physical block, and SLUB names an object inside a block the buddy already gave out.

Now check your understanding

You have finished the theory for this section. Before moving on, use the quick check below to find anything that still feels uncertain, then reinforce the idea with the practical lab when one is provided.

Quick theory check

Answer from memory first. Explanations appear only after you submit your choices.

- Start the interactive theory check
- View quiz source on GitHub

Put it into practice — Buddy Numa

Format: shell lab

What you will do: set -euo pipefail

Predict the important result first, then run the lab and explain any difference between prediction and observation.

- Open the lab file

Copy and run:

```
bash 13-virtual-memory-faults-and-reclaim/challenges/day-005-buddy-numa.sh
```

- View lab source on GitHub

Day 6 — Reclaim, Writeback, Swap, and the OOM Killer

Target time: approximately 3 hours

> Reclaim asks which pages can be turned into free frames. The OOM killer is what an eligible allocation does after reclaim has failed. A machine can take that path with plenty of RAM in the wrong zone, node, order, or cgroup.

The engineering tree is Linux 6.12, tag `v6.12`, commit `adc218676eef25575469234709c2d87185ca223a`, at `/home/jd/Documents/linux-6.12`. Linux 0.12 page numbers are printed pages of Zhao Jiong, Chinese Revision 5 (2019).

1. Syntax: a list head, a reference, and a flag bit

A folio that reclaim may select sits on a list. The node is the same `struct list_head` as Day 5: `next` and `prev`, embedded in the object. The lists themselves live on the memory cgroup's LRU vector:

```
/* tree adc218676eef (Linux 6.12)
 * path: include/linux/mmzone.h
 * lines: 276–282 and 608–609
 */
enum lru_list {
    LRU_INACTIVE_ANON = LRU_BASE,
    LRU_ACTIVE_ANON = LRU_BASE + LRU_ACTIVE,
    LRU_INACTIVE_FILE = LRU_BASE + LRU_FILE,
    LRU_ACTIVE_FILE = LRU_BASE + LRU_FILE + LRU_ACTIVE,
    LRU_UNEVICTABLE,
    NR_LRU_LISTS
};

struct lruvec {
    struct list_head        lists[NR_LRU_LISTS];
```

Five lists, not one. Anonymous pages and file pages are separate, and each of those has an active and an inactive list. `LRU_UNEVICTABLE` holds pages reclaim has been told not to take, such as pages locked with `mlock`. A scan that "walks the LRU" walks one of these lists under `lruvec->lru_lock`.

A reference count answers "how many users?" A flag bit answers "what state?" The flag names in this tree are an enum that starts at bit 0:

```
/* tree adc218676eef (Linux 6.12)
 * path: include/linux/page-flags.h
 * lines: 93–102
 */
enum pageflags {
    PG_locked,      /* Page is locked. Don't touch. */
    PG_writeback,       /* Page is under writeback */
    PG_referenced,
    PG_uptodate,
    PG_dirty,
    PG_lru,
    PG_head,        /* Must be in bit 6 */
    PG_waiters,     /* Must be bit #7 and in the same byte as PG_locked */
    PG_active,
```

In this enum, with no `#ifdef` between these enumerators, `PG_locked` is bit 0, `PG_writeback` is bit 1, `PG_referenced` is bit 2, `PG_dirty` is bit 4, `PG_lru` is bit 5, and the comments fix `PG_head` at bit 6 and `PG_waiters` at bit 7. Later enumerators and other configurations add bits above those. Quote the enum from the tree you build. Do not carry the numbers into a different version by memory.

`PG_dirty` means the folio's contents are newer than the backing store. `PG_writeback` means I/O has started. The comment at the top of the header says `PG_dirty` is cleared when writeback starts. `PG_referenced` and `PG_reclaim` are reclaim's marks. `PG_lru` means the folio is on one of the lists. `PG_active` selects the active list rather than the inactive one when generations are not in charge.

The reference count is not one of those bits. `post_alloc_hook` in Day 5's allocator sets the page refcounted when the buddy hands it out. Reclaim drops its LRU reference only when it has a plan for every other user: unmapped, not under writeback, and not held by a driver. A folio with `PG_writeback` set is busy. Isolating it and freeing it immediately would throw away I/O that has not finished.

A test in C is the same shape as the kernel's flag test. You mask one bit and you leave the others alone:

```
unsigned long flags = folio_flags;
int dirty = (flags >> 4) & 1ul; /* PG_dirty in this tree's enum */
```

Prefer `folio_test_dirty` in kernel code. The shift is here so the bit number stays tied to the enum you just read.

2. Linux 0.12 first: a buffer cache, then a desperate scan

Linux 0.12 has two stores that can give a page back.

The buffer cache in `fs/buffer.c` (chapter 12.2) holds recently used block-device blocks, hashed by device and block number, and linked on a free list. A `bread` that hits the cache does not touch the device. A miss allocates a buffer and reads the block. Dirty buffers are written back when the cache reclaims them or when the system syncs. This cache is a block cache. It is not the page cache `read` and `mmap` share in Linux 6.12. The block size, the hash of 307 heads, and the placement of the cache between the kernel image and main memory are historical.

The other store is the swap device. `get_free_page` calls `swap_out` when `mem_map` has no free byte. `try_to_swap_out` (`mm/swap.c`, printed page 906) looks at one page-table entry:

```
int try_to_swap_out(unsigned long *table_ptr)
{
    unsigned long page = *table_ptr;
    unsigned long swap_nr;

    if (!(PAGE_PRESENT & page))
        return 0;
    if (page - LOW_MEM > PAGING_MEMORY)
        return 0;
    if (PAGE_DIRTY & page) {
        page &= 0xfffff000;
        if (mem_map[MAP_NR(page)] != 1)
            return 0;
        if (!(swap_nr = get_swap_page()))
            return 0;
        *table_ptr = swap_nr << 1;
        invalidate();
        write_swap_page(swap_nr, (char *) page);
        free_page(page);
        return 1;
    }
    *table_ptr = 0;
    invalidate();
    free_page(page);
    return 1;
}
```

Trace. A missing PTE is skipped. A dirty page that is shared (`mem_map` byte not 1) is skipped. A dirty private page gets a swap slot. The PTE is rewritten with `swap_nr << 1`, which leaves the present bit clear, and the comment in the book says why: a present bit of 0 lets the rest of the PTE hold a swap identity. `write_swap_page` sends the page to the swap device. `free_page` drops the frame. A clean page is simpler. The PTE is cleared and the frame is freed, because the bytes can be read again from the executable image. `swap_out` walks directory entries from task 1's range and returns 1 when one page moves. If the walk finishes with nothing, it prints `Out of swap-memory`. `oom()` in this kernel is the path that gives up on the current task. There is no score across tasks.

What carries forward is the decision on one page: clean pages can be dropped, dirty anonymous pages must be written somewhere first, and a shared page is a poor victim. The historical details are the single swap device, the PTE encoding `swap_nr << 1`, the `invalidate()` of the whole TLB, and the call from `get_free_page` on the allocating CPU. There is no kswapd, no separate file LRU, and no OOM killer that picks a different task. The modern path is `try_to_free_pages`, `kswapd`, `swap_writepage`, and `out_of_memory`.

3. Evolution: one cache, two LRUs, a background thread, a last resort

The buffer cache and the page cache duplicated the same file bytes when a block and a page both cached them. Linux unified them: a file page is a page-cache folio, and the block layer maps buffer heads onto that folio when it needs block-sized I/O. Week 12 Day 3 and Day 6 already use that cache for `read` and `mmap`. Reclaim now sees one object with two reasons to be kept: a user mapping, and a cache that would save a disk read.

The single scan of every PTE became lists of victims. Active and inactive lists, for anonymous folios and for file folios, let reclaim prefer pages that have not been used lately. Referenced pages get a second chance. Unevictable pages leave the scan. That is the five-list `enum lru_list` above. A reverse map exists because one folio can be named by many PTEs. `try_to_unmap` is how reclaim clears those PTEs when it has decided the folio may leave.

Multi-generation LRU is a further change, and it is configuration dependent in this tree. `struct lruvec` contains `struct lru_gen_folio lrugen` only inside `#ifdef CONFIG_LRU_GEN` (`include/linux/mmzone.h` lines 625–632). `lru_gen_enabled()` has three bodies in `include/linux/mm_inline.h`:

- `CONFIG_LRU_GEN` off: the function returns false (lines 296–299).
- `CONFIG_LRU_GEN` on and `CONFIG_LRU_GEN_ENABLED` on: a static key that defaults likely (lines 105–110).
- `CONFIG_LRU_GEN` on and `CONFIG_LRU_GEN_ENABLED` off: a static key that defaults unlikely (lines 113–118).

`shrink_node` calls `lru_gen_shrink_node` only when `lru_gen_enabled()` is true and `root_reclaim(sc)` is true (`mm/vmscan.c` lines 5942–5944). A kernel built without `CONFIG_LRU_GEN` walks the active and inactive lists. A kernel built with the option still has to have the static key on, and a memcg-scoped reclaim can take a different branch. Say "this boot uses generations" only after you have read the config and the runtime key. The lesson does not assume either answer.

Writeback became its own path. A dirty file folio is written to the file's address space by the flusher, not by a swap device. The folio stays in the page cache, clean, unless reclaim later drops it. A dirty anonymous folio has no file. Swap, or a swap-like store such as zswap, is the place the bytes go so the frame can be freed. Clean file pages are the cheap victims. Dirty pages cost I/O. Anonymous pages cost swap space. Reclaim prefers work it can finish.

The allocating thread is a bad place to do all of that work. kswapd is a per-node kernel thread. The allocator wakes it when a zone is under the watermark and the `gfp` mask allows `ALLOC_KSWAPD`. The allocating thread tries the freelist again. If the zone is still short and the mask allows direct reclaim, the allocator itself calls `try_to_free_pages` and then retries. Direct reclaim is the stall the faulting task feels. kswapd is the attempt to get ahead of that stall.

The OOM killer runs when an allocation that is allowed to invoke it has not made progress. `__alloc_pages_may_oom` skips it for orders above `PAGE_ALLOC_COSTLY_ORDER` (the comment at lines 3613–3615 says the killer will not help higher-order allocations, and that the caller likely has a fallback). It skips it when `__GFP_FS` is missing, inside `out_of_memory`, except for a memcg OOM. It can do nothing useful when no task is killable. A cgroup can hit its own limit while the rest of the machine has free pages. A node can be below its watermark while another node is idle, and the nodemask can forbid the idle node. A high-order allocation can

fail in a sea of order-0 pages. "OOM because RAM is full" erases every one of those branches. The accurate sentence is: the killer is the last resort of an allocation that already failed reclaim under the constraints of its `gfp` mask, order, zonelist, and cgroup.

4. ToyOS: failure stops at an empty bitmap

ToyOS `frame_alloc` returns 0 when every bit is set. Callers in `memory.c` treat 0 as failure: `next_table` returns NULL, and a mapper that cannot allocate a page-table page stops. There is no `swap_out`, no kswapd, no writeback, and no task to kill. The physical memory the bitmap manages is the memory `memory_init` marked free. When it is gone, it is gone. An `EXPECTED-FAULT` line in the Week 7 profile is a diagnosed fault in a kernel task. It is not an OOM kill.

That is a deliberate bound. It makes the frame allocator easy to test, and it makes a fault under memory pressure a hard failure instead of a reclaim story. The lesson to carry into Linux is the shape of the failure, not a claim that stopping is what a general-purpose kernel should do. Linux inserts reclaim between "the freelist missed" and "the allocation returns NULL or kills a task." ToyOS has no place to insert it.

5. The slow path: wake kswapd, then maybe reclaim in line

Day 5's fast path returns NULL from `get_page_from_freelist`. `__alloc_pages_slowpath` continues. When the flags include `ALLOC_KSWAPD`, it wakes every relevant kswapd:

```
/* tree adc218676eef (Linux 6.12)
 * path: mm/page_alloc.c
 * lines: 4345–4348
 * function: __alloc_pages_slowpath
 */
retry:
    /* Ensure kswapd doesn't accidentally go to sleep as long as we loop */
    if (alloc_flags & ALLOC_KSWAPD)
        wake_all_kswapds(order, gfp_mask, ac);
```

`wake_all_kswapds` (lines 3973–3989) walks the zonelist and calls `wakeup_kswapd` once per node. The thread that wakes is the allocator. The thread that runs is `kswapd` (`mm/vmscan.c` line 7165). It sets `PF_MEMALLOC | PF_KSWAPD` so its own tiny allocations do not recurse into reclaim, then loops on `kswapd_try_to_sleep`.

`prepare_kswapd_sleep` (lines 6711–6739) returns true, allowing sleep, when `pgdat_balanced` says so or when `kswapd_failures` has reached `MAX_RECLAIM_RETRIES`. `pgdat_balanced` (lines 6659–6680) walks zones bottom-up. The mark is `high_wmark_pages(zone)`, unless `NUMA_BALANCING_MEMORY_TIERING` is set in `sysctl_numa_balancing_mode`, in which case the mark is `promo_wmark_pages(zone)`. One zone that meets the mark balances the node for this test. A node that never meets it is left to direct reclaim after too many failures. The high watermark is the usual target. The tiering promotion watermark is the config-dependent substitute. Neither target is "all RAM in use."

Direct reclaim is the allocating context's own attempt:

```
/* tree adc218676eef (Linux 6.12)
 * path: mm/page_alloc.c
 * lines: 3940–3966
 * function: __alloc_pages_direct_reclaim
 */
    psi_memstall_enter(&pflags);
    *did_some_progress = __perform_reclaim(gfp_mask, order, ac);
    if (unlikely(!(*did_some_progress)))
        goto out;

retry:
    page = get_page_from_freelist(gfp_mask, order, alloc_flags, ac);
    if (!page && !drained) {
        unreserve_highatomic_pageblock(ac, false);
        drain_all_pages(NULL);
        drained = true;
        goto retry;
    }
```

`__perform_reclaim` reaches `try_to_free_pages` (`mm/vmscan.c` lines 6478–6518). The `scan_control` starts with `nr_to_reclaim = SWAP_CLUSTER_MAX`, `may_writepage = !laptop_mode`, `may_unmap = 1`, and `may_swap = 1`. Priority starts at `DEF_PRIORITY` and the scan grows more aggressive as priority falls. `throttle_direct_reclaim` can return early with the value 1 so the caller does not treat "we were throttled because of a fatal signal" as "reclaim found nothing" and then OOM. That return value is a progress flag, not a page.

`shrink_node` is the per-node body. With generations enabled at the root, it delegates to `lru_gen_shrink_node` and returns. Otherwise it calls `shrink_node_memcgs`, which shrinks the active/inactive lists. Inside `shrink_folio_list`, a dirty file folio can reach `pageout` (the call is at line 1367) and an anonymous folio can reach `add_to_swap` (lines 1247 and 1263). Those calls are the file exit and the anonymous exit. They still depend on `may_writepage`, the `gfp` mask, and whether the folio is already under writeback:

```
/* tree adc218676eef (Linux 6.12)
 * path: mm/vmscan.c
 * lines: 1088–1098
 * function: shrink_folio_list (folio loop)
 */
        if (lru_gen_enabled() && !ignore_references &&
            folio_mapped(folio) && folio_test_referenced(folio))
            goto keep_locked;

        folio_check_dirty_writeback(folio, &dirty, &writeback);
        if (dirty || writeback)
```

A referenced mapped folio can be kept when generations are on. A dirty folio is a candidate for writeback, not an instant free frame. The comment a few lines later (lines 1327–1338 in the same file) says only kswapd may write back filesystem folios in that part of the path. Direct reclaim of a dirty file page often kicks the flusher and moves on, because the allocating thread may be holding locks that make writeback unsafe. Read the branch you hit. The durable rule is that dirty data is written or kept, and a clean unmapped file folio can be discarded.

6. Writeback and swap are different exits

A file folio that is being written enters `__folio_start_writeback`:

```
/* tree adc218676eef (Linux 6.12)
 * path: mm/page-writeback.c
 * lines: 3113–3135
 * function: __folio_start_writeback
 */
void __folio_start_writeback(struct folio *folio, bool keep_write)
{
    long nr = folio_nr_pages(folio);
    struct address_space *mapping = folio_mapping(folio);

    VM_BUG_ON_FOLIO(folio_test_writeback(folio), folio);

    folio_memcg_lock(folio);
    if (mapping && mapping_use_writeback_tags(mapping)) {
        XA_STATE(xas, &mapping->i_pages, folio_index(folio));
        /* lock, load the slot, then: */
        folio_test_set_writeback(folio);
        xas_set_mark(&xas, PAGECACHE_TAG_WRITEBACK);
```

The address space is the file's. The xarray mark is how the flusher finds folios already under I/O. The folio remains a page-cache folio. When the I/O completes, writeback clears and the folio is clean. Reclaim may drop it later. `fsync` and `msync(MS_SYNC)` from Week 12 wait for this I/O from the user's side. Reclaim starts a similar I/O for a different reason: it wants the frame, or it wants the folio clean so a later scan can drop it.

An anonymous folio's backing store, if it gets one, is swap. `swap_writepage` (`mm/page_io.c` lines 241–279) takes the folio, drops the swap slot again if the folio can be freed without I/O (`folio_free_swap`), rejects the write when `arch_prepare_to_swap` fails, and may avoid disk entirely:

```
/* tree adc218676eef (Linux 6.12)
 * path: mm/page_io.c
 * lines: 261–280
 * function: swap_writepage
 */
   if (is_folio_zero_filled(folio)) {
       swap_zeromap_folio_set(folio);
       folio_unlock(folio);
       return 0;
   } else {
       swap_zeromap_folio_clear(folio);
   }
   if (zswap_store(folio)) {
       folio_unlock(folio);
```

A zero-filled folio can be recorded in a bitmap instead of written. `zswap_store` can accept the folio into a compressed pool. Either success returns before the block device. A folio that misses both continues to swap I/O. The PTE, or the swap map, remembers how to bring the bytes back. The next fault that needs them allocates a frame and reads the slot. That fault is a major fault. `getrusage` counts it in `ru_majflt`. Day 7 reads the counter.

`arch_prepare_to_swap` is the architecture hook. The comment above it says the architecture may have to preserve more than the page bytes, for example memory tags. The generic policy — "this anonymous folio may leave RAM" — stays in mm/. The extra bytes are an architecture obligation. x86-64 without tags has little to add. An AArch64 build with memory tagging has a real hook. The reclaim decision does not move into the ISA.

7. OOM is a locked, retried, constrained last step

After direct reclaim fails to help, `__alloc_pages_may_oom` runs for allocations that reached it:

```
/* tree adc218676eef (Linux 6.12)
 * path: mm/page_alloc.c
 * lines: 3597–3615
 * function: __alloc_pages_may_oom
 */
   page = get_page_from_freelist((gfp_mask | __GFP_HARDWALL) &
                     ~__GFP_DIRECT_RECLAIM, order,
                     ALLOC_WMARK_HIGH|ALLOC_CPUSET, ac);
   if (page)
       goto out;

   if (current->flags & PF_DUMPCORE)
       goto out;
   /* The OOM killer will not help higher order allocs */
   if (order > PAGE_ALLOC_COSTLY_ORDER)
       goto out;
```

The freelist is tried again at the high watermark, without direct reclaim, under `oom_lock`. A page found here means someone else already freed memory. A core dump skips the killer. A costly order skips the killer. The function then calls `out_of_memory`.

`out_of_memory` (`mm/oom_kill.c` lines 1107–1175) returns immediately when the killer is disabled. A notifier that freed pages can satisfy a global OOM without a kill. A task that is already exiting is marked as the victim so it can finish and release its mm. An allocation without `__GFP_FS` returns true without killing, except inside a memcg, because the killer's own work may need filesystem locks the caller cannot take. `constrained_alloc` records whether the failure is a cgroup, a cpuset, a memory policy, or the whole system. `select_bad_process` scores candidates. A kill sends `SIGKILL`. The victim's pages come back as the task exits, which is asynchronous. The allocating thread retries. It does not receive the victim's pages as a return value.

`pagefault_out_of_memory` (line 1184) exists because a fault can fail its allocation too. The comment says the global killer is the allocation context's job, and this hook exists so the memcg case has a safe place to run. A user-space `SIGSEGV` from an unmapped address is a different event. It is not the OOM killer. Day 7's child fault is that different event, on purpose.

8. Execution trace: a slow order-0 allocation

1. `__alloc_pages_noprof` misses `get_page_from_freelist` at the low watermark. `__alloc_pages_slowpath` runs.
2. `wake_all_kswapds` wakes the node thread if `ALLOC_KSWAPD` is set. kswapd enters `balance_pgdat` and `shrink_node` until `pgdat_balanced` sees a zone at the high watermark, or until it gives up.
3. The allocator tries the freelist again. On another miss, and if the `gfp` mask allows it, `__alloc_pages_direct_reclaim` calls `try_to_free_pages`.
4. `shrink_node` either calls `lru_gen_shrink_node` or walks the inactive lists. A clean file folio that is unmapped is freed back toward the buddy. A dirty file folio is handed toward `pageout` and `__folio_start_writeback`, mostly by waking the flusher when the allocator

itself must not start filesystem I/O. A cold anonymous folio is passed toward `add_to_swap` and `swap_writepage`, which may end in the zeromap, in zswap, or in swap I/O.

5. `drain_all_pages` pulls per-CPU lists back to the buddy so a page that was free but cached on another CPU becomes visible. The freelist is tried again.
6. If the order is above `PAGE_ALLOC_COSTLY_ORDER`, the path can fail the allocation without `out_of_memory`. Compaction may have been tried to build a high-order block from free order-0 pages. Failure here is fragmentation plus pressure, and the caller sees NULL.
7. If the order is small and the other gates pass, `__alloc_pages_may_oom` takes `oom_lock`, retries at the high watermark, and calls `out_of_memory`. A chosen task receives `SIGKILL`. The allocator retries. The original caller either receives a page or, if the kill did not free a usable page under the same constraints, continues the slow path's retry logic until it succeeds or gives up under `__GFP_NORETRY` or a fatal signal.

Step 7 is rare on a quiet machine. Manufacturing it in a lab is how people lose unrelated work. Section 10 reads the counters that would move if it happened, and it does not make it happen.

9. Two pictures

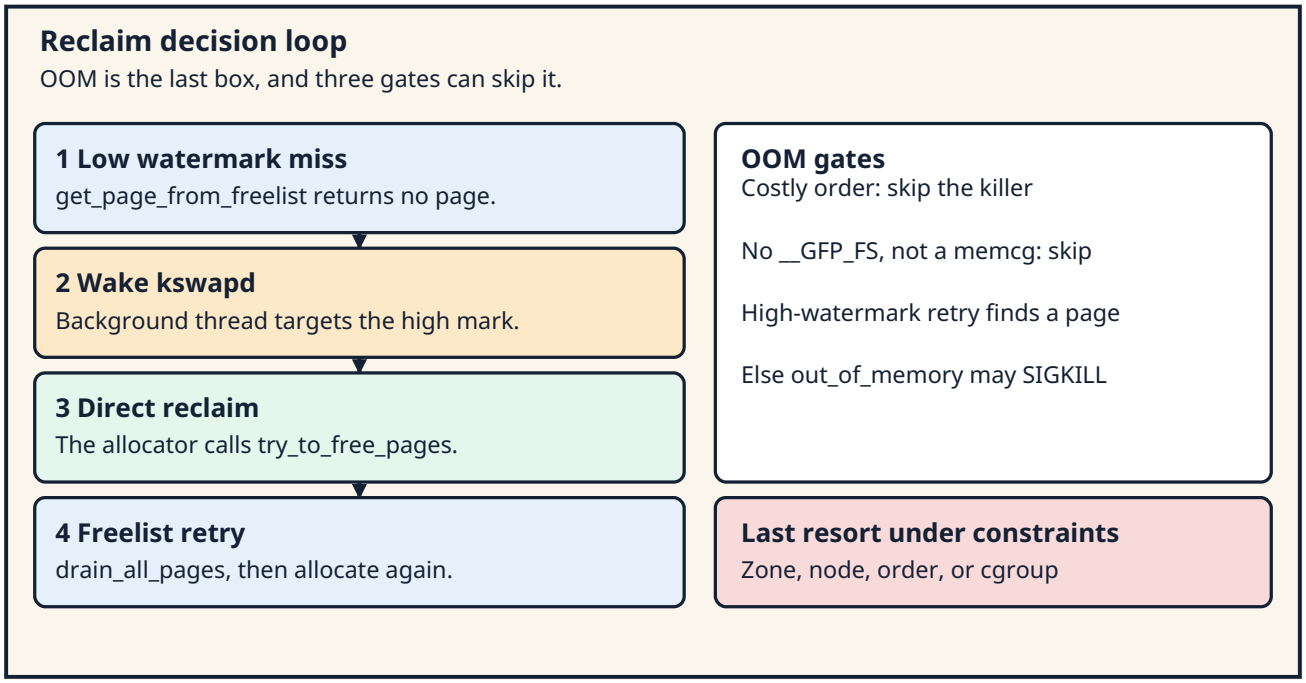


Figure 1. kswapd and direct reclaim are different contexts. The OOM killer sits after they fail, and several gates can skip it.

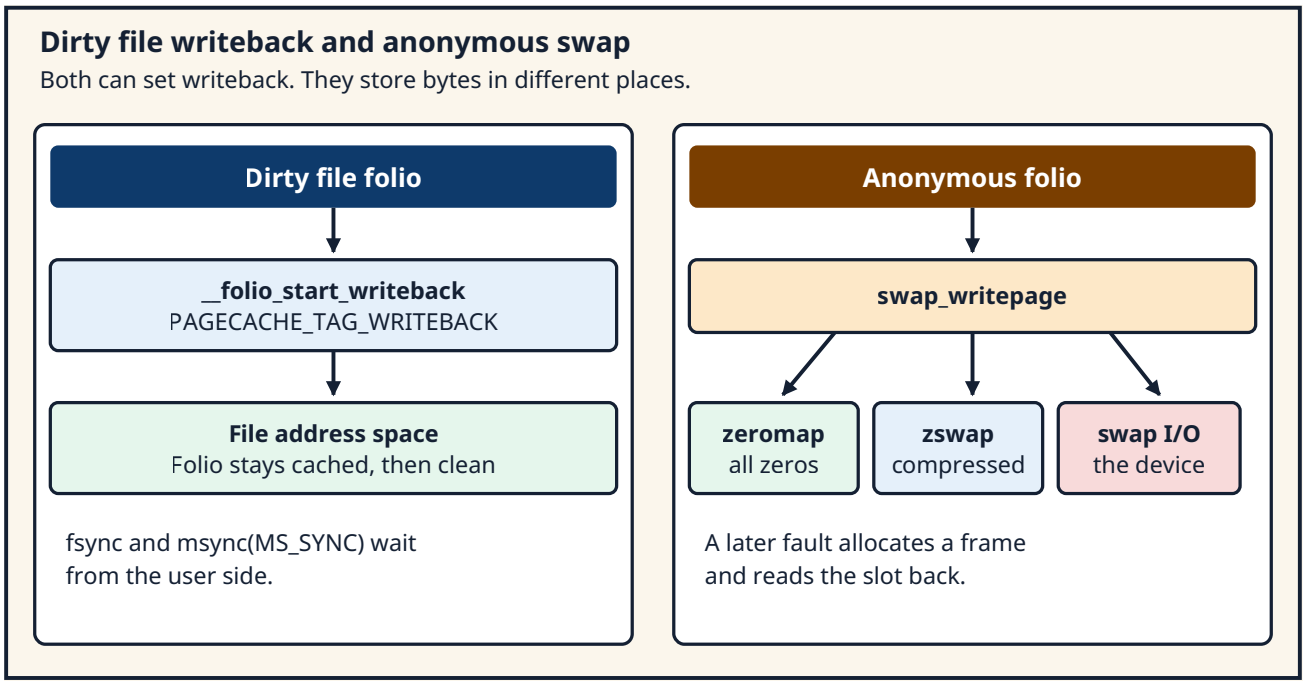


Figure 2. Both paths set writeback state when I/O starts. They write different stores, and a zero anonymous folio may write neither.

10. A lab that reads counters and creates no pressure

The wrapper is `../challenges/day-006-reclaim.sh`. It builds `projects/linux-vm` `reclaim_lab` and runs a teaching LRU on four frames and two swap slots. The pass line is a simulator result: one dirty file page written back, one clean file page dropped, two anonymous pages swapped, then a refusal that prints `simulated_oom_choice=task_b` and `oom_kills_host_process=no`. That policy is not `shrink_folio_list`, and it is not memcg OOM. The host's `pswpin`, `pswpout`, and OOM killer must stay untouched.

The program below is the host observation that stays inside that limit. Read the counters. Do not allocate a large buffer, do not start a stress tool, and do not tune `vm.overcommit_memory`.

```
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <string.h>

static void show_match(const char *path, const char *key)
{
    FILE *fp = fopen(path, "re");
    char line[256];

    if (!fp) {
        perror(path);
        return;
    }
    while (fgets(line, sizeof line, fp)) {
        if (strncmp(line, key, strlen(key)) == 0)
            fputs(line, stdout);
    }
    fclose(fp);
}

int main(void)
{
    const char *mem[] = {
        "MemTotal:", "MemFree:", "MemAvailable:",
        "Buffers:", "Cached:", "SwapTotal:", "SwapFree:", "Dirty:"
    };
    const char *vm[] = {
        "pgscan_kswapd ", "pgsteal_kswapd ",
        "pgscan_direct ", "pgsteal_direct ",
        "oom_kill ", "nr_dirty ", "pswpout "
    };
    unsigned i;

    puts("== meminfo");
    for (i = 0; i < sizeof mem / sizeof mem[0]; i++)
        show_match("/proc/meminfo", mem[i]);
    puts("== vmstat");
    for (i = 0; i < sizeof vm / sizeof vm[0]; i++)
        show_match("/proc/vmstat", vm[i]);
    puts("Record the numbers. Do not try to move oom_kill.");
    return 0;
}
```

`MemAvailable` is an estimate the kernel publishes for "how much could be given out without going into deep reclaim." It is not a watermark, and it is not a promise that an allocation of that size will succeed. `Cached` counts page-cache pages that are often reclaimable and still in use. `pgscan_kswapd` and `pgscan_direct` tell you which context has been scanning. A zero `oom_kill` on a desktop that is working is the expected reading. A non-zero value is a historical count since boot, so quote the number and the boot, and do not treat it as a live emergency by itself.

Predict that this program will not change `oom_kill`. Predict that `Cached` will be non-zero on a machine that has read files. If a key is absent, record its absence. The strings above match this 6.12 tree's `mm/vmstat.c` counter texts (`pgscan_kswapd`, `pgsteal_kswapd`, and the direct-reclaim counterparts, around lines 1304–1310 and 1331). If your running kernel is older or newer than the tree you read, a missing key is a version fact, not a failed lab.

11. Mastery: Explain · Draw · Observe · Build

The exercise is `../challenges/day-006-reclaim.sh`.

Explain

- What Linux 0.12 does with a clean PTE and with a dirty private PTE inside `try_to_swap_out`.
- Why the buffer cache and the unified page cache are different designs.
- When `shrink_node` calls `lru_gen_shrink_node`, and which three `CONFIG_` and static-key states `lru_gen_enabled` covers.
- What `pgdat_balanced` compares against, including the tiering branch.
- Why `swap_writepage` and `__folio_start_writeback` are different exits, and where the zeromap and zswap sit.
- Where `shrink_folio_list` calls `pageout` and `add_to_swap`.
- Which checks in `__alloc_pages_may_oom` and `out_of_memory` return without killing a task.

Draw

Draw Figure 1 as a loop: watermark, kswapd, freelist retry, direct reclaim, freelist retry, OOM gate. On the OOM gate, write three skips: costly order, missing `__GFP_FS` outside a memcg, and a high-watermark retry that finds a page. Draw Figure 2 under the reclaim box.

Observe

Run the wrapper. Record the simulator counts and `oom_kills_host_process=no`. Separately record `MemAvailable`, `Cached`, `SwapFree`, `pgscan_kswapd`, `pgscan_direct`, and `oom_kill` from the host, and say the host counters were not the target of the simulator. Record whether the running config enables `CONFIG_LRU_GEN` if you can read it. If you cannot, say so.

Build

Write a one-page trace of one order-0 `GFP_KERNEL` allocation that misses the low watermark, using `wake_all_kswapds`, `try_to_free_pages`, `shrink_node`, and `__alloc_pages_may_oom`. End with the sentence you will use when someone says the OOM killer runs because RAM is full: the killer runs for an allocation that failed reclaim under its own order, mask, zonelist, and cgroup, and several of those constraints return before any task is chosen.

Now check your understanding

You have finished the theory for this section. Before moving on, use the quick check below to find anything that still feels uncertain, then reinforce the idea with the practical lab when one is provided.

Quick theory check

Answer from memory first. Explanations appear only after you submit your choices.

- Start the interactive theory check
- View quiz source on GitHub

Put it into practice — Reclaim

Format: shell lab

What you will do: set -euo pipefail

Predict the important result first, then run the lab and explain any difference between prediction and observation.

- Open the lab file

Copy and run:

```
bash 13-virtual-memory-faults-and-reclaim/challenges/day-006-reclaim.sh
```

- View lab source on GitHub

Day 7 — Virtual-Memory Observation and a Debugging Report

Target time: approximately 3 hours

> A virtual-memory bug becomes an explanation when a symptom, a source path, and a measurement name the same layer. Counters from a different layer are evidence about that layer, and they stay labeled as such.

The engineering tree is Linux 6.12, tag `v6.12`, commit `adc218676eef25575469234709c2d87185ca223a`, at `/home/jd/Documents/linux-6.12`. Linux 0.12 page numbers are printed pages of Zhao Jiong, Chinese Revision 5 (2019). Today's lab faults one child on purpose and reads counters. It does not create memory pressure.

1. Syntax: a text line, a wait status, and a residency byte

`/proc` files are text. The reader splits on a key and treats the value as a snapshot. A snapshot can tear: `task_mem` says the high-water marks are updated only when a total is about to fall, and that a collector can observe an inconsistent pair. You record the instant and you avoid adding numbers from two reads as if they were one sample.

```
char line[256];
FILE *fp = fopen("/proc/self/status", "re");
while (fgets(line, sizeof line, fp)) {
    if (strncmp(line, "VmRSS:", 6) == 0)
        fputs(line, stdout);
}
```

`strncmp` on the key is the whole parser this lab needs. The value is kilobytes because the kernel shifted a page count by `PAGE_SHIFT - 10` before printing. A program that treats the field as bytes is off by 1024. `/proc/self` is this process. `/proc/vmstat` is the machine. `perf stat` around one command is that command, plus whatever the tool could not open. A `SKIP` line is a result. It is not a zero.

A child that faults does not return an `errno` to the parent. It stops with a signal. `waitpid` fills an `int`. The macros in `<sys/wait.h>` are the contract:

```
int status = 0;
waitpid(pid, &status, 0);
if (WIFSIGNALED(status)) {
    int sig = WTERMSIG(status);
    /* SIGSEGV is the usual unmapped-address signal */
}
```

`WIFEXITED` and `WEXITSTATUS` describe a child that called `_exit`. `WIFSIGNALED` and `WTERMSIG` describe a child the kernel delivered a signal to. Using the exit macros on a signal status yields a number that is not the signal. The report names which macro matched.

`mincore` fills one byte per page. The syscall checks alignment and the user range:

```
/* tree adc218676eef (Linux 6.12)
 * path: mm/mincore.c
 * lines: 232–247
 * function: mincore
 */
SYSCALL_DEFINE3(mincore, unsigned long, start, size_t, len,
        unsigned char __user *, vec)
{
    long retval;
    unsigned long pages;
    unsigned char *tmp;

    start = untagged_addr(start);

    /* Check the start address: needs to be page-aligned.. */
    if (start & ~PAGE_MASK)
        return -EINVAL;

    /* ..and we need to be passed a valid user-space range */
    if (!access_ok((void __user *) start, len))
        return -ENOMEM;
```

Inside the walk, a present PTE stores `1` in the vector (`mm/mincore.c` lines 128–129). An anonymous hole stores `0` (lines 84–86). A swap entry may consult the swap cache when `CONFIG_SWAP` is on (lines 130–142). Userspace is entitled to the low bit: 1 means resident in the sense this syscall reports, 0 means it does not. Higher bits are not a portable field. A byte of `1` after you have written the page, and a byte of `0` before a write on a fresh anonymous mapping, is the observation Day 4's zero-page and private-folio split predicts for a write. A read can be resident through the zero page, so a `1` after a read does not by itself prove a private folio. Say which access you performed.

`getrusage` fills a `struct rusage`. The faults the kernel adds are `ru_minflt` and `ru_majflt` (`kernel/sys.c` lines 1836–1837 for the thread-group path). `ru_maxrss` is converted from pages to kilobytes at line 1875: `maxrss * (PAGE_SIZE / 1024)`. Minor faults include faults the kernel satisfied without submitting I/O. Major faults include faults that waited for I/O, such as a page read back from swap or from a file that was not in the cache. The man page and this structure are the API. The decision of which fault increments which counter sits in the fault handler. Quote the counter you read, and name the access that you believe caused it.

2. Linux 0.12 first: one screen of `mem_map` counts

Linux 0.12 has no `/proc/pid/maps`. The observation tool is `show_mem` in `mm/memory.c`, printed page 899. `mem_init` marks every `mem_map` byte `USED` and then clears the bytes that cover the main-memory area. `show_mem` walks that array, counts free and shared pages, and prints them. The book says the function runs from `chr_drv/keyboard.S` when Shift+Scroll Lock is pressed. A fault's other evidence is the address the CPU saved and the `printk` from the fault path or from `oom()`.

The stable idea is a kernel-side census of frames: how many are free, how many are shared, taken from the same array the allocator updates. The historical details are the keyboard trigger, the byte array, and the absence of any per-process address-space listing. You cannot ask 0.12 which virtual range a process owns, because the task's memory is the segment bases plus the page tables, and nothing formats them as text for a shell. The modern census of

frames is `/proc/vmstat` and `/proc/zoneinfo`. The modern census of one process is `/proc/pid/maps`, `smaps`, and `status`. Day 5 and Day 6 own the frame and reclaim counters. Today owns the process view, and the habit of saying which view a number came from.

3. Evolution: from a keyboard dump to three files and two syscalls

A process address space became a list of VMAs. Once that list existed, printing it was more useful than printing only `mem_map`. `maps` is the list: start, end, permissions, offset, device, inode, and a name. `smaps` is the same list plus resident, proportional, dirty, and swap accounting per VMA. `status` is the mm rolled into a few totals: `VmSize`, `VmRSS`, `RssAnon`, `RssFile`, `VmData`, `VmStk`. The split matches the questions. "Did the mapping appear?" is `maps`. "Did the pages become resident, and are they private?" is `smaps`. "How big is the process, approximately?" is `status`. Adding them together from different instants invents a total nobody computed.

`mincore` and `getrusage` are syscalls rather than files. `mincore` answers a question about a range the caller already has mapped, in the caller's own mm. `getrusage` answers a question about fault counts and CPU time accumulated for the process or its children. `perf stat`, when the tool is installed, can count the same faults from outside by asking the performance counters for `page-faults`. It is optional. A missing `perf` binary, or a `SKIP` from the wrapper, is recorded as "tool absent." It is not a failed measurement of `/proc`, and it is not proof that no fault happened.

The files are a Linux interface. They are not POSIX. A lab report that says "the operating system" when it means `/proc/self/smaps` has named the wrong layer. The source that prints the file is the layer below the interface. The number you read on this boot is the layer above it. Using a machine-wide `pgfault` to explain one load in this process mixes those scopes.

4. ToyOS: a serial line instead of `/proc`

ToyOS has no procfs. `memory_init` in `projects/toyos/src/memory.c` prints `free_frames` on the serial port after it has marked the available ranges. That count is the bitmap census, the same role as 0.12 `show_mem`, with a bit per frame and a boot-time call instead of a keyboard chord. Week 7's injected fault prints an `EXPECTED-FAULT` line when the profile asks for it. That line is a diagnosed fault in a kernel task. It is not a demand-paged user process, and it is not an OOM kill.

A fault in ToyOS is an exception the kernel prints through its own handler. Week 6 built that path. There is no parent `waitpid`, because the teaching profile's user process does not have a POSIX parent collecting a signal code. The report you write today needs a parent, a status word, and a file. ToyOS gives you the idea of "print the allocator's own counter from inside the kernel." Linux gives you that counter already printed, plus a second set of counters that describe one mm. The comparison sentence for this week is concrete: ToyOS still has no VMA list, no buddy, no swap, and no OOM killer. Keep the serial number and the `/proc` number in different columns when you compare the two systems.

5. Linux 6.12: who prints `maps`, `smaps`, and `status`

`/proc/pid/maps` is `proc_pid_maps_operations`. The show function calls `show_map_vma` (`fs/proc/task_mmu.c` lines 325–358). The header is one line of addresses and permission letters:

```
/* tree adc218676eef (Linux 6.12)
 * path: fs/proc/task_mmu.c
 * lines: 304–321
 * function: show_vma_header_prefix
 */
static void show_vma_header_prefix(struct seq_file *m,
                   unsigned long start, unsigned long end,
                   vm_flags_t flags, unsigned long long pgoff,
                   dev_t dev, unsigned long ino)
{
    seq_setwidth(m, 25 + sizeof(void *) * 6 - 1);
    seq_put_hex_ll(m, NULL, start, 8);
    seq_put_hex_ll(m, "-", end, 8);
    seq_putc(m, ' ');
    seq_putc(m, flags & VM_READ ? 'r' : '-');
    seq_putc(m, flags & VM_WRITE ? 'w' : '-');
    seq_putc(m, flags & VM_EXEC ? 'x' : '-');
    seq_putc(m, flags & VM_MAYSHARE ? 's' : 'p');
    seq_put_hex_ll(m, " ", pgoff, 8);
    seq_put_hex_ll(m, " ", MAJOR(dev), 2);
    seq_put_hex_ll(m, ":", MINOR(dev), 2);
    seq_put_decimal_ull(m, " ", ino);
    seq_putc(m, ' ');
}
```

`VM_MAYSHARE` selects the `s` or `p` letter. That letter is a sharing permission, the distinction Week 12 Day 6 drew between a shared file mapping and a private one. It is not the residency bit. A line with `rw-p` and no pathname is an anonymous private VMA. `[heap]` is the name `get_vma_name` supplies when `vma_is_initial_heap` is true (lines 287–289). `[stack]` is the initial stack. A mapping the allocator created with `mmap` may have neither name. Absence of `[heap]` means this VMA is not the initial heap. It does not mean the process has no anonymous memory.

`smaps` reprints that header and then `Size`, which is `vm_end - vm_start` (line 1147), and the block from `__show_smap`:

```
/* tree adc218676eef (Linux 6.12)
 * path: fs/proc/task_mmu.c
 * lines: 1098–1116
 * function: __show_smap
 */
    SEQ_PUT_DEC("Rss:            ", mss->resident);
    SEQ_PUT_DEC(" kB\nPss:            ", mss->pss >> PSS_SHIFT);
    SEQ_PUT_DEC(" kB\nPss_Dirty:      ", mss->pss_dirty >> PSS_SHIFT);
    if (rollup_mode) {
        SEQ_PUT_DEC(" kB\nPss_Anon:       ",
            mss->pss_anon >> PSS_SHIFT);
        SEQ_PUT_DEC(" kB\nPss_File:       ",
            mss->pss_file >> PSS_SHIFT);
        SEQ_PUT_DEC(" kB\nPss_Shmem:      ",
            mss->pss_shmem >> PSS_SHIFT);
    }
```

`Rss` is pages resident in this VMA, scaled to kilobytes by `SEQ_PUT_DEC`. `Pss` is the proportional share: a page shared by four processes contributes a fraction to each. The rollup mode adds the anon, file, and shmem split, and those three lines are meaningful for `smaps_rollup`. On a per-VMA `smaps` the kernel still prints `Rss` and `Pss` for that VMA. `Swap` in the same function (line 1130) is pages that have been swapped out and are still charged to the VMA. A zero `Swap` field means this snapshot shows no swapped pages for that range. It does not mean the system has no swap device. `SwapTotal` in `meminfo` is the device. The VMA field is the process.

`status` comes from `task_mem` (line 34). The RSS line is the sum of three mm counters:

```
/* tree adc218676eef (Linux 6.12)
 * path: fs/proc/task_mmu.c
 * lines: 39–72
 * function: task_mem
 */
    anon = get_mm_counter(mm, MM_ANONPAGES);
    file = get_mm_counter(mm, MM_FILEPAGES);
    shmem = get_mm_counter(mm, MM_SHMEMPAGES);
    hiwater_rss = total_rss = anon + file + shmem;
    SEQ_PUT_DEC("VmPeak:\t", hiwater_vm);
    SEQ_PUT_DEC(" kB\nVmSize:\t", total_vm);
    SEQ_PUT_DEC(" kB\nVmLck:\t", mm->locked_vm);
    SEQ_PUT_DEC(" kB\nVmPin:\t", atomic64_read(&mm->pinned_vm));
    SEQ_PUT_DEC(" kB\nVmHWM:\t", hiwater_rss);
    SEQ_PUT_DEC(" kB\nVmRSS:\t", total_rss);
    SEQ_PUT_DEC(" kB\nRssAnon:\t", anon);
    SEQ_PUT_DEC(" kB\nRssFile:\t", file);
    SEQ_PUT_DEC(" kB\nRssShmem:\t", shmem);
    SEQ_PUT_DEC(" kB\nVmData:\t", mm->data_vm);
```

`VmSize` is `mm->total_vm`, virtual pages. `VmRSS` is resident pages across anonymous, file, and shmem. `VmData` is `mm->data_vm`, which Day 4's `do_brk_flags` increments when the brk VMA grows. A `brk` that extends the heap can move `VmData` and `VmSize` before the first write moves `RssAnon`. That lag is the measurement form of "the VMA exists and the folio does not yet." `VmHWM` is a high-water mark, and the comment at lines 43–48 says it can disagree with the current total because it is refreshed lazily. Quote `VmRSS` for "now" and `VmHWM` for "the peak this mm has recorded."

The fault handler's own accounting is a different function. `handle_mm_fault` calls `mm_account_fault` before it returns (`mm/memory.c` line 6103). That updates kernel counters. It does not format `/proc/self/status`. `/proc/vmstat` keys such as `pgfault` and `pgmajfault` are machine totals built from those counters. `pswpin` and `pswpout` are swap traffic for the machine. Reading them after a lab is an observation of the machine. It is not a request to move them.

`/proc/vmstat` is the machine, not the process. Day 6 listed `pgscan_kswapd`, `pgscan_direct`, and `oom_kill`. A debugging report that pastes those next to `VmRSS` labels each column. A climb in `pgscan_direct` during a stall is a reclaim fact. A climb in `VmRSS` after a write is a residency fact. One does not explain the other unless the trace shows the allocation entered the slow path.

6. Three layers, and the fault the child is allowed to take

Keep the layers in order when you write.

The source layer is the tree. `show_vma_header_prefix`, `__show_smap`, `task_mem`, `mincore`, `getrusage`, and `mm_account_fault` are functions you can cite with a path and a line. A report that says "Linux tracks RSS" without a function has not reached this layer.

The API layer is what a program may call or read. `mincore` returns `-EINVAL` for an unaligned start. `getrusage` with `RUSAGE_SELF` fills the calling process. `waitpid` fills a status word. `/proc/self/maps` is a file the process may open. These contracts are in the man pages and in the syscall checks. They are stable relative to the source only for the kernel you are running. A field this tree prints can be absent on a different version. The lab records the running `uname -r` beside the tree id.

The measurement layer is one read. It has a timestamp, a pid, and a value. It can be torn. It can be zero because the event has not happened, or zero because the counter has another name on this kernel. The report says which.

The deliberate fault stays inside a child so the parent can record the status and exit. The child maps one anonymous page, writes it, unmaps it, and writes again. The second write has no VMA. The user fault path finds no VMA and the process receives `SIGSEGV`. That signal is not `pagefault_out_of_memory`. Nothing in the child allocates until failure, and the parent allocates only the small buffers in section 8. `oom_kill` is not the subject of this experiment. If it changes, you were not looking at this child.

7. Architecture lens: the signal is portable, the fault register is not

`SIGSEGV` for an unmapped user address is the Linux userspace result on x86-64, AArch64, and RISC-V. The register the kernel read to classify the fault is not portable. x86-64 supplies a page-fault error code and the fault address in `CR2`. AArch64 supplies an ESR_ELx syndrome and a FAR. RISC-V supplies `scause` and `stval`. The generic mm code turns those into a `vm_fault` and a signal. A report written from userspace cites `WTERMSIG` and the address the child was told to touch. A report written from a kernel stack cites the architecture register and then the generic function. Mixing "the ESR" into an x86-64 lab, or "CR2" into an AArch64 lab, names a machine you are not running. Linux 0.12's console fault address is the 80386 `CR2` of that kernel.

`mincore`'s `untagged_addr` at the start of the syscall matters on architectures that store a tag in the high bits of a pointer. On a machine without tags it is an identity. The residency byte means the same thing either way: what the walk found in this mm. The page size that divides `len` is `PAGE_SIZE` of the running kernel. A vector indexed as if pages were 4 KiB on a 16 KiB AArch64 granule counts the wrong number of entries. `sysconf(_SC_PAGESIZE)` is the userspace query. Use it.

`perf` event names are a tool convention on top of the architecture's performance counters. `page-faults` is a software event the kernel emits. It is available when the tool and the kernel agree. It is optional in this lab because the parent already learns that a fault became `SIGSEGV` from `waitpid`, and it learns minor and major counts from `getrusage`. `perf` is a third view of the same event class when you have it.

8. A parent, a child, and a page that is resident and then gone

The wrapper is `../challenges/day-007-vm-debug-report.sh`. It writes `projects/linux-vm/build/week13-report.txt`, runs the course `debug_lab`, and may skip `perf` or `vmstat`. `debug_lab` checks one anonymous mapping, a zero-fill, a store, and an `smaps` Rss of one page. It prints `pressure_induced=no`. The limit line is the contract: no host reclaim, no host swap, and no OOM kill. A `WEEK13 PASS` means the markers matched. It does not mean the report explained `shrink_folio_list`.

The program below is the child fault that kind of report is about. Compile it with `cc -std=c17 -Wall -Wextra -Werror`. Run it once. Do not wrap it in a loop that forks thousands of children.

```
#define _DEFAULT_SOURCE
#include <errno.h>
#include <signal.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/mman.h>
#include <sys/resource.h>
#include <sys/wait.h>
#include <unistd.h>

static void child_fault(void)
{
    long page = sysconf(_SC_PAGESIZE);
    volatile char *p = mmap(NULL, (size_t)page, PROT_READ | PROT_WRITE,
                MAP_PRIVATE | MAP_ANONYMOUS, -1, 0);
    unsigned char vec = 0xff;

    if (p == MAP_FAILED)
        _exit(2);
    if (mincore((void *)p, (size_t)page, &vec) == 0)
        dprintf(1, "child mincore before write 0x%02x\n", vec);
    p[0] = 1;
    vec = 0xff;
    if (mincore((void *)p, (size_t)page, &vec) == 0)
        dprintf(1, "child mincore after write 0x%02x\n", vec);
    if (munmap((void *)p, (size_t)page) != 0)
        _exit(3);
    p[0] = 2;
    _exit(0);
}

static void show_key(const char *path, const char *key)
{
    FILE *fp = fopen(path, "re");
    char line[320];

    if (!fp)
        return;
    while (fgets(line, sizeof line, fp)) {
        if (strncmp(line, key, strlen(key)) == 0)
            fputs(line, stdout);
    }
    fclose(fp);
}

int main(void)
{
    pid_t pid = fork();
    int status = 0;
    struct rusage ru;

    if (pid < 0)
        return 1;
    if (pid == 0)
        child_fault();
    if (waitpid(pid, &status, 0) < 0)
        return 1;
    printf("wait status=0x%x signaled=%d exit=%d\n", status,
           WIFSIGNALED(status) ? WTERMSIG(status) : -1,
           WIFEXITED(status) ? WEXITSTATUS(status) : -1);
```

```
    if (getrusage(RUSAGE_SELF, &ru) == 0) {
        printf("self minflt=%ld majflt=%ld maxrss=%ld\n",
               ru.ru_minflt, ru.ru_majflt, ru.ru_maxrss);
    }
    if (getrusage(RUSAGE_CHILDREN, &ru) == 0) {
        printf("children minflt=%ld majflt=%ld maxrss=%ld\n",
               ru.ru_minflt, ru.ru_majflt, ru.ru_maxrss);
    }
    show_key("/proc/self/status", "VmSize:");
    show_key("/proc/self/status", "VmRSS:");
    show_key("/proc/self/status", "RssAnon:");
    show_key("/proc/self/status", "VmData:");
    show_key("/proc/self/smaps_rollup", "Rss:");
    show_key("/proc/self/maps", "[heap]");
    show_key("/proc/self/maps", "[stack]");
    puts("Optional: perf stat -e page-faults -- ./this-program");
    puts("Skip perf when the binary is absent. Create no pressure.");
    return WIFSIGNALED(status) && WTERMSIG(status) == SIGSEGV ? 0 : 1;
}
```

Predict before you run. The child prints a `mincore` byte of `0` before the write if the page is not resident yet, and `1` after the write. The second store does not return. `waitpid` reports `WIFSIGNALED` and `SIGSEGV`. `RUSAGE_CHILDREN` includes the child's faults in the parent's child-rusage; `RUSAGE_SELF` does not include them. `oom_kill` in `/proc/vmstat` stays where Day 6 left it.

If `mincore` before the write is already `1`, say so. A kernel that populated the anonymous page, or a read the runtime inserted, is a measurement. It changes the sentence about the zero page. It does not change the `SIGSEGV` after `munmap`. If `smaps_rollup` is missing, say so and use `status` alone.

`perf`, if you choose to run it, wraps one execution: `perf stat -e page-faults -- ./program`. Record the count and the tool version. A failure to exec `perf` ends the optional step. The required evidence is `waitpid`, `mincore`, `getrusage`, and the three `/proc` files.

9. The report

Write one page with these headings. Fill them from the run.

1. **Symptom.** The child stopped. State `WTERMSIG` and the address relationship: the store used the pointer `munmap` had just released.
2. **Layer.** This is a missing VMA, observed from userspace. It is not a buddy failure, a reclaim stall, or an OOM kill.
3. **Source.** Cite `show_vma_header_prefix` for the maps line that would have named the range, and the no-VMA fault path. Cite `task_mem` for the `VmRSS` line you quote. Cite `mincore` lines 128–129 for the byte `1`, and lines 84–86 for a byte `0` on an anonymous hole. Cite `mm_account_fault` only if you are talking about the kernel counter, not about the text file.
4. **API.** `mmap`, `mincore`, `munmap`, `waitpid`, `getrusage`, and the `/proc` paths. Note page alignment and `RUSAGE_SELF` versus `RUSAGE_CHILDREN`.
5. **Measurement.** Paste the two mincore bytes, the wait line, the two rusage lines, `VmRSS`, `RssAnon`, and one maps line. Paste `uname -r`. Paste the tree id `adc218676eef` when you opened the sources. If the wrapper printed `SKIP` for `perf`, paste that line.

6. **What would change the conclusion.** A `WIFEXITED` status would mean the second store returned, so the unmap did not happen. A changed `oom_kill` would mean something else on the machine invoked the killer; it would still not make this `SIGSEGV` an OOM. A `mincore` of `1` before any write would retire the "first touch populates" sentence for this run and would leave the unmap sentence standing.

That last heading is the one that makes the report a debug report. A stack of numbers without a falsifier is a log.

10. Symptom to the layer that can cause it

Use the same headings when the symptom is not the one you injected.

A `maps` range that grows while `RssAnon` stays flat is a VMA without new residency. The source is `do_brk_flags` or `mmap_region` updating `total_vm` before `do_anonymous_page` allocates. The measurement is a pair of `status` reads around a `brk` or `mmap` that has not been written. Day 4 is that story.

A `maps` range that stays put while `RssAnon` grows is a write into an existing VMA. The source is `do_anonymous_page` and `alloc_anon_folio`. The measurement is `mincore` flipping from 0 to 1, or `RssAnon` moving after a store and not after the `mmap` alone.

A stall with `pgscan_direct` climbing is the allocating context inside `try_to_free_pages`. `pgscan_kswapd` climbing is the node thread. Both can move on a busy machine while your process is fine. Attribute a stall to direct reclaim when the climb lines up with the stall and the allocator in question can enter `__alloc_pages_direct_reclaim`. Day 6 is that story. This lab does not produce the stall.

A high-order allocation returning failure with `oom_kill` unchanged matches `__alloc_pages_may_oom` skipping orders above `PAGE_ALLOC_COSTLY_ORDER`. The buddy can be fragmented. `buddyinfo` from Day 5 is the file. Killing a task would not assemble a contiguous block, which is why the source skips the killer.

`SIGBUS` with a VMA still present, on a file mapping whose offset is past `i_size`, is the case Week 12 Day 6 separated from a missing VMA. Check the signal. `SIGBUS` and `SIGSEGV` are different answers. `WTERMSIG` tells you which one the child received.

11. Two pictures

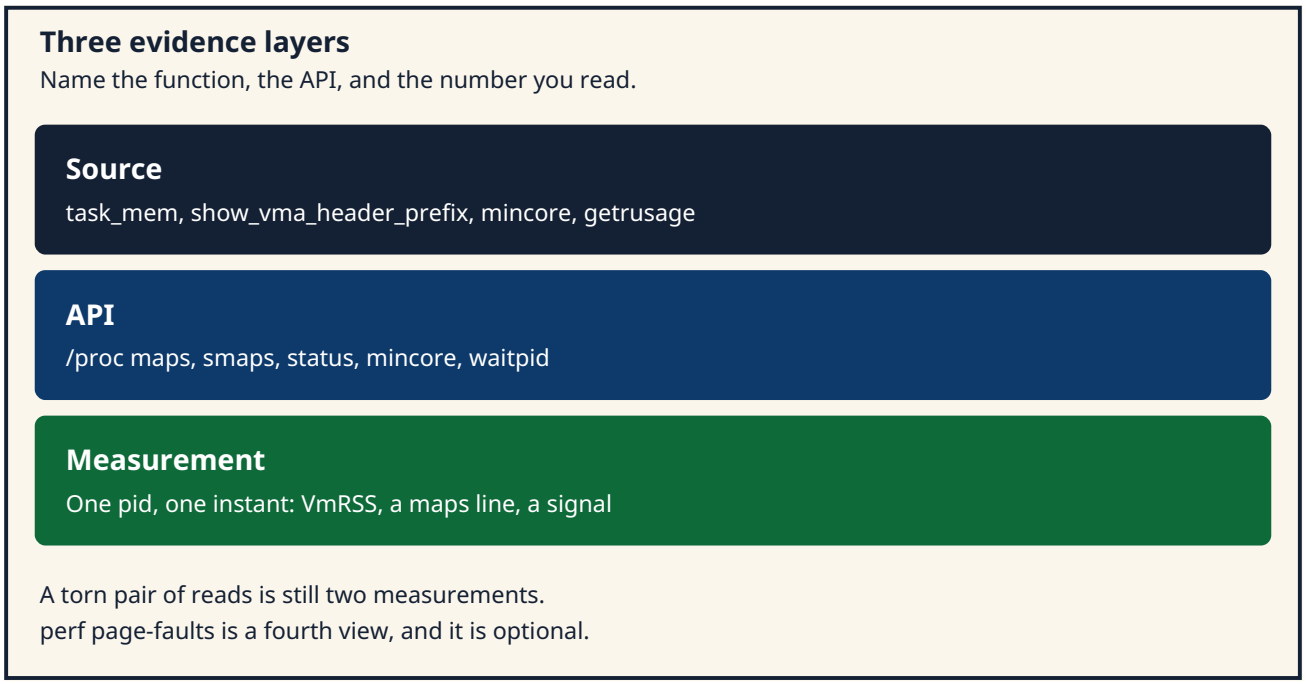


Figure 1. Cite a function, then the API you called, then the number you actually read. The number is about one pid at one time.

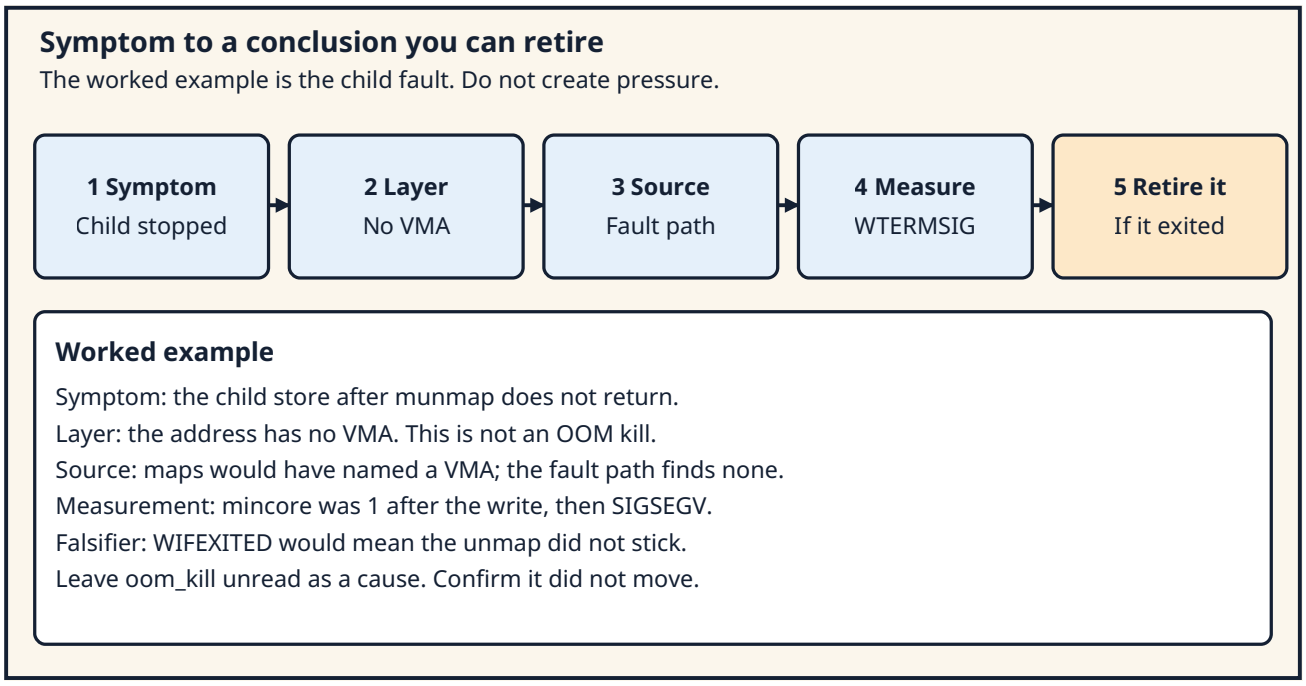


Figure 2. The child `SIGSEGV` is the worked example. The same steps apply to a residency lag or a reclaim counter, and they stop before you create pressure.

12. Mastery: Explain · Draw · Observe · Build

The exercise is `../challenges/day-007-vm-debug-report.sh`.

Explain

- What `show_mem` counted in Linux 0.12, and which `/proc` files now split that curiosity into frames versus one mm.
- Which function prints a maps header, and what the `p` / `s` letter means.
- Why `VmSize` can move while `RssAnon` stays still after `brk`.
- What `mincore` stores for a present PTE and for an anonymous hole.
- Which `getrusage` who-argument includes the child, and what `ru_maxrss` is measured in.
- Why `mm_account_fault` and `task_mem` are different layers.
- Why this child's `SIGSEGV` is a missing VMA and not `out_of_memory`.

Draw

Draw Figure 1 with one concrete triple: `task_mem`, the path `/proc/self/status`, and the `VmRSS` value from your run. Draw Figure 2 for the child fault, with the falsifier "if `WIFEXITED`, the unmap did not stick."

Observe

Run the wrapper once. Record `pressure_induced=no`, any `SKIP` line, both mincore bytes from the child program if you ran it, the wait line, both rusage lines, `VmRSS`, `RssAnon`, one maps line, and `uname -r`. Record `oom_kill` only to confirm the lab left it alone.

Build

Write the one-page report from section 9. End with the sentence you will use when a log dump is offered as a diagnosis: a diagnosis names the layer, the function, the API, the measurement, and the observation that would have retired it.

Now check your understanding

You have finished the theory for this section. Before moving on, use the quick check below to find anything that still feels uncertain, then reinforce the idea with the practical lab when one is provided.

Quick theory check

Answer from memory first. Explanations appear only after you submit your choices.

- Start the interactive theory check
- View quiz source on GitHub

Put it into practice — Vm Debug Report

Format: shell lab

What you will do: set -euo pipefail

Predict the important result first, then run the lab and explain any difference between prediction and observation.

- Open the lab file

Copy and run:

```
bash 13-virtual-memory-faults-and-reclaim/challenges/day-007-vm-debug-report.sh
```

- View lab source on GitHub